

GNN-Based Adaptive Coin Policy Learning for Discrete-Time Quantum Walks

1 What is a GNN Here & How We Use It

Non-traditional vs. standard ANNs. Unlike CNNs/MLPs, GNNs respect *permutation symmetries* of graphs. Our message-passing GNN is *permutation-equivariant*: relabeling nodes relabels outputs in the same way (Thm. in main text).

Why a GNN for ACPL. (1) per-node adaptivity for coins $C_v(t)$, (2) robust cross-graph generalization, (3) compatibility with positional encodings (Laplacian eigenvectors for irregular graphs; normalized coordinates for grids), (4) small receptive fields (2–4 hops) suffice for local control yet can be increased if tasks demand larger context.

Practical choices. Sum aggregation for strict equivariance; 2–3 layers for stability; hidden sizes 64–128; Laplacian PE for irregular graphs; add attention only if beneficial (keeps compute in check).

2 Notation and Symbols

Sets, indices, and basic objects.

$$G = (V, E)$$

Finite graph; $|V|$ vertices, $|E|$ (undirected) edges.

$$u, v, w \in V$$

Vertex indices. $\mathcal{N}(v)$ is the (out-)neighborhood of v .

d_v Degree (number of outgoing ports) at vertex v .

$$a \in \{1, \dots, d_v\}$$

Local *port* (half-edge) index at v .

$$t \in \{0, \dots, T-1\}$$

Discrete time step; T is the *horizon* (episode length).

Graph tensors and features.

A Adjacency (sparse); $\tilde{A} = A + I$ if self-loops are added.

$$\text{edge_index} \in \mathbb{N}^{2 \times |E|}$$

COO list of *directed* arcs (both directions for undirected graphs).

$$X = \{x_v\}_{v \in V} \in \mathbb{R}^{|V| \times F}$$

Node-feature matrix. Each $x_v \in \mathbb{R}^F$ concatenates degree, positional encodings (e.g., Laplacian eigenvectors), and, when available, coordinates/bitstrings (for grids/hypercubes). (*Resolves “Need to define X ”*)

$$\pi(\text{pos}_v)$$

Positional encoding (PE) for node v ; e.g., top- K Laplacian eigenvectors with fixed sign convention or normalized grid coordinates.

Hilbert spaces, states, and measurement.

$$\mathcal{H}_P$$

Position space, $\text{span}\{|v\rangle : v \in V\}$.

$$\mathcal{H}_C(v) \cong \mathbb{C}^{d_v}$$

Local coin space at vertex v .

$\mathcal{H}_C = \bigoplus_{v \in V} \mathcal{H}_C(v)$
Global (arc) coin space.

$\mathcal{H} = \mathcal{H}_P \otimes \mathcal{H}_C$
Total walk space in the *arc basis* $\{|v, a\rangle\}$.

$\psi_t \in \mathcal{H}$
Pure state at time t ; $\|\psi_t\|_2 = 1$.

$\rho_t = |\psi_t\rangle\langle\psi_t|$
Density matrix of the total state.

$\rho_t^{(P)} = \text{Tr}_C(\rho_t)$
Reduced position density. *Purpose:* marginalizes over coin DOFs so tasks that depend only on *position* use $P_t(v) = \langle v | \rho_t^{(P)} | v \rangle$ without measuring the coin. (*Resolves “what’s the purpose of getting the trace?”*)

$P_t \in \Delta^{|V|-1}$
Position probability vector: $P_t(v) = \langle v | \rho_t^{(P)} | v \rangle$.

Unitary dynamics (coins and shift).

$C_v(t) \in U(d_v)$
Local coin at vertex v and time t ; unitary on $\mathcal{H}_C(v)$.

$C_t = \bigoplus_{v \in V} C_v(t)$
Block-diagonal coin on $\mathcal{H}_C$.

S **Flip-flop shift** (permutation unitary) acting on the arc basis:

$$S |v, a\rangle = |u, a'\rangle, \quad \text{where } (u, a') = \text{port_map}(v, a).$$

Matrix-wise, S is a permutation with entries $[S]_{(u,a'),(v,a)} = 1$ iff $(u, a') = \text{port_map}(v, a)$, else 0. This definition works for *irregular degrees* because ports are local to each vertex; S never mixes different port cardinalities, it only swaps matched arc indices. (*Resolves the red note about how S acts with different d_v .*)

$U_t = S C_t$
One-step propagator; $\psi_{t+1} = U_t \psi_t$.

Policy, parameters, and mappings.

π_ω
Learning policy with weights ω (all trainable NN parameters).

$z_v \in \mathbb{R}^H$
GNN node embedding at v .

$\tau_t = t/T$
Normalized time feature.

$s_{v,t} = [z_v; \tau_t; \pi(\text{pos}(v))]$
Node-time feature given to the temporal controller.

$h_{v,t}$
Controller hidden state at node v , time t (or a global state η_t in a shared controller variant).

$\theta_{v,t} \in \mathbb{R}^p$
Coin parameter vector output by the policy head at node v , time t .

$\Phi: \mathbb{R}^p \rightarrow U(d_v)$
Unitary lift mapping parameters to a physical coin:

$$\Phi(\theta_{v,t}) = \begin{cases} R_z(\alpha) R_y(\beta) R_z(\gamma) \in \text{SU}(2), & d_v = 2, \\ \exp(K_{v,t}) \in U(d_v), \text{ with } K_{v,t}^\dagger = -K_{v,t}, & d_v > 2, \end{cases}$$

with $K_{v,t} = i \sum_{k=1}^q \theta_{v,t}^{(k)} G_k$ in an orthonormal Hermitian basis $\{G_k\}$. The Cayley retraction $(I - K_{v,t})(I + K_{v,t})^{-1}$ is an alternative.

$$p = 3 \text{ for } d_v = 2; \quad p = q \text{ for } d_v > 2$$

Rationale: $SU(2)$ is 3-dimensional (Euler ZYZ angles), while a general $U(d_v)$ has d_v^2 real DoF; we choose a generator basis of size $q \leq d_v^2$ for efficiency (often $q = d_v^2$). (*Resolves the red note on p .*)

$\varphi_{v,t}$

(Optional alias) If avoiding overload, use $\varphi_{v,t}$ for coin output parameters and reserve θ for policy weights; in this paper we fix ω for weights and $\theta_{v,t}$ for coin parameters. (*Resolves the red “ φ and θ connection”.*)

Losses, metrics, and evaluation.

$f(\psi_T)$

Task return (to be maximized); $L = -f(\psi_T)$ minimized in training.

$$P_\Omega(T) = \sum_{m \in \Omega} P_T(m)$$

Success probability on a target set Ω (state transfer/search).

$$TV(\rho_T^{(P)}, \rho^*)$$

Total-variation mixing error:

$$TV = \frac{1}{2} \sum_{v \in V} |P_T(v) - \rho^*(v)|, \quad \rho^*(v) = \frac{1}{|V|} \text{ (uniform).}$$

CVaR $_\alpha$

Conditional Value at Risk of a random success probability $P_T(j^*; \xi)$ under disorder ξ : $CVaR_\alpha = \frac{1}{\alpha} \int_0^\alpha Q_u du$ with Q_u the u -quantile.

$\mathcal{J}(\omega)$

Generalization value $\mathbb{E}_{\text{episode} \sim \mathcal{D}}[f(\psi_T)]$.

GNN internals (message passing).

$$h_v^{(0)} = x_v$$

Layer-0 node state (input features).

$$m_{u \rightarrow v}^{(\ell)}$$

Message from u to v at layer ℓ .

$$\bar{m}_v^{(\ell)} = \sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}^{(\ell)}$$

Permutation-invariant aggregation.

$$h_v^{(\ell+1)} = U^{(\ell)}(h_v^{(\ell)}, \bar{m}_v^{(\ell)})$$

Update; $z_v = h_v^{(L)}$.

Complexity symbols (used in scaling discussion).

H GNN hidden width; L GNN depth (layers).

$\bar{d}$ Typical coin dimension / degree (e.g., 2 on lines, 4 on grids).

Cost

Per step: $O(|E| \cdot H)$ (message passing) + $O(|V| \cdot \bar{d}^2)$ (assembling/applying C_t); total $O(T(|E|H + |V|\bar{d}^2))$. (*Matches the analysis in the text; see [28, 29] for retraction/exponential Jacobians and [30, 31] for GNN kernel costs.*)

Linear-algebra conventions.

$\langle x, y \rangle$

Complex inner product; $\text{Re}(\cdot)$ real part.

$\oplus$ Direct sum (block-diagonal concatenation).

$\otimes$ Tensor (Kronecker) product.

Tr, Tr_C

Trace and partial trace over coin space.

$\text{blkdiag}(\cdot)$

Block-diagonal assembly.

Noise/disorder.

ξ Randomness (e.g., static edge phases, coin dephasing) drawn from law $\mathcal{D}$.

$\mathbb{E}_\xi[\cdot]$

Expectation over disorder used in robust-objective training/evaluation.

Horizon T (recap).

T Number of control actions and walk steps in one episode; larger T enables richer interference but increases compute linearly. For state transfer over graph distance D , $T = \Theta(D)$ (ballistic). For spatial search on symmetric graphs, successful schedules often scale as $T = \tilde{\Theta}(\sqrt{|V|})$.

3 Primitive (Exact Object of Study)

Adaptive Coin Policy Learning (ACPL) for DTQWs. Given a graph $G = (V, E)$, horizon T , and an objective f (e.g., hit a marked node, accelerate mixing, or maximize transport), learn a policy π_θ that outputs coin operators $C_v(t) \in U(d_v)$ at each step $t \in \{0, \dots, T-1\}$ (optionally per vertex v), producing the final state ψ_T that maximizes $f(\psi_T)$.

Inputs. Graph G , initial state ψ_0 , horizon T , and task specification (target set Ω or desired distribution ρ^*).

Outputs A learned mapping π_θ that yields coin parameters for $C_v(t)$.

Walk dynamics Let $\mathcal{H}_P = \text{span}\{|v\rangle : v \in V\}$ and $\mathcal{H}_C = \bigoplus_{v \in V} \mathbb{C}^{d_v}$. With flip-flop shift S and block-diagonal coin $C_t = \bigoplus_{v \in V} C_v(t)$, one step is $U_t = SC_t$ and the walk evolves as $\psi_{t+1} = U_t \psi_t$ [1, 2].

Objective Maximize $f(\psi_T)$, e.g. terminal success probability at Ω , fast mixing to uniform, expected hitting time minimization, or transport to a target vertex.

4 Why this Scope (with GNNs)

The coin operator is the primary control knob in coined DTQWs [3]. ACPL makes ML the engine: π_θ is learned, adaptive in time (and optionally in space), and trained over distributions of graphs/tasks to generalize. A *graph neural network* (GNN) encoder provides permutation-equivariant node embeddings, enabling per-node coin adaptivity and cross-graph transfer. The scope is concrete (finishable) and extensible (higher-degree graphs, noise, multi-objective control).

5 Concrete Problems (All Four)

We will do all four, with shared simulator/policy/training infrastructure:

1. **State transfer on a line/cycle** ($d = 2$): maximize probability at target j^* at step T . Baselines: Hadamard, best static global $\text{SU}(2)$ coin [3].
2. **Marked-node search** (2D grid, hypercube): maximize success probability at step T given an oracle mark. Baselines: SKW on the hypercube [4], AKR on the grid [5, 6, 7].
3. **Fast mixing** (2D lattice): minimize $\|\rho_T - \text{Uniform}\|_{\text{TV}}$. Compare to mixing bounds and recent analyses [8, 9, 10, 11].
4. **Robust transport under disorder** (1D): maximize arrival probability under static/dynamic disorder; benchmark (de)localization thresholds [17, 18, 19].

6 Extensive ML Usage (GNN + Temporal Controller)

6.1 GNN-based Policy Architecture

Let A be the adjacency and $X = \{x_v\}_{v \in V}$ node features (degree, optional positional encodings). We compute L layers of message passing (MPNN/GCN/GAT-style [30, 31, 32]):

$$h_v^{(0)} = x_v, \quad (1)$$

$$m_v^{(\ell)} = \sum_{u \in \mathcal{N}(v)} \phi_m(h_v^{(\ell)}, h_u^{(\ell)}, e_{uv}), \quad (2)$$

$$h_v^{(\ell+1)} = \phi_u(h_v^{(\ell)}, m_v^{(\ell)}), \quad \ell = 0, \dots, L-1, \quad (3)$$

with sum aggregator for permutation-equivariance. Let $z_v := h_v^{(L)}$.

Time and position conditioning. We form $s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)]$ where $\tau_t = t/T$ and $\pi(\text{pos}_v)$ is an optional positional encoding (e.g., top- k Laplacian eigenvectors/LapPE [33] or normalized coordinates on grids).

Controller and coin parameter head. A GRU/Transformer reads $s_{v,t}$ and produces coin parameters:

$$h_{v,t}^{\text{tmp}}, \eta_{t+1} = \text{GRU}(s_{v,t}, \eta_t), \quad (4)$$

$$\theta_{v,t} = W h_{v,t}^{\text{tmp}} + b. \quad (5)$$

Map $\theta_{v,t}$ to a unitary coin:

$$\text{Degree } 2: \quad C_v(t) = R_z(\alpha_{v,t}) R_y(\beta_{v,t}) R_z(\gamma_{v,t}) \in \text{SU}(2), \quad (6)$$

$$\text{Degree } d > 2: \quad C_v(t) = \exp(K_{v,t}), \quad K_{v,t}^\dagger = -K_{v,t}, \quad (7)$$

or Cayley/QR retractions for efficiency and stability.

6.2 Mathematical Note: Permutation-Equivariance of Message Passing

Let π be a permutation of V , represented by permutation matrix P so that $X' = PX$ and $A' = PAP^\top$ reindex the graph.

Theorem 1 (Equivariance). *Consider a message-passing GNN with sum aggregation and shared parameters (ϕ_m, ϕ_u) over nodes and edges. If $H^{(\ell)} = \{h_v^{(\ell)}\}$ are the representations at layer ℓ for (A, X) and $H'^{(\ell)}$ those for (A', X') , then*

$$H'^{(\ell)} = PH^{(\ell)} \quad \text{for all } \ell = 0, \dots, L.$$

Consequently $Z' = PZ$ for the node embeddings $Z = \{z_v\}$, and any per-node policy $\Psi(z_v, \tau_t)$ satisfies $\Psi'(Pz, \tau_t) = P\Psi(z, \tau_t)$.

Proof. The claim holds trivially for $\ell = 0$ since $H'^{(0)} = X' = PX = PH^{(0)}$. Assume $H'^{(\ell)} = PH^{(\ell)}$. Under permutation, neighborhoods map bijectively: $\mathcal{N}'(Pv) = \{Pu : u \in \mathcal{N}(v)\}$. With shared parameters and sum aggregation,

$$m_{Pv}^{(\ell)} = \sum_{u \in \mathcal{N}(v)} \phi_m(h_{Pv}^{(\ell)}, h_{Pu}^{(\ell)}, e_{Pv, Pu}) = \sum_{u \in \mathcal{N}(v)} \phi_m((Ph_v^{(\ell)}), (Ph_u^{(\ell)}), e_{v,u}),$$

which equals $Ph_v^{(\ell)}$ since ϕ_m is applied coordinatewise with tied weights. Applying the shared update ϕ_u gives $h_{Pv}^{(\ell+1)} = Ph_v^{(\ell+1)}$. By induction, the result holds for all layers; setting $z_v = h_v^{(L)}$ yields $Z' = PZ$. The policy head is shared per node, hence $\Psi'(Pz, \tau_t) = P\Psi(z, \tau_t)$. $\square$

Remark. Equivariance guarantees that ACPL respects graph isomorphisms: training on (G, X) yields policies that transfer to relabeled copies and, with domain randomization, to unseen graphs that share local structure.

Note. The above proof is a self-contained write-up of a standard result; see, e.g., [30, 31, 32, 38, 35] for formal treatments of permutation-(in)equivariance in message-passing GNNs.

6.3 Differentiable Control (Backprop through the Simulator)

Loss. $\mathcal{L}(\theta) = -f(\psi_T(\theta))$. Train with autodiff (JAX/PyTorch/PennyLane) [25, 26, 27].

Adjoint recursion and gradients. Let $L = f(\psi_T)$ be real. With

$$\lambda_T = \partial_{\psi_T} L, \quad \lambda_t = U_t^\dagger \lambda_{t+1} \quad (t = T-1, \dots, 0),$$

the parameter gradient is

$$\frac{\partial L}{\partial \theta} = 2 \operatorname{Re} \sum_{t=0}^{T-1} \left\langle \lambda_{t+1}, S \frac{\partial C_t}{\partial \theta} \psi_t \right\rangle.$$

For $C = \exp(K)$, $d \exp_K[H] = \int_0^1 e^{(1-s)K} H e^{sK} ds$; autodiff frameworks implement this efficiently.

6.4 Reinforcement Learning (RL)

When non-differentiable oracles/constraints are involved, treat the simulator as an environment with episodic reward $r_T = f(\psi_T)$ and learn π_θ via PPO/A2C [22, 23, 24].

6.5 Meta-Learning, Positional Encodings, and Robustness

Meta-learning. Train over graph distributions (sizes/topologies) for zero-shot transfer; optionally use few-shot fine-tuning.

Positional encodings (PE). For irregular graphs use LapPE: if $L = D - A$ has eigenpairs (λ_k, ϕ_k) , take $\pi(\text{pos}_v) = (\phi_1(v), \dots, \phi_K(v))$ [33]. On grids, use normalized coordinates.

Robustness. Inject coin dephasing, position dephasing, and edge disorder during training; optimize expected or CVaR-style objectives [?, 18].

7 Tasks and Objectives

Reduced position density $\rho_t = \operatorname{Tr}_C(|\psi_t\rangle\langle\psi_t|)$.

State transfer to target j^* : maximize $P_{j^*}(T) = \langle j^* | \rho_T | j^* \rangle$, $\mathcal{L} = -P_{j^*}(T)$.

Search for marked set M : with oracle $O_M = I - 2 \sum_{m \in M} |m\rangle\langle m| \otimes I$, maximize $\sum_{m \in M} \langle m | \rho_T | m \rangle$ [4, 5, 6].

Fast mixing to uniform ρ^* : minimize $\|\rho_T - \rho^*\|_{\text{TV}} = \frac{1}{2} \sum_{v \in V} |P_v(T) - 1/V|$ [8, 10].

Robust transport under disorder $\xi \sim \mathcal{D}$: maximize $\mathbb{E}_\xi[P_{j^*}(T; \xi)]$ or CVaR_α .

8 Baselines, Metrics, and Evaluation

Primary metrics Terminal success probability $P_\Omega(T)$, expected hitting time (via repeated-measure definitions [20]), TV mixing error, transfer fidelity, robustness curves, generalization gaps, and compute/sample efficiency.

Baselines

- **Fixed coins:** Hadamard/Grover; best static global coin via grid or Bayesian optimization [3].
- **Global time-varying coin,** no spatial adaptivity.
- **Analytical schedules:** SKW (hypercube) [4]; AKR (grid) and variants [5, 6, 7].
- **QWNN-style learned coins** as prior art for learnability [21].

Generalization Zero-shot to larger graphs (e.g., lines 64–128 trained $\rightarrow$ test at 256); cross-topology tests (train on grids of $L \in [32, 64]$, test on $L = 96$); few-shot adaptation.

9 Optimizing the Coin Operator with a GNN Policy

9.1 Which coin do we use? (Project-specific choice)

We adopt the standard *coined DTQW* with vertex-local coins:

$$C_t = \bigoplus_{v \in V} C_v(t), \quad C_v(t) \in U(d_v),$$

where d_v is the degree of vertex v . We use two parameterizations, chosen by graph degree:

$$\textbf{Degree 2 (lines/cycles)} : C_v(t) = R_z(\alpha_{v,t}) R_y(\beta_{v,t}) R_z(\gamma_{v,t}) \in \text{SU}(2), \quad (8)$$

$$\textbf{Degree } d_v > 2 \text{ (grids, hypercubes, irregular)} : C_v(t) = \exp(K_{v,t}), \quad K_{v,t}^\dagger = -K_{v,t} \in \mathfrak{u}(d_v). \quad (9)$$

The exp parameterization *enforces unitarity by construction* and is numerically stable; we optionally use the Cayley retraction $C_v(t) = (I - K_{v,t})(I + K_{v,t})^{-1}$ for speed. These choices cover all four tasks and both regular and irregular graphs.

9.2 What does “optimizing the coin” mean?

Given a task objective f computed from the terminal state ψ_T (e.g., state-transfer success, search success, TV mixing error, robust transport), we seek a *policy* that *chooses the coins*:

$$\max_{\theta} f(\psi_T(\theta)) \quad \text{s.t.} \quad \psi_{t+1} = U_t(\theta) \psi_t, \quad U_t(\theta) = S \left(\bigoplus_v C_v(\theta_{v,t}) \right), \quad t = 0, \dots, T-1.$$

The variables being optimized are *not* the raw unitary matrices directly, but the *policy parameters* θ of a function π_θ that produces the per-node, per-time coin parameters $\{\theta_{v,t}\}$.

Why not optimize $C_v(t)$ directly? Directly optimizing free unitaries $C_v(t) \in U(d_v)$ is (i) over-parameterized, (ii) ignores graph symmetries, and (iii) does not generalize to new graphs. A *graph neural network (GNN) policy* produces *structured* coin schedules that (a) are *permutation-equivariant* and (b) *share* parameters across nodes/graphs, enabling cross-graph generalization.

9.3 Explicit learning policy and its map to coins

The policy is a composition of a GNN encoder, a temporal controller, and a coin head:

$$\underbrace{(G, X) \xrightarrow{\Phi_{\text{GNN}}} \{z_v \in \mathbb{R}^H\}}_{\text{permutation-equivariant node embeddings}} \implies s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)] \xrightarrow{\text{GRU/Transformer}} h_{v,t} \xrightarrow{W \cdot + b} \theta_{v,t},$$

where $\tau_t = t/T$ and $\pi(\text{pos}_v)$ is a positional encoding (e.g., Laplacian PE). The *coin map* Φ converts $\theta_{v,t}$ into a unitary:

$$C_v(t) = \Phi(\theta_{v,t}) = \begin{cases} R_z(\alpha_{v,t}) R_y(\beta_{v,t}) R_z(\gamma_{v,t}) \in \text{SU}(2), & d_v = 2, \\ \exp(K_{v,t}(\theta_{v,t})) \in U(d_v), & d_v > 2. \end{cases}$$

Thus the GNN *does not* output $C_v(t)$ directly; it outputs *compact parameters* $\theta_{v,t}$ that are then lifted to $U(d_v)$.

9.4 Differentiable training objective (backprop-through-time)

For differentiable tasks we minimize $L(\theta) = -f(\psi_T(\theta))$ by backpropagating through the unitary evolution:

$$\psi_{t+1} = U_t(\theta) \psi_t, \quad \lambda_T = \partial_{\psi_T} L, \quad \lambda_t = U_t(\theta)^\dagger \lambda_{t+1},$$

$$\frac{\partial L}{\partial \theta} = 2 \text{Re} \sum_{t=0}^{T-1} \left\langle \lambda_{t+1}, S \frac{\partial C_t}{\partial \theta} \psi_t \right\rangle, \quad C_t = \bigoplus_v C_v(\theta_{v,t}).$$

Autodiff (PyTorch/JAX) computes $\partial C_t / \partial \theta$ automatically. For $C = \exp(K)$ we rely on the Fréchet derivative $d \exp_K[H] = \int_0^1 e^{(1-s)K} H e^{sK} ds$, implemented efficiently by modern libraries. We add regularizers such as $\sum_{v,t} \|K_{v,t} - K_{v,t-1}\|_F^2$ (temporal smoothness) and $\sum_{v,t} \|K_{v,t}\|_F^2$ (magnitude control).

9.5 RL training when gradients are not available

If an oracle/measurement introduces non-differentiability, we treat the simulator as an episodic environment with terminal reward $r_T = f(\psi_T)$ and learn π_θ with PPO/A2C. The action is $\theta_{v,t}$; the environment step applies $C_v(t) = \Phi(\theta_{v,t})$ and $U_t = S C_t$.

9.6 What does the GNN “learn” about the coin?

At a given node v and time t , the GNN embedding z_v summarizes *local graph context* (degree, motifs, spectral cues) in a permutation-equivariant way. The temporal controller conditions on t/T to output $\theta_{v,t}$. During training, gradients flowing from $f(\psi_T)$ push $\theta_{v,t}$ so that $C_v(t) = \Phi(\theta_{v,t})$ orchestrates *constructive/destructive interference* patterns that make f large (e.g., concentration at a target vertex for state transfer, near-uniform spread for mixing, focus on marked nodes for search, or resilient transport under disorder). In short:

“Optimizing the coin” $\equiv$ learning $\theta_{v,t} = \pi_\theta(v, t; G, X)$ so that $C_v(t) = \Phi(\theta_{v,t})$ yields favorable interference.

9.7 Identifiability, gauge, and symmetry

Local rephasing/relabeling of ports at a vertex conjugates $C_v(t)$ by diagonal/permutation matrices and changes how S pairs ports. Many of these transformations are *gauge* (unobservable at the level of P_T). To avoid ambiguity we fix a canonical port ordering and (optionally) pin one global phase per $C_v(t)$.

9.8 Algorithm (training loop; executable pseudocode)

Listing 1: Backprop-through-time training for coin optimization

```
psi = psi0
h = zeros_like(GRU_state)
for t in range(T):
    # 1) Policy: per-node parameters from GNN + temporal controller
    theta_t, h = policy(G, X, pos_enc, t/T, h) # -> [|V|, p]
    # 2) Map to unitaries (enforce U(d_v))
    C_blocks = coin_param.to_unitary(theta_t) # list of C_v(t)
    # 3) One step of QW
    psi = sim.step(psi, C_blocks) # psi <- S @ blkdiag(C) @ psi
rho_T = partial_trace_coin(psi)
loss = task.loss(psi, rho_T) # (-f): state xfer, search, mixing, robust
loss.backward(); opt.step(); opt.zero_grad() # or PPO update if non-diff
```

9.9 Task-specific losses (how “best coin” is measured)

$$\text{State transfer: } \mathcal{L} = -\langle j^* | \rho_T | j^* \rangle. \quad (10)$$

$$\text{Search: } \mathcal{L} = -\sum_{m \in M} \langle m | \rho_T | m \rangle. \quad (11)$$

$$\text{Fast mixing: } \mathcal{L} = \frac{1}{2} \sum_{v \in V} |P_T(v) - 1/|V||. \quad (12)$$

$$\text{Robust transport: } \mathcal{L} = -\mathbb{E}_{\xi \sim \mathcal{D}} [\langle j^* | \rho_T(\xi) | j^* \rangle] \text{ (or CVaR)}. \quad (13)$$

Minimizing $\mathcal{L}$ yields the “most optimized” coin schedule for the chosen task and horizon T .

9.10 Complexity and practical settings

Per time step: $O(|E|H)$ for GNN message passing + $O(|V|\bar{d}^2)$ to build/apply C_t (with $\bar{d}$ a typical degree); total $O(T(|E|H + |V|\bar{d}^2))$. We use small depths (GNN layers $L \in \{2, 3\}$), hidden sizes $H \in [64, 128]$, and horizons $T \in [50, 200]$. Regularizers and curriculum in T stabilize training.

9.11 Summary (one line)

We *optimize the coin* by learning a permutation-equivariant, time-conditioned policy π_θ (GNN + temporal controller) that outputs compact parameters $\theta_{v,t}$; a unitary map Φ lifts these to $C_v(t) \in U(d_v)$, which drive the DTQW so as to *maximize* the task objective $f(\psi_T)$.

10 Inputs and Outputs (Explicit)

Inputs.

- Graph $G = (V, E)$ with degrees $\{d_v\}_{v \in V}$; oriented-arc structure for the flip-flop shift S .
- Horizon $T \in \mathbb{N}$.
- Initial quantum state $\psi_0 \in \mathcal{H}_P \otimes \mathcal{H}_C$, typically localized or a simple superposition.
- Task specification f and any task parameters (target set Ω , target vertex j^* , target distribution ρ^* , oracle O_M , disorder law $\mathcal{D}$).
- Optional node features x_v (degree, Laplacian positional encodings, coordinates on grids) and time feature $\tau_t = t/T$.

Outputs.

- A learned **policy** π_θ that produces coin parameters for each node/time: $\theta_{v,t} = \pi_\theta(v, t; G, X)$. **Need to define X**
- A sequence of unitary, vertex-local **coin operators** $C_v(t) \in U(d_v)$ ($C_v(t)$ was constructed from $\theta_{v,t}$), hence $C_t = \bigoplus_{v \in V} C_v(t)$ and step operators $U_t = S C_t$.
- The terminal quantum state ψ_T and reduced position density $\rho_T = \text{Tr}_C(|\psi_T\rangle\langle\psi_T|)$. **(what's the purpose of getting the trace?)**
- Task metrics (success probability, TV = ??? mixing error, expected hitting time, robustness curves), plus generalization scores.

Explicit definition of X (used in $\pi_\theta(v, t; G, X)$). Let F be the node–feature dimension. We define the node–feature matrix

$$X = \{x_v\}_{v \in V} \in \mathbb{R}^{|V| \times F}, \quad x_v = \left[\underbrace{\tilde{d}_v}_{\text{degree}} ; \underbrace{\pi_{\text{pos}}(v)}_{\text{positional enc.}} ; \underbrace{\chi_{\text{geom}}(v)}_{\text{coordinates/bitstrings}} ; \underbrace{\chi_{\text{role}}(v)}_{\text{(optional) role/structural}} \right].$$

- **Degree (scalar):** $\tilde{d}_v = d_v / \max_{u \in V} d_u$ (normalized degree).
- **Positional encodings (PE):** $\pi_{\text{pos}}(v) \in \mathbb{R}^K$. By default we use *Laplacian positional encodings (LapPE)* on irregular graphs: let $L = D - A$ (combinatorial Laplacian) or $\mathcal{L} = I - D^{-1/2} A D^{-1/2}$ (normalized); take the K smallest nontrivial eigenpairs (λ_k, ϕ_k) and set

$$\pi_{\text{pos}}(v) = (\phi_1(v), \dots, \phi_K(v)),$$

with a deterministic sign convention per eigenvector (e.g., flip so that $\arg \max_u |\phi_k(u)|$ is positive) to avoid ambiguity.

- **Geometric encodings $\chi_{\text{geom}}(v)$ (graph-family specific):**
 - *Grids* ($L \times L$): $\chi_{\text{geom}}(v) = (x/L, y/L) \in \mathbb{R}^2$ with $x, y \in \{0, \dots, L-1\}$.
 - *Hypercube* ($\{0, 1\}^n$): $\chi_{\text{geom}}(v) = \mathbf{b}(v) \in \{0, 1\}^n$ (the node's bitstring), optionally with Hamming weight $w(v)$ appended and normalized by n .
 - *Lines/cycles*: $\chi_{\text{geom}}(v) = x/(N-1) \in \mathbb{R}$ (or $(\sin, \cos)$ positional codes on cycles).
 - *Irregular graphs*: omit unless native coordinates exist (LapPE already provides positional structure).
- **(Optional) structural/role features $\chi_{\text{role}}(v)$:** e.g., one-hot boundary flags, shortest-path distance to a designated source/target (normalized), or simple centralities if precomputable; these are task-agnostic and fixed per episode.

Normalization. All continuous features are min–max scaled to $[0, 1]$ (per episode) or standardized (zero mean, unit variance) before feeding the GNN. Discrete features (bitstrings) are left as $\{0, 1\}$.

Shapes and typical choices. For the experiments in this paper we use:

$$\begin{aligned} \text{Line/cycle:} \quad & x_v = [\tilde{d}_v; x/(N-1)] \in \mathbb{R}^2. \\ \text{Grid:} \quad & x_v = [\tilde{d}_v; x/L; y/L] \in \mathbb{R}^3. \\ \text{Hypercube:} \quad & x_v = [\tilde{d}_v; \mathbf{b}(v)] \in \mathbb{R}^{1+n}. \\ \text{Irregular:} \quad & x_v = [\tilde{d}_v; \pi_{\text{pos}}(v)] \in \mathbb{R}^{1+K}, \quad K \in \{8, 16\}. \end{aligned}$$

Note on $\pi(\text{pos}_v)$ in the controller input. We sometimes *also* pass $\pi(\text{pos}_v)$ to the temporal controller as a separate term in $s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)]$ (Sec. 16). This mild redundancy is intentional: it allows (i) clean ablations (*No-PE* vs *PE-in-X*), and (ii) the controller to access positional cues even if the encoder bottleneck z_v is narrow. In code, this is a one-line switch.

11 Horizon T : Definition and Practical Choice

Definition. The horizon $T \in \mathbb{N}$ is the number of discrete time steps in one episode. At each step $t = 0, \dots, T-1$ we apply a vertex-local coin $C_t = \bigoplus_v C_v(t)$ followed by the shift S :

$$\psi_{t+1} = U_t \psi_t, \quad U_t = S C_t.$$

Thus T is (i) the episode length and (ii) the number of control actions emitted by the policy π_θ . We provide the controller with normalized time $\tau_t = t/T$ so that policies can generalize across nearby horizons.

Role in control. A larger T increases the *control bandwidth* (more coin updates) and allows richer interference patterns, but also increases computation and may complicate optimization (longer backprop chains). Our training cost scales linearly with T :

$$\text{cost} = O(T(|E|H + |V|\bar{d}^2)),$$

where H is the GNN width and $\bar{d}$ a typical degree.

11.1 Computational Cost: Per-Step Decomposition and Scaling with T

We justify the synthesis cost $= \tilde{O}(T(|E|H + |V|\bar{d}^2))$ from first principles by accounting for the kernels executed at each time step.

Per-step operations. At time t the loop performs:

1. **GNN message passing (encode z_v).** One layer with sum aggregation is essentially a sparse–dense multiply $H^{(\ell+1)} = \sigma(\hat{A}H^{(\ell)}W^{(\ell)})$, implemented as SPMM plus small dense ops. With hidden width H ,

$$\text{cost}_{\text{GNN per layer}} = \Theta(|E| \cdot H).$$

With L small (2–3) layers, this is $\tilde{O}(|E|H)$ per step [31, 30].

2. **Temporal controller + head (produce $\theta_{v,t}$).** A GRU/Transformer over node–time inputs adds $\tilde{O}(|V|H)$ work, dominated by the GNN term on sparse graphs; we fold it into $\tilde{O}(|E|H)$.

3. **Parameter $\rightarrow$ local coin $C_v(t)$.** For degree $d_v=2$, building $R_z(\alpha)R_y(\beta)R_z(\gamma)$ is $O(1)$. For $d_v>2$, mapping θ to $C_v(t) \in U(d_v)$ via the exponential $C_v = \exp(K_v)$ or the *Cayley retraction* $C_v = (I - K_v)(I + K_v)^{-1}$ is a tiny dense operation per node. With fixed small degrees (grids $d_v=4$, hypercubes $d_v=n \leq 10$), this contributes

$$\tilde{O}\left(\sum_v d_v^2\right) = \tilde{O}(|V|\bar{d}^2),$$

where $\bar{d}$ is a typical degree [29, 28].

4. **Apply coin and shift.** Applying the block-diagonal coin $C_t = \bigoplus_v C_v(t)$ to the state is $O(\sum_v d_v^2) = \tilde{O}(|V|\bar{d}^2)$ (tiny dense multiplies per node). The flip–flop shift S is an index permutation over arcs, $O(|V|\bar{d})$, negligible vs. $|V|\bar{d}^2$ for $\bar{d} \geq 2$.

Per-step total and attention variant. Summing the components,

$$\boxed{\text{cost per step} = \tilde{O}(|E|H) + \tilde{O}(|V|\bar{d}^2)}.$$

Graph attention layers (GAT) add edge-wise score/softmax but remain edge-linear, preserving the $\tilde{O}(|E|H)$ scaling with a larger constant [32].

Scaling with the horizon. The loop executes these operations for $t = 0, \dots, T-1$, so runtime adds across steps:

$$\boxed{\text{total cost} = \tilde{O}\left(T(|E|H + |V|\bar{d}^2)\right)}.$$

Backpropagation-through-time doubles constants (forward+backward) but keeps the dependence *linear in T* . Memory without checkpointing is $O(T(|V|H + |E|))$ for stored activations plus $O(|V|\bar{d})$ for the quantum state.

When constants matter. Exact eigendecompositions for $\exp(K_v)$ scale as $O(d_v^3)$ per node; with small fixed d_v this is still effectively constant. Using Cayley/structured parameterizations keeps the stated bound tight [29, 28]. Attention heads increase constants but not the big- O [32].

Physical scaling heuristics. DTQWs exhibit ballistic spreading (distance $\propto T$), so for state transfer over graph distance D one typically sets $T = \Theta(D)$. For canonical spatial search on symmetric graphs, successful schedules scale as $T = \tilde{\Theta}(\sqrt{|V|})$. Mixing targets are assessed by plotting total-variation error versus T .

Practices in this work. We use a curriculum in T (e.g., $64 \rightarrow 96 \rightarrow 128$), gradient clipping, and temporal smoothness regularization on coin parameters. For task-specific choices:

- **State transfer:** pick T proportional to the source–target distance.
- **Search (grid/hypercube):** pick $T = c\sqrt{|V|}$ with c tuned on validation.
- **Fast mixing:** sweep a small grid of T and report TV error vs. T .
- **Robust transport:** reuse transfer T ; enforce robustness via expectation/CVaR over disorder, not via larger T .

Optional free-time variant. One can also optimize T by introducing a stopping policy or a penalty λT and minimizing $-f(\psi_T) + \lambda T$. In this paper we adopt fixed, task-specific horizons for simplicity and comparability.

12 Shift S on irregular graphs (arc/port model).

Let $\mathcal{A} = \{(v, a) : v \in V, a \in \{1, \dots, d_v\}\}$ be the set of *oriented ports* (half-edges). Define a fixed bijection (index map)

$$\iota : \mathcal{A} \longrightarrow \{1, \dots, D\}, \quad D := \sum_{v \in V} d_v \quad (= 2|E| \text{ for simple undirected graphs}).$$

For each port (v, a) , let $\text{port_map}(v, a) = (u, a')$ be the *flip-flop pairing* across the unique edge that exits v via port a and re-enters u via its return port a' . This map is an *involution*, i.e., $\text{port_map}(\text{port_map}(v, a)) = (v, a)$.

The shift S is then the $D \times D$ *permutation unitary* defined by

$$S |v, a\rangle = |\text{port_map}(v, a)\rangle \iff S_{\iota(u, a'), \iota(v, a)} = \begin{cases} 1, & \text{if } (u, a') = \text{port_map}(v, a), \\ 0, & \text{otherwise.} \end{cases}$$

By construction $S^\dagger = S^{-1} = S$ and $S^2 = I$. *Crucially*, S never mixes subspaces of different local dimensions d_v ; it only *permutes* coordinates in the global arc basis. This is why S is well-defined on irregular graphs, while the “moving-shift” variant that retains a global direction label requires regularity.

Why this works with different degrees. Coins act locally as $C_t = \bigoplus_v C_v(t)$ on the direct sum $\bigoplus_v \mathbb{C}^{d_v}$ (one copy per vertex). The subsequent S simply *reindexes* those local amplitudes onto the neighbor’s ports via the involution port_map . No padding or dimension alignment is needed across vertices.

Tiny irregular example. Consider the path 1–2–3 with degrees $(d_1, d_2, d_3) = (1, 2, 1)$. Let the arc basis (in order) be

$$\{ |1 \rightarrow 2\rangle, |2 \rightarrow 1\rangle, |2 \rightarrow 3\rangle, |3 \rightarrow 2\rangle \},$$

so $D = 1 + 2 + 1 = 4$. The flip-flop pairings are $|1 \rightarrow 2\rangle \leftrightarrow |2 \rightarrow 1\rangle$ and $|2 \rightarrow 3\rangle \leftrightarrow |3 \rightarrow 2\rangle$, hence

$$S = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad S^2 = I, \quad S^\dagger = S.$$

Even though $d_1 \neq d_2 \neq d_3$, S is just a permutation on the $D = 4$ arc coordinates.

Directed graphs and self-loops. For directed graphs, define `port_map` to pair each *outgoing* arc (v, a_{out}) with the corresponding *incoming* arc (u, a_{in}) according to your chosen arc indexing; S remains a permutation unitary. Self-loops correspond to pairings within the same vertex (possibly different ports); degree-1 vertices yield a trivial 1×1 phase coin followed by a deterministic swap under S .

Implementation note. In code, store `port_map[v, a] = (u, a')` and a contiguous index map `idx = iota(v, a)`. Applying S is an *index permutation* (gather/scatter) of the state vector; you never form a dense matrix. Coins remain block-diagonal (one $d_v \times d_v$ unitary per vertex), so one step is

$$\psi \leftarrow S (\text{blkdiag}(C_v) \psi),$$

with S realized as a permutation of entries.

12.1 Why the Partial Trace over the Coin? (Operational Role)

What we observe. Our objectives (state transfer/search/mixing/robust transport) depend only on the *position* of the walker at the end of the episode, not on the internal coin state. The joint Hilbert space is

$$\mathcal{H} = \mathcal{H}_P \otimes \mathcal{H}_C, \quad \rho_T = |\psi_T\rangle\langle\psi_T|.$$

If we measure *only position*, the relevant projectors are $\Pi_v = |v\rangle\langle v| \otimes I_C$ and the Born rule gives

$$P_T(v) = \text{Tr}[\Pi_v \rho_T] = \text{Tr}[|v\rangle\langle v| \rho_T^{(P)}] = \langle v | \rho_T^{(P)} | v \rangle,$$

where the **reduced position density** is defined by the partial trace over the coin:

$$\rho_T^{(P)} = \text{Tr}_C(\rho_T).$$

Thus $\rho_T^{(P)}$ is the unique object that reproduces *all* statistics of observables of the form $A_P \otimes I_C$:

$$\mathbb{E}[A_P] = \text{Tr}[(A_P \otimes I_C) \rho_T] = \text{Tr}[A_P \rho_T^{(P)}].$$

Concrete formula and fast computation (arc basis). In the arc (port) basis $\{|v, a\rangle\}$ with $a = 1, \dots, d_v$,

$$\rho_T^{(P)}(v, v') = \sum_{a=1}^{d_v} \langle v, a | \rho_T | v', a \rangle, \quad P_T(v) = \rho_T^{(P)}(v, v) = \sum_{a=1}^{d_v} |\psi_T(v, a)|^2.$$

Implementation note: in code, reshape ψ_T as `[position, coin]` and compute `P_T = (psi.abs()**2).sum(axis=coin)`; no explicit ρ_T is required.

Why we need it for our tasks. All evaluation metrics use P_T (or $\rho_T^{(P)}$):

- **State transfer/search:** $P_T(j^*)$ or $\sum_{m \in M} P_T(m)$.
- **Fast mixing:** TV distance $\frac{1}{2} \sum_v |P_T(v) - 1/|V||$ to the uniform target.
- **Robust transport:** $\mathbb{E}_\xi[P_T(j^*; \xi)]$ (and CVaR) under disorder ξ .

Using $\rho_T^{(P)}$ avoids any dependence on arbitrary coin measurements and prevents backaction during training/evaluation.

Conceptual and practical benefits.

1. **Operational correctness:** exact position-only statistics via Born’s rule.
2. **Gauge/basis invariance:** local rephases/relabelings in the coin leave $\rho_T^{(P)}$ unchanged.
3. **No backaction:** we compute probabilities without measuring the coin.
4. **Efficiency:** $P_T(v) = \sum_a |\psi_T(v, a)|^2$ is a simple reduction over the coin axis.
5. **Diagnostics (optional):** $S(\rho_T^{(P)})$ quantifies position–coin entanglement at terminal time.

Gradients flow cleanly. Our losses depend on P_T (or $\rho_T^{(P)}$), which are linear in ρ_T and differentiable in ψ_T :

$$\psi_{t+1} = U_t \psi_t \Rightarrow \rho_T = |\psi_T\rangle\langle\psi_T| \Rightarrow \rho_T^{(P)} = \text{Tr}_C(\rho_T) \Rightarrow \mathcal{L}(P_T).$$

Autodiff/backprop carries gradients through the DTQW, the partial-trace reduction (sum over coin ports), and into the policy parameters.

Why we reduce to position: the role of $\rho_T^{(P)} = \text{Tr}_C(|\psi_T\rangle\langle\psi_T|)$. Most tasks in this work (state transfer, search, mixing, robustness) depend only on the *position* of the walker, not on the unobserved coin degrees of freedom. The coin is an *internal control knob* used to shape interference; it is not measured at evaluation time. The correct mathematical way to discard an unobserved subsystem is the *partial trace*. Concretely:

$$\rho_T^{(P)} := \text{Tr}_C(|\psi_T\rangle\langle\psi_T|) \in \mathbb{C}^{|V| \times |V|}.$$

What it gives us (Born rule on position only). Let $\{\Pi_v = |v\rangle\langle v|\}_{v \in V}$ be the rank-1 projectors on position space. The probability of finding the walker at vertex v is

$$P_T(v) = \text{Tr}((\Pi_v \otimes I_C) |\psi_T\rangle\langle\psi_T|) = \text{Tr}(\Pi_v \rho_T^{(P)}) = \langle v | \rho_T^{(P)} | v \rangle.$$

Equivalently, in the arc basis with amplitudes $\psi_T(v, a)$,

$$P_T(v) = \sum_{a=1}^{d_v} |\psi_T(v, a)|^2,$$

i.e., the diagonal of $\rho_T^{(P)}$ is exactly the marginal position distribution.

Reduced position density (concrete formula). Let $\rho_T = |\psi_T\rangle\langle\psi_T|$ be the joint state on $\mathcal{H}_P \otimes \mathcal{H}_C$ and write the arc basis as $\{|v, a\rangle\}$ with $a = 1, \dots, d_v$ the local port index at v . The reduced position density is

$$\rho_T^{(P)} = \text{Tr}_C(\rho_T), \quad [\rho_T^{(P)}]_{v, v'} = \sum_{a=1}^{d_v} \langle v, a | \rho_T | v', a \rangle. \quad (14)$$

In particular, the position probabilities are the diagonal entries

$$P_T(v) = \langle v | \rho_T^{(P)} | v \rangle = \sum_{a=1}^{d_v} |\psi_T(v, a)|^2, \quad (15)$$

so in code one can reshape the statevector ψ_T to **[position, coin]** and sum $|\cdot|^2$ over the coin axis. More generally, for any observable acting only on positions, $A_P \otimes I_C$, we have

$$\text{Tr}[(A_P \otimes I_C) \rho_T] = \text{Tr}[A_P \rho_T^{(P)}], \quad (16)$$

which shows that $\rho_T^{(P)}$ contains exactly the statistics needed for our position-only objectives (transfer/search/mixing/robustness) while remaining invariant to coin-gauge choices and avoiding explicit coin measurements.

Why trace and not “measure the coin”? Measuring the coin would induce back-action and condition the state on a coin outcome, producing *different* statistics. Our objectives ask, “What is the position distribution *if I ignore the*

coin?”—that is a *marginal*, not a conditional. Partial trace is the unique completely-positive trace-preserving (CPTP) map that discards an unobserved subsystem; it preserves all expectation values of *position-only* observables:

$$\text{Tr}((A \otimes I_C) |\psi_T\rangle\langle\psi_T|) = \text{Tr}(A \rho_T^{(P)}) \quad \forall A \text{ on } \mathcal{H}_P.$$

Thus any task function built from position measurements (e.g., $P_T(j^*)$ or total variation distance to uniform) can—and should—be computed from $\rho_T^{(P)}$.

Gauge- and symmetry-compatibility. Local rephasings and relabelings in the coin space (gauge choices) change $|\psi_T\rangle$ but leave $\rho_T^{(P)}$ invariant with respect to position-only observables. Working with $\rho_T^{(P)}$ therefore removes spurious dependence on coin gauge and makes results comparable across implementations.

Open-system and noise viewpoint. If coin dephasing or edge disorder is present, the global state may be mixed: $\rho_T = \mathbb{E}[|\psi_T\rangle\langle\psi_T|]$. Reducing gives $\rho_T^{(P)} = \text{Tr}_C(\rho_T)$, which correctly aggregates classical randomness and quantum coherence into the position statistics we evaluate.

Computational practice. We almost never need the full $|V| \times |V|$ matrix $\rho_T^{(P)}$. For all objectives used here it suffices to compute the diagonal probabilities,

$$P_T(v) = \sum_a |\psi_T(v, a)|^2,$$

which is $O(|V|\bar{d})$ and avoids forming density matrices entirely. We only construct $\rho_T^{(P)}$ when off-diagonal position coherences are explicitly analyzed.

Summary. Taking Tr_C : (i) yields the exact position statistics implied by $|\psi_T\rangle$ without coin measurement, (ii) makes objectives (transfer/search/mixing) well-defined and gauge-invariant, (iii) composes cleanly with noise models, and (iv) is computationally cheap when only P_T is needed.

13 Learning Policy π_θ (Definition)

The *learning policy* is a permutation-equivariant mapping that outputs coin parameters per node and time: **why 3 and d_v^2 , rationale is?**

$$\pi_\theta : (G, \{x_v\}_{v \in V}, t) \mapsto \{\theta_{v,t} \in \mathbb{R}^p\}_{v \in V}, \quad p = \begin{cases} 3, & d_v = 2 \text{ (SU(2) Euler angles)}, \\ d_v^2, & d_v > 2 \text{ (skew-Hermitian } K_{v,t} \text{ params)} \end{cases}$$

$$\text{and} \quad C_v(t) = \Phi(\theta_{v,t}) = \begin{cases} R_z(\alpha_{v,t}) R_y(\beta_{v,t}) R_z(\gamma_{v,t}) \in \text{SU}(2), & d_v = 2, \\ \exp(K_{v,t}), \quad K_{v,t}^\dagger = -K_{v,t} \in U(d_v), & d_v > 2. \end{cases}$$

Equivariance means: reindexing G by any permutation π reindexes outputs identically: $\pi_\theta(\pi \cdot G, \pi \cdot X, t) = \pi \cdot \pi_\theta(G, X, t)$ (see Theorem in the main text (invariant paper)).

13.1 Why $p = 3$ (when $d_v = 2$) and $p = d_v^2$ (when $d_v > 2$)

Degree $d_v = 2$: 3 parameters via SU(2). Any 2×2 unitary factors as a global phase times a special unitary:

$$U(2) \cong U(1) \times \text{SU}(2), \quad \dim U(2) = 4, \quad \dim \text{SU}(2) = 3.$$

A standard ZYZ Euler factorization is

$$U = e^{i\phi} R_z(\alpha) R_y(\beta) R_z(\gamma),$$

with $\alpha, \beta, \gamma \in \mathbb{R}$. In a coined DTQW, a *local* scalar $e^{i\phi_v} I_2$ applied at vertex v multiplies all outgoing amplitudes at v by the same phase and can be absorbed into a time-local gauge rephasing of the arc basis without affecting position-space observables. Hence the physically relevant action of a degree-2 coin is fully captured by the SU(2) part, requiring **3** real parameters (α, β, γ) . This yields our choice $p = 3$ for $d_v = 2$.

Degree $d_v > 2$: d_v^2 parameters via $U(d_v)$. The unitary group $U(d)$ is a d^2 -dimensional Lie group; its Lie algebra $\mathfrak{u}(d)$ is the real vector space of skew-Hermitian matrices K ($K^\dagger = -K$) of dimension d^2 . We parameterize the coin by the exponential (or Cayley) map:

$$C_v(t) = \exp(K_{v,t}), \quad K_{v,t} = i \sum_{k=1}^{d_v^2} \theta_{v,t}^{(k)} G_k, \quad K_{v,t}^\dagger = -K_{v,t},$$

where $\{G_k\}$ is a fixed orthonormal Hermitian basis (e.g., generalized Gell–Mann plus identity). Thus the coefficient vector $\theta_{v,t} \in \mathbb{R}^{d_v^2}$ spans all of $\mathfrak{u}(d_v)$, giving $p = d_v^2$.

Why not $SU(d_v)$ ($d_v^2 - 1$) by default? Enforcing $\text{Tr } K_{v,t} = 0$ restricts to $\mathfrak{su}(d_v)$ and yields $p = d_v^2 - 1$. We keep the $U(1)$ (trace) direction because a per-*node* scalar phase generally affects *relative* phases across nodes and times after the shift, and can therefore change interference patterns downstream. If desired for identifiability or efficiency, one can drop the identity generator (use $\text{Tr } K_{v,t} = 0$) and work with $p = d_v^2 - 1$; our framework supports both.

Low-dimensional structured alternatives. When d_v is moderate/large, one may choose a subspace with $p = q < d_v^2$ (e.g., a subset of generators or a product of complex Givens rotations) to inject inductive bias and reduce compute. All gradients and training mechanics remain unchanged.

Operational intuition. For $d_v = 2$, the 3 Euler angles control the split and relative phase between the two ports, setting up constructive/destructive interference. For $d_v > 2$, the d_v^2 parameters shape both redistribution and phases across all d_v ports; retaining the trace direction lets the policy exploit per-node common phase offsets that matter once amplitudes propagate and interfere globally.

One-line summary. We use $p = 3$ for $d_v = 2$ (coins in $SU(2)$ via ZYZ) and $p = d_v^2$ for $d_v > 2$ (coins in $U(d_v)$ via $K^\dagger = -K$ and $\exp$), with an optional $SU(d_v)$ variant ($p = d_v^2 - 1$) or a reduced q -dimensional subspace when warranted.

13.2 Where the Policy π_θ Lives (Composition and Notation)

Avoiding overload. In §13 we wrote $\pi_\theta(\cdot) \mapsto \{\theta_{v,t}\}$, using the same symbol θ for both the *policy weights* and the *coin parameters*. To avoid confusion, we henceforth use:

Policy weights = ω (trainable parameters of the network),	Coin parameters = $\varphi_{v,t}$ (outputs per node/time).
--	--

With this convention, the policy is π_ω and its outputs are $\{\varphi_{v,t}\}_{v \in V}$.

Policy as a composition. The policy is exactly the composition of the three modules from §16:

$\pi_\omega = \underbrace{\text{linear map}}_{W_{\text{head}}, b_{\text{head}}} \circ \underbrace{\text{Temporal Controller}}_{\text{GRU/Transformer } (\omega_{\text{tmp}})} \circ \underbrace{\Phi_{\text{GNN}}}_{\text{GNN encoder } (\omega_{\text{gnn}})}$

so that, for each time t and node v ,

$$z_v = \Phi_{\text{GNN}}(G, X; \omega_{\text{gnn}})_v, \quad s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)], \quad h_{v,t}^{\text{tmp}} = \text{Controller}(s_{v,t}; \omega_{\text{tmp}}), \quad \varphi_{v,t} = W_{\text{head}} h_{v,t}^{\text{tmp}} + b_{\text{head}}.$$

We then map $\varphi_{v,t}$ into a *physical* coin by the unitary lift Φ :

$$C_v(t) = \Phi(\varphi_{v,t}) = \begin{cases} R_z(\alpha) R_y(\beta) R_z(\gamma) \in SU(2), & d_v = 2, \\ \exp(K_{v,t}(\varphi_{v,t})) \in U(d_v), & d_v > 2, \end{cases} \quad C_t = \bigoplus_v C_v(t), \quad U_t = S C_t.$$

Compact definition (matches §13). With the new symbols, §13 reads:

$$\pi_\omega : (G, \{x_v\}, t) \mapsto \{\varphi_{v,t} \in \mathbb{R}^p\}_{v \in V}, \quad C_v(t) = \Phi(\varphi_{v,t}),$$

where $p = 3$ for $d_v = 2$ and $p = q$ (the chosen generator basis size) for $d_v > 2$. Permutation-equivariance follows from Φ_{GNN} and the nodewise application of the controller/head.

Notational Reconciliation: $\varphi_{v,t}$ vs. $\theta_{v,t}$

Why two symbols appeared. Earlier drafts overloaded θ for both *policy weights* and *coin parameters*. To avoid this, we renamed the trainable policy weights to ω (cf. §13.2). In some sections (e.g., §13) the per-node, per-time *coin parameters* were written as $\theta_{v,t}$, whereas in §13.2 we temporarily denoted the same object by $\varphi_{v,t}$ to visually separate it from the weights.

Single-source-of-truth convention. Throughout the paper we henceforth use:

$$\boxed{\text{Policy weights} = \omega, \quad \text{Coin parameters} = \theta_{v,t} \in \mathbb{R}^{p(d_v)}.$$

Whenever $\varphi_{v,t}$ appears, read it as an alias for the coin parameters:

$$\boxed{\varphi_{v,t} \equiv \theta_{v,t}}.$$

Where $\theta_{v,t}$ is produced in the stack. At time t , node v :

$$\underbrace{z_v}_{\text{GNN } \Phi_{\text{GNN}}(G, X; \omega_{\text{gnn}})}, \quad s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)], \quad \underbrace{h_{v,t}}_{\text{Controller}(s_{v,t}; \omega_{\text{tmp}})}, \quad \boxed{\theta_{v,t} = W_{\text{head}} h_{v,t} + b_{\text{head}}}.$$

Thus $\theta_{v,t}$ is the *output* of the (controller→head) part of π_ω .

Dimensionality $p(d_v)$. As detailed in §13.1:

$$p(d_v) = \begin{cases} 3, & d_v = 2 \text{ (SU(2) Euler ZYZ)}, \\ d_v^2 \text{ (or } d_v^2 - 1 \text{ for SU}(d_v)), & d_v > 2 \text{ (via } K^\dagger = -K). \end{cases}$$

From parameters to physical coins (unchanged). The unitary lifting map Φ consumes $\theta_{v,t}$ (or $\varphi_{v,t}$, identically) to produce a coin:

$$C_v(t) = \Phi(\theta_{v,t}) = \begin{cases} R_z(\alpha)R_y(\beta)R_z(\gamma) \in \text{SU}(2), & d_v = 2, \\ \exp(K_{v,t}(\theta_{v,t})) \in U(d_v) \text{ (or Cayley)}, & d_v > 2. \end{cases}$$

Gradients: what is differentiated and where it flows. Training minimizes $L(\omega) = -f(\psi_T(\omega))$. Chain rule gives

$$\frac{\partial L}{\partial \omega} = \sum_{t,v} \underbrace{\frac{\partial L}{\partial \theta_{v,t}}}_{\text{from coin-map \& simulator}} \underbrace{\frac{\partial \theta_{v,t}}{\partial \omega}}_{\text{head} \rightarrow \text{controller} \rightarrow \text{GNN}}, \quad \frac{\partial L}{\partial \theta_{v,t}} = 2 \text{Re} \left\langle \lambda_{t+1}, S \frac{\partial C_t}{\partial \theta_{v,t}} \psi_t \right\rangle,$$

exactly as stated in §9.4. If one prefers to keep $\varphi_{v,t}$ in intermediary formulas, replace $\theta_{v,t}$ by $\varphi_{v,t}$ everywhere—the derivatives are identical since $\varphi_{v,t} \equiv \theta_{v,t}$.

(Optional) Stochastic/RL policy notation. In RL settings we sometimes parametrize a distribution over actions (coin parameters). Then

$$\theta_{v,t} \sim \pi_\omega(\cdot \mid s_{v,t}), \quad \text{e.g., } \theta_{v,t} = \mu_{v,t}(\omega) + \sigma_{v,t}(\omega) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

where $\mu_{v,t}, \sigma_{v,t}$ are produced by the same head/controller and were denoted $\varphi_{v,t}$ in some drafts. Even then, the physical coin still uses the sampled $\theta_{v,t}$ via $C_v(t) = \Phi(\theta_{v,t})$; policy gradients (e.g., PPO) update ω through μ, σ .

Code-level mnemonic (one-liner).

$$\text{theta} = \text{head}(\text{controller}([z, t/T, \text{PE}])) \equiv \text{phi} = \text{head}(\text{controller}([z, t/T, \text{PE}])).$$

They are the same tensor; $\Phi(\text{theta})$ and $\Phi(\text{phi})$ produce identical coins and identical gradients.

Where π_ω appears in the forward pass. At each step t :

$$\boxed{\varphi_{\cdot,t} = \pi_\omega(G, X, t), \quad C_t = \Phi(\varphi_{\cdot,t}), \quad \psi_{t+1} = S C_t \psi_t.}$$

Training: gradients w.r.t. the policy (not the coins directly). We minimize $L(\omega) = -f(\psi_T(\omega))$ with backpropagation through time:

$$\frac{\partial L}{\partial \omega} = \sum_{t,v} \frac{\partial L}{\partial \varphi_{v,t}} \cdot \underbrace{\frac{\partial \varphi_{v,t}}{\partial \omega}}_{\text{through Head} \circ \text{Controller} \circ \text{GNN}} \quad \text{with} \quad \frac{\partial L}{\partial \varphi_{v,t}} = 2 \operatorname{Re} \langle \lambda_{t+1}, S(\partial C_t / \partial \varphi_{v,t}) \psi_t \rangle.$$

Thus optimization updates *policy weights* ω ; the coins $C_v(t)$ change as a result via $\varphi_{v,t} = \pi_\omega(\cdot)$.

One-line summary. In this project, π_ω is “GNN encoder + temporal controller + coin head”: given (G, X, t) it outputs coin parameters $\varphi_{v,t}$ that the unitary map Φ converts into the physical coins $C_v(t)$ driving the walk.

14 How the Policy is Implemented (GNN + Temporal Controller)

We implement π_θ as a *GNN encoder* composed with a *temporal controller* and a *coin head*:

$$\underbrace{(G, X) \xrightarrow{\text{GNN}} \{z_v\}}_{\text{permutation-equivariant node embeddings}} \implies \underbrace{s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)]}_{\text{node+time features}} \xrightarrow{\text{GRU/Transformer}} h_{v,t} \xrightarrow{W \cdot + b} \theta_{v,t}.$$

GNN encoder (Message Passing):

$$h_v^{(0)} = x_v, \quad m_v^{(\ell)} = \sum_{u \in \mathcal{N}(v)} \phi_m(h_v^{(\ell)}, h_u^{(\ell)}), \quad h_v^{(\ell+1)} = \phi_u(h_v^{(\ell)}, m_v^{(\ell)}), \quad \ell = 0, \dots, L-1,$$

with shared parameters and sum aggregation; set $z_v = h_v^{(L)}$.

Temporal controller: A GRU/Transformer ingests $s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)]$ and outputs $h_{v,t}$.

Coin head & unitary map: Linear head gives $\theta_{v,t}$, then Φ maps $\theta_{v,t}$ to $C_v(t)$ via $\text{SU}(2)$ angles (degree 2) or the matrix exponential/Cayley retraction (degree > 2).

15 Permutation-Equivariance in ACPL: Role, Guarantees, and How We Exploit It

What symmetry are we exploiting? Relabeling the vertices of a graph by any permutation $\pi \in S_{|V|}$ should not change the *physics* of a coined DTQW, provided we consistently relabel (i) the graph tensors, (ii) any node-indexed features, (iii) the designated targets/marks used by a task, and (iv) the outputs of the control policy. Formally, let P be the permutation matrix acting on node-indexed arrays; for the arc (port) basis we extend it to

$$R = \bigoplus_{v \in V} (|\pi(v)\rangle\langle v|) \otimes I_{d_v},$$

which permutes vertex labels and leaves the local port ordering at each vertex intact. This R acts on the total Hilbert space $\mathcal{H} = \mathcal{H}_P \otimes \mathcal{H}_C$ by $R|v, a\rangle = |\pi(v), a\rangle$. Physically observable quantities (e.g., position marginals $P_t(v)$) must transform covariantly under P .

Two levels of equivariance. We use permutation symmetry at *two* coupled levels:

1. **Classical policy level (GNN $\Rightarrow$ coin parameters).** Our message-passing encoder is permutation-equivariant (Thm. in the text): for (A, X) and P ,

$$\Phi_{\text{GNN}}(PAP^\top, PX) = P \Phi_{\text{GNN}}(A, X),$$

hence the node embeddings permute as $Z' = PZ$. Because the temporal controller and the coin head act *nodewise with shared weights*, the emitted parameters $\{\theta_{v,t}\}$ permute in the same way:

$$\pi_\omega(P \cdot G, P \cdot X, t) = P \pi_\omega(G, X, t). \quad (\text{PE-Policy})$$

2. **Quantum evolution level (coins $\Rightarrow$ dynamics).** Assembling the per-node coins into the block-diagonal $C_t = \bigoplus_v C_v(t)$ yields

$$C'_t = \bigoplus_v C'_{\pi(v)}(t) = R C_t R^\dagger, \quad \text{and} \quad S' = R S R^\dagger$$

because the flip-flop shift S is a permutation unitary on the arc indices (ports map locally and are relabeled consistently). Therefore the one-step unitary obeys

$$U'_t = S' C'_t = (R S R^\dagger)(R C_t R^\dagger) = R (S C_t) R^\dagger = R U_t R^\dagger. \quad (\text{PE-Evolution})$$

For any initial state transformed covariantly, $\psi'_0 = R \psi_0$, the entire trajectory satisfies $\psi'_t = R \psi_t$ and the reduced position density transforms as $\rho_t^{(P)'} = P \rho_t^{(P)} P^\top$.

Consequences for tasks and losses. Let f be a task functional of ψ_T (or $\rho_T^{(P)}$). For each of our tasks,

- **State transfer** to j^* : under P , the target index maps to $\pi(j^*)$ and $P'_T(\pi(j^*)) = P_T(j^*)$; hence the scalar objective is invariant: $f'(\psi'_T) = f(\psi_T)$.
- **Search** with marked set M : under P , marks map to $\pi(M)$ and $\sum_{m \in \pi(M)} P'_T(m) = \sum_{m \in M} P_T(m)$.
- **Fast mixing** to uniform: the uniform distribution is fixed by any permutation, and $\text{TV}(\rho_T^{(P)'}, \text{Uniform}) = \text{TV}(\rho_T^{(P)}, \text{Uniform})$.
- **Robust transport** to j^* (and expectation/CVaR over disorder) are likewise invariant after relabeling the designated target.

Thus, for any permutation-consistent relabeling,

$$\mathcal{L}(\pi_\omega(P \cdot G, P \cdot X, t)) = \mathcal{L}(\pi_\omega(G, X, t)),$$

where $\mathcal{L}$ is any of our task losses. This shows *policy-level equivariance* couples with *physics-level covariance* to yield *loss invariance*.

How we use this symmetry in practice.

1. **Parameter sharing and data efficiency.** Equivariance allows a *single set* of weights ω to apply at *every* node of *every* graph instance, independent of labeling. This drastically reduces sample complexity: each episode implicitly contains all of its $|V|!$ relabelings for free.
2. **Cross-graph generalization.** Because the encoder computes local structural summaries (equivariantly) and the controller/head act nodewise, policies trained on graphs of certain sizes/topologies *transfer* to new sizes/topologies without changes to ω . This is the formal underpinning of our OOD-size tests.
3. **Leak-free augmentation.** Permutation of node indices is a *valid data augmentation* that leaves the loss invariant. We do not need to instantiate it explicitly; Eq. (PE-Policy) ensures the network would produce permuted outputs automatically if we did.
4. **Validation: permutation sanity checks.** In Sec. 31.1, the `PermuteNodes` ablation applies random P to inputs and targets. By Eqs. (PE-Policy)–(PE-Evolution), the scalar metrics must be unchanged up to sampling noise. Any drift indicates a bug (e.g., feature leakage from raw IDs or a non-equivariant module).
5. **Design of positional encodings (PE).** We use Laplacian PEs with *deterministic sign fixing* so that $\pi(\text{pos}_v)$ transforms as $P\pi(\text{pos})$ under relabeling. This preserves the encoder’s equivariance. (Raw eigenvectors are defined up to sign; fixing the sign by the largest-magnitude component makes the map deterministic.)
6. **Compatibility with the coin map and shift.** The unitary lift $C_v(t) = \Phi(\theta_{v,t})$ is *local* and applied independently per vertex; hence it commutes with node permutations in the block-diagonal sense: $C'_t = R C_t R^\dagger$. The flip-flop shift is a *permutation* on arc indices; relabeling vertices simply conjugates S by R (Eq. (PE-Evolution)). Therefore the full propagator obeys $U'_t = R U_t R^\dagger$, establishing end-to-end covariance of the dynamics.
7. **Hypothesis class control (regularization by symmetry).** Equivariance reduces the hypothesis class to *graph-compatible* functions, acting as an architectural prior that prevents overfitting to arbitrary IDs. This complements statistical regularizers (DropEdge, weight decay) with a hard symmetry constraint.

A compact compositional lemma (policy-to-coin equivariance).

Proposition 1 (Nodewise composition preserves equivariance). *Let Φ_{GNN} be permutation-equivariant, and let Ψ be a nodewise map applied with shared parameters (temporal controller + linear head), i.e., $(\Psi(Z))_v = \psi(z_v)$ for all v . Then $\Psi \circ \Phi_{\text{GNN}}$ is permutation-equivariant:*

$$\Psi(\Phi_{\text{GNN}}(PAP^\top, PX)) = P \Psi(\Phi_{\text{GNN}}(A, X)).$$

If $\Phi : \mathbb{R}^p \rightarrow U(d_v)$ is applied locally at each node v , then the block-diagonal assembly $C_t = \bigoplus_v \Phi(\theta_{v,t})$ transforms by conjugation: $C'_t = R C_t R^\dagger$.

Proof sketch. Equivariance of Φ_{GNN} gives $Z' = PZ$. Nodewise application with shared parameters satisfies $\Psi(Z') = P\Psi(Z)$. Local application of Φ yields per-node blocks that are simply reordered by R , hence the block-diagonal conjugation statement. $\square$

What this buys us in the results section. When we report that the learned policy *generalizes* from, say, lines with $N \in [64, 192]$ to $N = 256$, or from $L \in [32, 80]$ grids to $L = 96$, we are leveraging the fact that: (i) the input preprocessing and encoder preserve permutation-equivariance; (ii) the controller/head are nodewise; (iii) the coin construction is local; and (iv) the DTQW evolution is covariant under such relabelings. Therefore the policy genuinely captures *structural motifs and temporal choreography*, rather than overfitting to indices or a single canonical labeling.

Implementation checklist for symmetry safety.

1. Never use raw node IDs as features.
2. Use PE with deterministic sign fixing; concatenate only functions of the graph (degree, LapPE, coordinates on grids).
3. Ensure the controller/head are shared *per node* (no node-specific parameters).
4. Build the shift S purely from the `port_map` permutation; avoid any ID-based shortcuts.
5. In tests, randomize node orderings and verify metric invariance.

One-line summary. Permutation-equivariance is the architectural guarantee that: relabel in, relabel out. Together with the covariance of the quantum evolution, it makes the loss invariant under graph relabelings, enabling data efficiency, cross-graph transfer, and robust evaluation without leakage or spurious index cues.

16 How the Policy is Implemented (GNN Encoder + Temporal Controller + Coin Map)

Terminology. Throughout, “temporal controller” (time-aware controller) is intended; any instance of “temporary controller” in earlier drafts should be read as “temporal controller.”

16.1 GNN Encoder: What It Is and Why We Use It

Role in ACPL. The GNN encoder Φ_{GNN} turns the raw, permutation-symmetric graph input (G, X) into *node embeddings* $\{z_v \in \mathbb{R}^H\}$ that summarize each vertex’s local structure (degree, motifs, spectral cues) in a way that is *equivariant* to relabeling nodes. This is essential because the coin we choose at vertex v should depend on *structure*, not on arbitrary vertex IDs.

Inputs and outputs. Inputs are the sparse graph $G = (V, E)$ (via adjacency/edge list), node features x_v (degree, positional encodings, coordinates/bitstrings), and (optionally) edge features e_{uv} . The encoder outputs $z_v = \Phi_{\text{GNN}}(G, X)_v \in \mathbb{R}^H$ for all $v \in V$.

What Exactly Are X and x_v ?

Definition (collection vs. per-node). We denote by

$$X = \{x_v\}_{v \in V} \in \mathbb{R}^{|V| \times F}$$

the *node-feature matrix* (row-stacked), where each row is the full feature vector for a single node:

$$x_v \in \mathbb{R}^F \quad (\text{the complete set of features attached to node } v).$$

Equivalently, X is sometimes written as X_V to emphasize it is the collection over all vertices; both notations mean the same object.

What lives in x_v (project-specific composition). Unless explicitly stated otherwise, we use the following canonical composition for this project:

$$x_v = \left[\underbrace{\deg(v)}_{\text{degree (scalar)}} \mid \underbrace{\pi_{\text{Lap}}(v)}_{\text{Laplacian positional encodings}} \mid \underbrace{\pi_{\text{geom}}(v)}_{\text{graph-family coordinates/bitstrings}} \mid \underbrace{\text{optional role/spectral stats}}_{\text{if enabled}} \right] \in \mathbb{R}^F,$$

with the precise blocks depending on the graph family:

- **Lines/cycles (degree $d_v \in \{1, 2\}$):** degree (1D scalar), normalized coordinate x/L (for lines/cycles with canonical ordering), and optionally top- K Laplacian eigenvectors $\pi_{\text{Lap}}(v) \in \mathbb{R}^K$ (sign-fixed).
- **2D grids (degree $d_v \in \{2, 3, 4\}$):** degree, normalized grid coordinates $(x/L, y/L)$, and optionally $\pi_{\text{Lap}}(v) \in \mathbb{R}^K$ for extra spectral context.
- **Hypercubes Q_n (degree n):** degree n , the node’s bitstring in $\{0, 1\}^n$ (optionally centered to $\{-1, +1\}$), its Hamming weight, and optionally $\pi_{\text{Lap}}(v)$.
- **Irregular graphs (ER, d -regular mix, small-world):** degree d_v , $\pi_{\text{Lap}}(v) \in \mathbb{R}^K$ (top- K eigenvectors of the graph Laplacian with a deterministic sign convention), and optionally simple structural features (e.g., local clustering or 1–2 role encodings).

What *does not* belong to X . Time is not a node attribute, so the normalized time $\tau_t = t/T$ is *not* part of x_v ; we concatenate it later at control time via $s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)]$ (§16). Likewise, edge features e_{uv} (if used) are kept separately and fed into the message function $M^{(\ell)}(\cdot)$, not into x_v .

Normalization and invariances. We min-max or z-score scalar features (degree, coordinates) within each graph; Laplacian eigenvectors are already L^2 -normalized and their global sign is fixed by making the largest-magnitude entry positive. This keeps X numerically stable and ensures that permutation of node indices only permutes the rows of X , preserving the GNN’s permutation equivariance.

Shapes, dtypes, batching. On a single graph with $N = |V|$:

$$X \in \mathbb{R}^{N \times F}, \quad x_v \in \mathbb{R}^F \quad (\text{float32}).$$

In mini-batches of many small graphs we row-concatenate X and use a **batch** vector to record graph membership; in a single large-graph setting we keep one X and train with neighborhood/cluster sampling.

Recommended defaults (used in experiments). Unless otherwise noted: degree (scaled to $[0, 1]$); $K = 8$ –16 Laplacian eigenvectors for irregular graphs; $(x/L, y/L)$ for grids; $\{0, 1\}^n$ bitstrings and Hamming weight for hypercubes. These defaults give strong, permutation-respecting structural signals to the GNN without leaking absolute IDs.

How X is consumed by the policy. The GNN encoder Φ_{GNN} takes (G, X) and produces permutation-equivariant node embeddings $z_v = \Phi_{\text{GNN}}(G, X)_v$. Those embeddings—not X directly—drive the temporal controller and coin head to emit $\theta_{v,t}$ (see §16 and §32).

Message-passing mechanics (under the hood). We use an L -layer message-passing network (GCN/GraphSAGE/GAT/GIN/PNA—instantiated per experiment). At layer ℓ ,

$$\text{(message)} \quad m_{u \rightarrow v}^{(\ell)} = M^{(\ell)}(h_u^{(\ell)}, h_v^{(\ell)}, e_{uv}), \quad \text{(aggregate)} \quad \bar{m}_v^{(\ell)} = \sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}^{(\ell)},$$

$$\text{(update)} \quad h_v^{(\ell+1)} = U^{(\ell)}(h_v^{(\ell)}, \bar{m}_v^{(\ell)}), \quad h_v^{(0)} = x_v, \quad z_v = h_v^{(L)}.$$

Sum (or attention-weighted sum) aggregation with *shared* $M^{(\ell)}, U^{(\ell)}$ across nodes yields permutation-equivariance (cf. our theorem).

Message-Passing Mechanics (Expanded)

Layer blueprint (what is instantiated). We implement an L -layer Message Passing Neural Network (MPNN) with *shared* message/update functions across all nodes/edges at a given layer [30]:

$$\text{(message)} \quad m_{u \rightarrow v}^{(\ell)} = M^{(\ell)}(h_u^{(\ell)}, h_v^{(\ell)}, e_{uv}), \quad (17)$$

$$\text{(aggregate)} \quad \bar{m}_v^{(\ell)} = \sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}^{(\ell)}, \quad (18)$$

$$\text{(update)} \quad h_v^{(\ell+1)} = U^{(\ell)}(h_v^{(\ell)}, \bar{m}_v^{(\ell)}), \quad h_v^{(0)} = x_v, \quad z_v = h_v^{(L)}. \quad (19)$$

Sum aggregation (or attention-weighted sum) together with weight-sharing ensures *permutation-equivariance* (our theorem in the text).

Canonical instantiations (how $M^{(\ell)}$ and $U^{(\ell)}$ are realized). We choose among standard, well-understood variants, per experiment:

- **GCN [31]** (linear mixing by renormalized adjacency). With $\tilde{A} = A + I$, $\tilde{D}_{vv} = \sum_u \tilde{A}_{vu}$, and $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$,

$$H^{(\ell+1)} = \sigma(\hat{A} H^{(\ell)} W^{(\ell)}).$$

This corresponds to $M^{(\ell)}(h_u, h_v) = \hat{A}_{vu} h_u W^{(\ell)}$ and $U^{(\ell)}(h_v, \bar{m}_v) = \sigma(\bar{m}_v)$; edge features are unused (or folded into weights when present).

- **GraphSAGE (mean) [36]** (nonlinear update on concatenation):

$$\bar{m}_v^{(\ell)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} h_u^{(\ell)}, \quad h_v^{(\ell+1)} = \sigma(W^{(\ell)}[h_v^{(\ell)} \parallel \bar{m}_v^{(\ell)}]).$$

Variants include *max/mean-pool* and an MLP inside $M^{(\ell)}$.

- **GAT [32]** (attention-weighted aggregation):

$$e_{uv}^{(\ell)} = \text{LeakyReLU}(a^{(\ell)\top} [W^{(\ell)} h_u^{(\ell)} \parallel W^{(\ell)} h_v^{(\ell)}]),$$

$$\alpha_{uv}^{(\ell)} = \frac{\exp e_{uv}^{(\ell)}}{\sum_{w \in \mathcal{N}(v)} \exp e_{wv}^{(\ell)}}, \quad h_v^{(\ell+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv}^{(\ell)} W^{(\ell)} h_u^{(\ell)}\right).$$

Multi-head attention is implemented by concatenating or averaging heads.

- **GIN [38]** (expressivity $\approx$ 1-WL):

$$h_v^{(\ell+1)} = \text{MLP}^{(\ell)}\left((1 + \epsilon^{(\ell)}) h_v^{(\ell)} + \sum_{u \in \mathcal{N}(v)} h_u^{(\ell)}\right).$$

This is our default on lines/grids when we want strong local discrimination.

- **PNA [39]** (degree-aware, multi-aggregator):

$$\bar{m}_v^{(\ell)} = \text{Combine}(\{\text{AGG}_k(\{h_u^{(\ell)}\})\}_k, \text{scale}(d_v)), \quad h_v^{(\ell+1)} = \text{MLP}^{(\ell)}(h_v^{(\ell)} \parallel \bar{m}_v^{(\ell)}),$$

with $\text{AGG}_k \in \{\text{sum}, \text{mean}, \text{max}\}$ and degree-based scalars. We use this on irregular graphs to tame degree skew.

Edge features and degrees (how they enter). When present, edge features e_{uv} (e.g., weights or types) are injected into $M^{(\ell)}$ via concatenation or bilinear terms, e.g.,

$$m_{u \rightarrow v}^{(\ell)} = \phi_m^{(\ell)}(W_u^{(\ell)} h_u^{(\ell)} \parallel W_v^{(\ell)} h_v^{(\ell)} \parallel B^{(\ell)} e_{uv}).$$

Degrees d_v are included in x_v and/or used implicitly by $\hat{A}$ (GCN) or by degree scalars (PNA). This respects permutation symmetry while providing structural signal.

Positional encodings (PE) inside message passing. Laplacian PEs $\pi_{\text{Lap}}(v) \in \mathbb{R}^K$ are concatenated to x_v . Because PEs are functions of the graph *structure*, permuting node indices jointly permutes rows of X and hence the embeddings z_v , preserving equivariance.

Stabilizers (depth, smoothing, squashing). We use residual/skip connections $h_v^{(\ell+1)} \leftarrow h_v^{(\ell+1)} + h_v^{(\ell)}$ when widths match, plus normalization (GraphNorm/LayerNorm [45], PairNorm [44]) and DropEdge [43]. These help with:

- **Over-smoothing:** node states becoming too similar at depth; mitigated by residuals, norm layers, and Jumping Knowledge [37] (optional).
- **Over-squashing:** long-range signals forced through few edges; mitigated by attention (GAT), PNA scalars, or adding virtual/long-range edges in ablations (not in the main policy).

Why this matters for ACPL (how z_v drives coin control). The encoder maps (G, X) to *node embeddings* z_v that summarize each vertex’s local scattering context (degree, motif fingerprints, spectral position). The temporal controller then conditions on $s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)]$ to produce *coin parameters* $\theta_{v,t}$. Thus:

$$(G, X) \xrightarrow{\Phi_{\text{GNN}}} \{z_v\} \xrightarrow{\text{controller}} \{\theta_{v,t}\} \xrightarrow{\Phi} \{C_v(t)\} \xrightarrow{\text{DTQW}} \psi_T.$$

Gradients from the task loss $-f(\psi_T)$ flow back through Φ (unitary lift), the controller, and into $M^{(\ell)}, U^{(\ell)}$ to make z_v maximally predictive for choosing coins that orchestrate constructive/destructive interference (transfer/search/mixing/robustness).

Permutation-equivariance (why sum/softmax works). Let P be a node permutation. With shared $M^{(\ell)}, U^{(\ell)}$ and sum/softmax aggregation over neighbors, permuting nodes simply permutes the indexing of inputs and neighbor sets. One checks by induction that $H'^{(\ell)} = PH^{(\ell)}$ for all ℓ ; hence $Z' = PZ$ (cf. our theorem). This ensures policies trained on one labeling transfer to any relabeling and, with meta-training, to unseen graphs.

Computational form (sparse kernels). Each layer is implemented with GPU-friendly sparse primitives:

- **Gather:** take $h_u^{(\ell)}$ per edge ($u \rightarrow v$) to form messages.
- **Edge MLP / attention kernel:** compute $m_{u \rightarrow v}^{(\ell)}$ or scores $e_{uv}^{(\ell)}$.
- **Segment reduce (scatter_add/mean/max):** aggregate to $\bar{m}_v^{(\ell)}$ over incoming edges.
- **Update MLP:** compute $h_v^{(\ell+1)}$ and apply norm/activation/residual.

For GCN this collapses to an SPMM: $\hat{A}$ (sparse) times $H^{(\ell)} W^{(\ell)}$ (dense). For GAT we compute edgewise scores, apply a segment-wise softmax over $\mathcal{N}(v)$, then a weighted SPMM.

Backpropagation (how gradients reach coins). Let $\mathcal{L} = -f(\psi_T)$ and denote $\partial \mathcal{L} / \partial \theta_{v,t}$ the gradient from the DTQW adjoint (see §9.4). Chain rule delivers

$$\frac{\partial \mathcal{L}}{\partial \omega_{\text{head}}}, \quad \frac{\partial \mathcal{L}}{\partial \omega_{\text{tmp}}}, \quad \frac{\partial \mathcal{L}}{\partial \omega_{\text{gnn}}}$$

by backpropagating through the controller and encoder. In the encoder, gradients flow edge-wise through gather $\rightarrow$ message MLPs/attention $\rightarrow$ segment-reduce $\rightarrow$ updates. Sparse kernels supply custom adjoints; memory is linear in $|E|$ per layer.

Choice per task (why these variants).

- **Lines/cycles** ($d = 2$): GIN (2–3 layers) or GCN with residuals; simple local structures suffice for state transfer.
- **Grids** ($d = 4$): GCN/GIN; optional GAT for marked-node search where anisotropic attention can help.
- **Hypercubes** ($d = n$): GCN/GIN with bitstring features; attention is optional (degree is large but regular).
- **Irregular graphs**: PNA to handle degree skew and heterogeneous neighborhoods robustly.

Complexity (per layer and end-to-end). For hidden width H , the per-layer cost is $O(|E|H)$ (message/aggregation) plus $O(|V|H^2)$ (update MLP if used); in practice the sparse edge term dominates. End-to-end policy cost per control step is

$$O(|E|H) \text{ (GNN)} + O(|V|\bar{d}^2) \text{ (assemble/apply coins)},$$

hence total $O(T(|E|H + |V|\bar{d}^2))$ (see §11).

What z_v captures for coin control. Each z_v condenses:

- **Connectivity patterns** (degree, local motifs) that affect how amplitude can split/interfere from v .
- **Spectral/positional context** via Laplacian positional encodings (LapPE) or coordinates (grids).
- **Heterogeneity** (irregular degrees d_v) that determines whether the local coin should mimic a Grover-like reflection, a Fourier-like spread, or a focused rotation.

In short, z_v is the structural “state” the controller will consult before emitting coin parameters at time t .

Stability and depth. We keep $L \in \{2, 3\}$ with residual connections and Graph/LayerNorm to avoid over-smoothing. On graphs that require wider context, we increase L or use attention (GAT) or multi-aggregator PNA.

16.2 Temporal Controller: What It Is and How It Uses Time

Role in ACPL. Interference patterns depend on *when* we mix or focus amplitude. The temporal controller reads the node embedding *and* the current time (normalized) to produce *per-node, per-time* control codes that specify the coin at that instant.

Inputs and outputs. For each $t = 0, \dots, T - 1$ we build

$$s_{v,t} = [z_v ; \tau_t ; \pi(\text{pos}_v)] \in \mathbb{R}^{H+1+K_{\text{PE}}}, \quad \tau_t = \frac{t}{T},$$

and feed $\{s_{v,t}\}_v$ to a shared time-conditioned module (GRU or small Transformer). It returns hidden states $h_{v,t}^{\text{tmp}}$ (per-node) or a shared global state η_t , then a linear head produces *coin parameters* $\theta_{v,t}$.

Why GRU vs. Transformer. A GRU is light-weight and enforces causal dependence on past time steps ($\eta_{t+1} = \text{GRU}(s_{:,t}, \eta_t)$). A small Transformer lets the controller attend across a short time window if temporal patterns (e.g., pulse trains) matter. We start with GRU for stability and speed.

Causality and generalization in T . Using $\tau_t = t/T$ (rather than bare t) makes the controller *scale-aware* so a schedule learned at $T = 96$ often transfers to $T = 128$. Curriculum in T plus temporal smoothness penalties encourages physically plausible, slowly varying coin schedules.

16.3 Coin Head and Unitary Map: From Parameters to Physical Coins

What the head predicts. The linear head maps $h_{v,t}^{\text{tmp}}$ to a low-dimensional parameter vector $\theta_{v,t}$:

$$\theta_{v,t} \in \begin{cases} \mathbb{R}^3 & \text{if } d_v = 2 \text{ (Euler angles for SU(2))}, \\ \mathbb{R}^q & \text{if } d_v > 2 \text{ (coeffs for a skew-Hermitian generator } K_{v,t}). \end{cases}$$

We deliberately *do not* predict the full $d_v \times d_v$ unitary matrix: predicting *parameters* keeps the controller compact and enforces structure/symmetry.

Unitary lifting (ensuring $C_v(t) \in U(d_v)$). We map $\theta_{v,t}$ to a valid unitary coin via

$$C_v(t) = \Phi(\theta_{v,t}) = \begin{cases} R_z(\alpha)R_y(\beta)R_z(\gamma) \in \text{SU}(2), & d_v = 2, \\ \exp(K_{v,t}) \in U(d_v) \text{ with } K_{v,t}^\dagger = -K_{v,t}, & d_v > 2, \end{cases}$$

where, for $d_v > 2$, we expand $K_{v,t} = i \sum_{k=1}^q \theta_{v,t}^{(k)} G_k$ using an orthonormal Hermitian basis $\{G_k\}$ (e.g., generalized Gell-Mann). For speed, we optionally use the Cayley retraction

$$C_v(t) = (I - K_{v,t})(I + K_{v,t})^{-1},$$

which is unitary whenever $I + K_{v,t}$ is invertible and has cheap, stable gradients.

Why this matters for ACPL. The *only* physical knob we have in coined DTQW is the coin. The coin head + unitary map ensure that the controller’s outputs are *physically realizable* and that optimization happens in a numerically stable, low-dimensional space while still spanning rich families of local unitaries ($\text{SU}(2)$ and subspaces of $U(d_v)$).

16.4 Putting It Together (Episode-Time Loop)

$$\begin{aligned} \text{Encode: } z_v &= \Phi_{\text{GNN}}(G, X)_v. \\ \text{Condition on time: } s_{v,t} &= [z_v; \tau_t; \pi(\text{pos}_v)]. \\ \text{Control step: } h_{v,t}^{\text{tmp}} &= \text{Controller}(s_{v,t}, \eta_t), \quad \theta_{v,t} = W h_{v,t}^{\text{tmp}} + b. \\ \text{Physical coin: } C_v(t) &= \Phi(\theta_{v,t}) \in U(d_v), \quad C_t = \bigoplus_v C_v(t). \\ \text{Walk step: } \psi_{t+1} &= S C_t \psi_t. \\ \text{Loss: } \mathcal{L} &= -f(\psi_T) \text{ (state transfer/search/mixing/robustness)}. \end{aligned}$$

16.5 Design Choices Tied to Project Goals

Generalization across graphs. The GNN encoder guarantees permutation-equivariant dependence on local structure, so a policy trained on one set of graphs can be deployed on unseen graphs/sizes.

Temporal choreography. The controller learns *when* to spread (e.g., early-time Fourier-like coins on a line) and *when* to focus (late-time phase alignment near a target for transfer/search).

Regularization for physical realism. We add (i) temporal smoothness $\sum_{v,t} \|K_{v,t} - K_{v,t-1}\|_F^2$ and (ii) magnitude penalties $\sum_{v,t} \|K_{v,t}\|_F^2$, preventing implausible, rapidly oscillating control.

Gradient flow and efficiency. Backpropagating $\partial \mathcal{L} / \partial \theta_{v,t}$ through Φ (matrix exponential or Cayley) is supported by autodiff. The overall cost per step is $O(|E|H)$ (GNN) + $O(|V|\bar{d}^2)$ (coins), linear in horizon T .

Ablations (what proves each part matters).

- Replace GNN with coord-MLP on regular grids: expect smaller gains on grids, large drops on irregulars.
- Freeze time (static coins): expect worse transfer/search, poorer mixing.
- Replace exp with unconstrained linear heads: violates unitarity, breaks dynamics.

17 Graph Convolutional Networks (GCN): Mechanics, Guarantees, and Role in ACPL

17.1 From Spectral Theory to the Renormalized GCN Layer

Spectral view. Let $G = (V, E)$ with adjacency A , degree matrix D , and (symmetric) normalized Laplacian $L_{\text{sym}} = I - D^{-1/2} A D^{-1/2}$. A (real) graph signal is $h : V \rightarrow \mathbb{R}^d$ (matrix $H \in \mathbb{R}^{|V| \times d}$). Spectral convolution defines $g_\theta \star h = U g_\theta(\Lambda) U^\top h$, where $L_{\text{sym}} = U \Lambda U^\top$. Chebyshev- K truncation [31] yields a localized polynomial filter $g_\theta \star h \approx \sum_{k=0}^K \theta_k T_k(\tilde{L}) h$ with $\tilde{L} = \frac{2}{\lambda_{\max}} L_{\text{sym}} - I$.

First-order localization (Kipf–Welling). Setting $K=1$, tying $\theta_0 = -\theta_1$, and using $\lambda_{\max} \approx 2$ gives

$$g_\theta \star h \approx \theta \left(I + D^{-1/2} A D^{-1/2} \right) h.$$

Adding self-loops $A \mapsto \tilde{A} = A + I$ and $\tilde{D} = \text{diag}(\tilde{A}\mathbf{1})$, then absorbing constants into weights yields the *renormalized* GCN layer:

$$\boxed{H^{(\ell+1)} = \sigma\left(\hat{A} H^{(\ell)} W^{(\ell)}\right), \quad \hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}},} \quad (20)$$

with $H^{(0)} = X$ and nonlinearity σ (ReLU/GELU). Equation (20) is also the *message-passing* GCN: neighbors are averaged by $\hat{A}$, then a shared linear map $W^{(\ell)}$ is applied.

17.2 Permutation-Equivariance (GCN-Specific Statement)

Let P be a node permutation matrix. Then

$$\hat{A}' = P \hat{A} P^\top, \quad X' = P X.$$

By induction,

$$H'^{(\ell+1)} = \sigma(\hat{A}' H'^{(\ell)} W^{(\ell)}) = \sigma(P \hat{A} P^\top P H^{(\ell)} W^{(\ell)}) = P \sigma(\hat{A} H^{(\ell)} W^{(\ell)}) = P H^{(\ell+1)}.$$

Thus GCN is permutation-equivariant; under relabeling, node embeddings permute in the same way (this specializes the general theorem in our policy section).

17.3 What Exactly Does $\hat{A}$ Do? (Random-Walk & Smoothing Views)

Normalized neighbor mixing. $\hat{A}$ averages each node with its neighbors, weighting by $1/\sqrt{d_u d_v}$; self-loops ensure the node’s own state is retained. Stacking L layers applies an L -hop, degree-normalized diffusion *interleaved* with learned linear transforms and nonlinearities.

Low-pass bias. GCN acts as a learnable low-pass filter on the graph spectrum (attenuating high-frequency components that vary sharply across edges). This is ideal on *homophilous* graphs (neighbors should be similar). On heterophilous graphs we use tweaks (Sec. ??).

17.4 GCN in ACPL: Why It’s a Strong Default Encoder

Recall our policy pipeline:

$$(G, X) \xrightarrow{\Phi_{\text{GNN}}} \{z_v\} \rightarrow s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)] \xrightarrow{\text{Controller}} \theta_{v,t} \xrightarrow{\Phi} C_v(t) \in U(d_v) \xrightarrow{S} \psi_{t+1}.$$

Choosing Φ_{GNN} as GCN makes z_v a *structural summary* with four properties crucial to coin control:

1. **Equivariance:** z_v respects relabelings $\Rightarrow$ policy generalizes across isomorphic regions and unseen graph sizes (backbone of our zero-shot tests).
2. **Locality:** $L \in \{2, 3\}$ yields 2–3-hop receptive fields—exactly the *local* context driving how a coin should split/interfere.
3. **Stability:** Degree normalization and self-loops tame exploding/vanishing signals and stabilize gradients, important since gradients also traverse the quantum evolution.
4. **Efficiency:** Each layer is one SPMM (sparse–dense matmul) + a dense linear map: per step cost $O(|E|H)$ and trivial to batch across episodes.

How z_v steers the coin. On the line/cycle ($d_v=2$), z_v encodes edge/endpoint proximity and degree (always 2 except boundaries), letting the controller emit SU(2) angles that *spread* early and *focus* near the target. On grids/hypercubes ($d_v=4/n$), z_v captures axis/bit symmetries and local motif counts; the controller can mimic Grover-like reflections early (fast spread), then adjust phases to accumulate amplitude at marks (search) or flatten the distribution (mixing). On irregular graphs, $\hat{A}$ ’s degree normalization prevents high-degree hubs from dominating and encodes degree heterogeneity that the coin must respect (e.g., avoid over-spreading at leaves).

17.5 Implementation Detail: What a GCN Layer Computes (MPNN Form)

GCN is an MPNN with

$$m_{u \rightarrow v}^{(\ell)} = \frac{1}{\sqrt{\tilde{d}_u \tilde{d}_v}} h_u^{(\ell)}, \quad \bar{m}_v^{(\ell)} = \sum_{u \in \mathcal{N}(v) \cup \{v\}} m_{u \rightarrow v}^{(\ell)}, \quad h_v^{(\ell+1)} = \sigma(\bar{m}_v^{(\ell)} W^{(\ell)}),$$

where $\tilde{d}_v = d_v + 1$ counts the self-loop. The single learnable object per layer is $W^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell+1}}$.

17.6 Complexity, Kernels, and Backprop

Forward. Compute $Z = \hat{A}H^{(\ell)}$ as SPMM: $O(|E|d_\ell)$; then apply $W^{(\ell)}$: $O(|V|d_\ell d_{\ell+1})$. With $d_\ell \approx d_{\ell+1} = H$ this is $O(|E|H + |V|H^2)$. In practice $|E|H$ dominates.

Backward. Autograd reuses the same sparse patterns; gradients w.r.t. $W^{(\ell)}$ are dense GEMMs; grads w.r.t. $H^{(\ell)}$ pass through another SPMM with $\hat{A}^\top = \hat{A}$.

Memory. Store $H^{(\ell)}$ for L layers ($O(|V|HL)$) plus the sparse graph ($O(|E|)$). For big batches, gradient checkpointing trades compute for memory.

17.7 Expressivity, Depth, and Oversmoothing

Expressivity. One GCN layer cannot distinguish nodes with the same degree and isomorphic one-hop neighborhoods; stacking L layers increases discrimination to L -hop neighborhoods but remains less expressive than GIN (which matches 1-WL). For our control targets, $L \in \{2, 3\}$ suffices: the coin is local and we only need a few hops.

Oversmoothing. Repeated $\hat{A}$ multiplications homogenize $H^{(\ell)} \rightarrow$ low-rank subspace. Remedies we use: residual/skip connections, GraphNorm/LayerNorm, DropEdge, Jumping Knowledge readout, and *shallow* L .

17.8 Heterophily and Variants We Drop-In When Needed

On heterophilous graphs (neighbors differ), plain GCN can underperform. Drop-in remedies (keeping overall ACPL unchanged):

- **GCN+APPNP**: separate transform and propagation via Personalized PageRank (long-range but controlled diffusion).
- **GAT**: attention-weighted $\hat{A}$ learns edge importances (helpful when not all directions are equal).
- **PNA**: multi-aggregator (sum/mean/max) with degree scalers—robust under skewed degrees.

We start with GCN for stability/speed and swap in these only where validation suggests.

17.9 Interface With Coins and Time (Why GCN $\times$ Controller Works)

GCN alone is *time-agnostic*: z_v encodes *where* we are (structure), not *when*. The temporal controller consumes $[z_v; \tau_t; \pi(\text{pos}_v)]$ and emits $\theta_{v,t}$. This separation mirrors control design:

$$\text{Structure (space)} \xrightarrow{\text{GCN}} z_v \quad + \quad \text{Schedule (time)} \xrightarrow{\text{Controller}} \theta_{v,t} \Rightarrow C_v(t) = \Phi(\theta_{v,t}).$$

It lets us (i) train one encoder across tasks, (ii) re-use z_v across t , and (iii) adapt to new horizons via $\tau_t = t/T$.

17.10 GCN Hyperparameters and Settings We Use

- **Depth** $L = 2$ or 3 ; **width** $H = 64$ – 128 ; **activation** GELU/ReLU; **norm** GraphNorm/LayerNorm after each layer; **dropout** 0.1–0.3 on features and/or DropEdge 0.1.
- **Inputs** X : degree (scaled), positional encodings (LapPE with sign-fixing) for irregular graphs; coordinates/bitstrings on regular families.
- **Initialization**: Xavier uniform for $W^{(\ell)}$; zero bias.
- **Ablations**: replace GCN by coord-MLP on grids (expect small drop), by GAT/PNA on heterophilous irregulars (expect gains if structure demands).

17.11 Why GCN is Enough for Our Four Tasks (and Where We'd Switch)

State transfer (line/cycle). Homophilous, degree-regular (except boundaries): GCN with PE encodes distance-to-boundary and local parity; controller learns $SU(2)$ pulses that spread then refocus.

Search (grid/hypercube). Moderately regular; GCN embeddings reflect lattice/bit symmetries; controller learns near-Grover phases early, then target-focused phases late. On hypercube, bitstring features + GCN capture Hamming shells.

Fast mixing (grid). Goal is low-pass by design (flatten P_T); GCN's smoothing bias is aligned with emitting coins that favor uniform spread.

Robust transport (1D with disorder). GCN encodes degree (constant) + position; disorder is in the *quantum* dynamics; controller learns phase schedules robust to random edge phases. If we injected strong structural heterophily, we would consider PNA/GAT.

17.12 Mini-Proof: GCN + Nodewise Head Gives an Equivariant Policy

Let $Z = \text{GCN}(A, X)$. For time t , define $S_t = [Z; \tau_t \mathbf{1}; \Pi]$ with positional encodings Π . A nodewise affine head gives $\Theta_t = S_t W + \mathbf{1} b^\top$. Under any node permutation P with consistent Π (sign-fixed LapPE or coordinates),

$$Z' = PZ, \quad S'_t = PS_t, \quad \Theta'_t = PS_t W + P\mathbf{1} b^\top = P\Theta_t,$$

establishing Θ_t (hence $\{\theta_{v,t}\}$) permutes with P . The local unitary lift $C_v(t) = \Phi(\theta_{v,t})$ then block-diagonally conjugates by R , yielding end-to-end covariance of the DTQW (Sec. 15).

One-line summary. GCN provides a fast, stable, permutation-equivariant encoder that distills exactly the *local structural context* a vertex needs; the temporal controller turns that context into *time-conditioned* coin parameters, and the unitary lift ensures physical $U(d_v)$ coins—together powering our adaptive coin policy across all four tasks.

18 GraphSAGE (Mean): Mechanics, Guarantees, and Role in ACPL

18.1 Inductive Message Passing with Mean Aggregation

GraphSAGE [36] learns *local* aggregation functions that generalize to unseen graphs. With hidden states $h_v^{(\ell)} \in \mathbb{R}^{d_\ell}$ and (optional) edge features e_{uv} , the **mean** variant is:

$$\text{(neighbor message)} \quad m_{u \rightarrow v}^{(\ell)} = \phi_m^{(\ell)}(h_u^{(\ell)}) \quad (\text{often the identity}), \quad (21)$$

$$\text{(mean aggregate)} \quad \bar{m}_v^{(\ell)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}^{(\ell)}, \quad (22)$$

$$\text{(update)} \quad h_v^{(\ell+1)} = \sigma\left(W^{(\ell)} [h_v^{(\ell)} \parallel \bar{m}_v^{(\ell)}] + b^{(\ell)}\right), \quad (23)$$

with $h_v^{(0)} = x_v$ and σ a nonlinearity (ReLU/GELU). We optionally apply layer norm / graph norm and residual connections. The final node embeddings are $z_v = h_v^{(L)}$.

Self-information vs. neighbor information. Unlike GCN (which mixes self and neighbors before applying a linear map), GraphSAGE concatenates a node's own representation with the (normalized) neighbor mean, then learns how much to use each via $W^{(\ell)}$. This yields robustness when self-information and neighbor-information play different roles (e.g., at degree-1 leaves or marked vertices).

18.2 Permutation-Equivariance (SAGE-Specific Statement)

Let P permute nodes, $X' = PX$, $A' = PAP^\top$. Mean aggregation is permutation-invariant over sets; with shared $W^{(\ell)}$,

$$\bar{m}_{Pv}^{(\ell)} = \frac{1}{|\mathcal{N}(Pv)|} \sum_{u' \in \mathcal{N}(Pv)} h_{u'}^{(\ell)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} (Ph_u^{(\ell)}) = P \bar{m}_v^{(\ell)}.$$

Then from (23): $h_{Pv}^{(\ell+1)} = \sigma(W^{(\ell)} [Ph_v^{(\ell)} \parallel P\bar{m}_v^{(\ell)}] + b^{(\ell)}) = Ph_v^{(\ell+1)}$. Induction gives $H' = PH$. Therefore $Z' = PZ$ and nodewise heads remain equivariant (as required by our policy).

18.3 Why GraphSAGE (Mean) is Attractive for ACPL

In our pipeline

$$(G, X) \xrightarrow{\Phi_{\text{GNN}}} \{z_v\} \rightarrow s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)] \xrightarrow{\text{Controller}} \theta_{v,t} \xrightarrow{\Phi} C_v(t) \in U(d_v) \xrightarrow{S} \psi_{t+1},$$

choosing Φ_{GNN} as SAGE-mean confers:

1. **Inductive generalization:** Trained aggregators transfer to unseen graphs/sizes (lines $\rightarrow$ longer lines; grids $\rightarrow$ larger grids; cubes $\rightarrow$ higher- n), matching our zero-shot goals.
2. **Degree-robustness:** Mean normalization mitigates hub dominance and keeps magnitudes stable across irregular degrees; crucial when coins must adapt to $d_v \in \{1, 2, 3, 4, \dots\}$.
3. **Self/neighbor disentanglement:** The controller can exploit $[h_v^{(\ell)} \parallel \bar{m}_v^{(\ell)}]$ differently over time (e.g., early-time spreading uses $\bar{m}$, late-time focusing leverages h_v).
4. **Sampling-friendly:** For very large graphs, GraphSAGE admits efficient *neighbor sampling*; even though our current tasks use moderate sizes, the design scales.

18.4 Relation to the Physics of Coin Control

Local interference choreography. Coins determine how amplitude splits from v along its ports. SAGE’s z_v summarizes (i) the node’s intrinsic role (via $h_v^{(\ell)}$) and (ii) the *average* neighborhood tendency (via $\bar{m}_v^{(\ell)}$). The controller uses this to emit $C_v(t)$ that either *equalizes* phases for mixing, *biases* toward paths to a target (transfer/search), or *counters* disorder (robust transport).

Leaves and bottlenecks. At degree-1 nodes, $\bar{m}_v^{(\ell)}$ collapses to the sole neighbor; SAGE naturally keeps strong self-information so the controller can choose near-identity or phase-keeping coins to avoid leakage, a common failure mode if the encoder over-smooths.

18.5 Implementation as MPNN (Mean)

As an MPNN,

$$m_{u \rightarrow v}^{(\ell)} = h_u^{(\ell)}, \quad \bar{m}_v^{(\ell)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} h_u^{(\ell)}, \quad h_v^{(\ell+1)} = \sigma(W^{(\ell)}[h_v^{(\ell)} \parallel \bar{m}_v^{(\ell)}] + b^{(\ell)}).$$

We commonly add: residual connection ($h_v^{(\ell+1)} + h_v^{(\ell)}$ if dims match), GraphNorm/LayerNorm after the affine, and dropout on the concatenated vector.

18.6 Complexity, Kernels, and Sampling

Full-batch cost. Per layer: compute neighbor means via a *segment-mean* (SPMM with row-normalized adjacency) $O(|E|d_\ell)$; then a dense affine on $[h \parallel \bar{m}]$ costs $O(|V|(d_\ell + d_\ell)d_{\ell+1})$. With $d_\ell \approx d_{\ell+1} = H$, practical cost is $O(|E|H + |V|H^2)$; $|E|H$ typically dominates.

Neighbor sampling (large graphs). Pick a seed set S and sample k_1 neighbors at hop 1, k_2 at hop 2, etc., constructing a layered subgraph per batch. Aggregation is then over the sampled neighborhoods, giving $O(|S|(k_1 + \dots + k_L)H)$ per batch. Our ACPL interface is unchanged; only the data loader differs.

18.7 Expressivity and Stability vs. GCN

Expressivity. SAGE-mean (with concatenation) can learn asymmetric weighting between self and neighbors and is less constrained than the single linear map in GCN. It is still less expressive than GIN (1-WL level), but adequate for *local* control patterns we require (2–3 hops).

Stability. Mean aggregation yields scale-invariant messages across varying degrees. This is beneficial when the controller must output coins on graphs with mixed d_v (e.g., irregulars or grids with obstacles).

18.8 Where We Prefer SAGE in ACPL

- **Irregular graphs / mixed degrees:** Mean normalization stabilizes signals, improving coin choices at leaves and hubs.
- **Heterophily pockets:** When neighbor statistics are not fully aligned with self features, the $[h||\bar{m}]$ split lets the controller lean more on h_v to avoid harmful over-smoothing.
- **Scaling up:** If future extensions consider massive graphs, SAGE integrates cleanly with neighbor sampling.

18.9 Hyperparameters and Settings

Depth $L \in \{2, 3\}$; width $H \in [64, 128]$; activation GELU/ReLU; dropout 0.1–0.3 on $[h||\bar{m}]$; GraphNorm/LayerNorm; residual connections when dimensions match. Inputs X : degree (scaled), LapPE (irregulars), coordinates/bitstrings (grids/cubes). Xavier init; small weight decay.

18.10 Mini-Proof: Equivariant Policy from SAGE + Nodewise Head

Let $Z = \text{SAGE}(A, X)$. For a fixed t , $S_t = [Z; \tau_t \mathbf{1}; \Pi]$ (positional encodings Π sign-fixed). A nodewise affine head gives $\Theta_t = S_t W + \mathbf{1} b^\top$. For any permutation P consistent with Π , $Z' = PZ$, $S'_t = PS_t$, hence $\Theta'_t = P\Theta_t$. The local unitary lift $C_v(t) = \Phi(\theta_{v,t})$ acts blockwise, preserving end-to-end permutation-covariance.

18.11 Integration with the Temporal Controller and Coins

As in our main architecture, the controller consumes $s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)]$ to emit $\theta_{v,t}$. SAGE’s separation of self/neighbor summaries helps the controller switch regimes: early-time *spread* (heavier weight on $\bar{m}$) versus late-time *focus/phase-align* (heavier weight on h_v). The unitary lift Φ (SU(2) ZYZ for $d_v=2$, exponential/Cayley for $d_v>2$) then ensures $C_v(t) \in U(d_v)$.

One-line summary. GraphSAGE (mean) is an inductive, permutation-equivariant encoder that disentangles self from neighborhood information via $[h_v||\text{mean}(\mathcal{N}(v))]$; in ACPL, this yields stable, degree-robust structural codes z_v that the temporal controller converts into time-conditioned coin parameters, enabling precise, local interference control across all graph families we study.

19 Graph Attention Networks (GAT): Mechanics, Guarantees, and Role in ACPL

19.1 Neighborhood Attention with Multi-Head Aggregation

GAT [32] replaces fixed (mean/degree-normalized) mixing with *learned, content-based* attention over neighbors. With hidden states $h_v^{(\ell)} \in \mathbb{R}^{d_\ell}$, a shared linear map $W^{(\ell)} \in \mathbb{R}^{d_{\ell+1} \times d_\ell}$, and a learnable vector $a^{(\ell)} \in \mathbb{R}^{2d_{\ell+1}}$, the single-head attention at layer ℓ is:

$$\tilde{h}_v^{(\ell)} = W^{(\ell)} h_v^{(\ell)}, \quad \tilde{h}_u^{(\ell)} = W^{(\ell)} h_u^{(\ell)}, \quad (24)$$

$$e_{uv}^{(\ell)} = \text{LeakyReLU}\left(a^{(\ell)\top} [\tilde{h}_u^{(\ell)} || \tilde{h}_v^{(\ell)}]\right), \quad u \in \mathcal{N}(v) \cup \{v\}, \quad (25)$$

$$\alpha_{uv}^{(\ell)} = \frac{\exp(e_{uv}^{(\ell)})}{\sum_{w \in \mathcal{N}(v) \cup \{v\}} \exp(e_{vw}^{(\ell)})} \quad (\text{softmax over the target } v\text{'s incident set}), \quad (26)$$

$$h_{v,\text{head}}^{(\ell+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{uv}^{(\ell)} \tilde{h}_u^{(\ell)}\right). \quad (27)$$

We use H parallel heads, either *concatenated* $h_v^{(\ell+1)} = \parallel_{k=1}^H h_{v,\text{head}=k}^{(\ell+1)}$ (early layers) or *averaged* at the last layer.

Edge features and biases (optional). If edge features e_{uv} are available, we incorporate them as an additive bias in the score (or via a small MLP):

$$e_{uv}^{(\ell)} \leftarrow \text{LeakyReLU}\left(a^{(\ell)\top} [\tilde{h}_u^{(\ell)} || \tilde{h}_v^{(\ell)}] + c^{(\ell)\top} \phi_e(e_{uv})\right).$$

This is useful on weighted/directed graphs and to inject oracle/mark hints in search tasks.

19.2 Why Attention Helps for ACPL

Coins steer *interference*; which neighbors matter changes across the graph and across time. Attention lets the encoder learn *data-dependent* importance weights $\alpha_{uv}^{(\ell)}$ instead of hard-coding degree-normalized mixing. Concretely:

- **Directional emphasis:** near a target or along a preferred manifold of transport, attention can up-weight the relevant ports; the controller then emits coins that amplify these directions.
- **Heterophily/obstacles:** when neighbor features disagree (heterophily) or obstacles create dead-ends, attention down-weights misleading neighbors, preventing the policy from over-smoothing.
- **Phase-aware surrogates:** although GAT consumes *classical* features, its learned weights correlate with structural patterns that translate into phase-alignment choices by the coin head.

19.3 Permutation-Equivariance of GAT

Let P be a permutation on V , with $X' = PX$ and $A' = PAP^\top$. Because (25) uses *shared* $W^{(\ell)}$, $a^{(\ell)}$ and computes $\alpha_{uv}^{(\ell)}$ by a softmax over the set $\mathcal{N}(v) \cup \{v\}$, which is permuted as a whole under P , one has

$$\alpha_{P_u, P_v}^{(\ell)}(A', X') = \alpha_{uv}^{(\ell)}(A, X), \quad \tilde{h}_{P_u}^{(\ell)}(A', X') = \tilde{h}_u^{(\ell)}(A, X),$$

hence the aggregated outputs satisfy $h_{P_v}^{(\ell+1)} = h_v^{(\ell+1)}$ up to the same permutation P . Induction yields $Z' = PZ$, preserving nodewise policy equivariance.

19.4 Integration in Our Policy Pipeline

We instantiate Φ_{GNN} as multi-head GAT:

$$(G, X) \xrightarrow{\text{GAT}} \{z_v\}, \quad s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)] \xrightarrow{\text{Controller}} \theta_{v,t} \xrightarrow{\Phi} C_v(t).$$

Because z_v already encodes *which* neighbors are salient (via α_{uv}), the temporal controller can more easily switch between *spreading* and *focusing* regimes across time.

19.5 Complexity, Kernels, and Stability

Cost. Per head and layer: compute scores e_{uv} on edges ($O(|E|d_{\ell+1})$), softmax by target (segment-softmax on CSR/COO), then weighted SPMM ($O(|E|d_{\ell+1})$). With H heads, cost $\approx O(H|E|d_{\ell+1})$; memory holds edge-wise scores and weights.

Regularization. We use feature dropout and *attention dropout* (on α_{uv}) to prevent head collapse; residuals + LayerNorm/GraphNorm stabilize deeper stacks; clip gradients.

Heads. Early layers: $H = 4$ –8 small heads with *concat*. Final layer: $H = 4$ with *average* for a compact z_v .

19.6 Edge Cases Relevant to Coins

Degree-1 nodes. Including the self-loop in $\mathcal{N}(v) \cup \{v\}$ prevents degenerate softmax; attention usually places mass on the self term, letting the controller keep coin phases stable at leaves (reducing leakage).

Hubs. GAT can learn to *sparsify* influence at hubs by focusing on a few structurally relevant neighbors, which is beneficial before coin-induced splitting.

Irregular degrees. No explicit degree normalization is needed; the softmax per target node automatically normalizes the incoming attention weights.

19.7 Expressivity vs. GCN/SAGE and When to Use GAT

GAT is more flexible than GCN (fixed propagation) and SAGE-mean (uniform neighbor mixing) because it *learns* per-edge importances conditioned on node content. We prefer GAT when:

- local routing requires *selective* emphasis (state transfer around junctions, grid corners, or hypercube subfaces),
- graphs exhibit heterophily or obstacles,
- we want robustness to degree skew without explicit scalars.

We may prefer SAGE/GCN when compute/memory are tightly constrained or when attention gives no measurable gain on a given family.

19.8 Hyperparameters (Defaults for ACPL)

Layers $L \in \{2, 3\}$; heads $H \in \{4, 8\}$; hidden per head $d_{\text{head}} \in \{16, 32\}$ so that $H \cdot d_{\text{head}} \in \{64, 128\}$; LeakyReLU slope 0.2; attention dropout 0.1–0.3; feature dropout 0.1–0.3; residuals + LayerNorm; self-loops included.

19.9 Mini-Proof Sketch: Equivariant Policy with Nodewise Head

Let $Z = \text{GAT}(A, X)$, $S_t = [Z; \tau_t \mathbf{1}; \Pi]$, and nodewise head $\Theta_t = S_t W + \mathbf{1} b^\top$. For any permutation P consistent with Π , we have $Z' = PZ$, thus $S'_t = PS_t$, hence $\Theta'_t = P\Theta_t$. The unitary lift $C_v(t) = \Phi(\theta_{v,t})$ applies locally, maintaining end-to-end permutation-covariance.

19.10 Physics-Intuition: Attention as a Pre-Coin Lens

Attention learns a *soft routing mask* that highlights structural conduits (e.g., along geodesics to j^*) *before* the coin mixes amplitudes. The controller then translates this lens into $C_v(t)$ choices that (i) amplify preferred ports (constructive interference) and (ii) damp detours (destructive interference), improving transfer/search and stabilizing mixing under disorder.

One-line summary. GAT provides a permutation-equivariant, *data-dependent* neighborhood weighting that serves as a learnable, structural lens for coin control; combined with the temporal controller and the unitary lift, it yields precise, time-varying coin policies that adapt to topology, degree, and task context in ACPL.

20 Graph Isomorphism Network (GIN): Mechanics, Expressivity, and Role in ACPL

20.1 Update Rule, Design Intent, and 1-WL Expressivity

GIN [38] was designed to match the discriminative power of the *1-dimensional Weisfeiler–Lehman (1-WL)* graph isomorphism test. Its layer is a strict sum-aggregator followed by an MLP:

$$h_v^{(0)} = x_v, \tag{28}$$

$$h_v^{(\ell+1)} = \text{MLP}^{(\ell)} \left((1 + \epsilon^{(\ell)}) h_v^{(\ell)} + \sum_{u \in \mathcal{N}(v)} h_u^{(\ell)} \right), \quad \ell = 0, \dots, L-1, \tag{29}$$

where $\epsilon^{(\ell)}$ is either a learned scalar or a fixed constant (often 0). Two key choices make GIN maximally expressive among MPNNs aligned with 1-WL:

- **Sum aggregation** (not mean/max): preserves *multiset* information (counts), letting the MLP distinguish neighbor multisets that mean/max would collapse.
- **Injective update** via a sufficiently powerful MLP (two+ layers with nonlinearity), which can approximate injective functions over multisets of bounded size.

Under mild assumptions, (29) matches 1-WL’s power: if 1-WL distinguishes two graphs/nodes, a sufficiently wide/deep GIN can learn embeddings that separate them.

20.2 Permutation-Equivariance and Stability

Because the parameters of $\text{MLP}^{(\ell)}$ and $\epsilon^{(\ell)}$ are *shared* across nodes and the neighbor aggregation is a *sum* over an unordered set, GIN is permutation-equivariant: relabeling G by a permutation P permutes $\{h_v^{(\ell)}\}$ and hence the outputs in the same way (cf. our theorem). Residual/LayerNorm/GraphNorm are commonly added to stabilize deeper stacks.

20.3 Relation to ACPL (Why GIN Helps Coin Control)

For ACPL we need node embeddings that tightly reflect *local structure* (degree, motifs, symmetry classes) because coin choices $C_v(t)$ must choreograph interference relative to this structure:

- **Counting sensitivity:** the sum aggregator preserves counts (e.g., number of neighbors of each type), which influences how amplitude can split at v and which phase patterns $C_v(t)$ should implement.
- **Motif discrimination:** GIN distinguishes neighborhood multisets better than mean/max; near junctions/corners/bottlenecks it yields distinct z_v that the controller can map to different coin regimes (spread vs. focus).
- **Simplicity and speed:** no attention softmax or degree scalars; kernels reduce to SPMM + MLP, which is fast and memory-friendly in our differentiable simulator loop.

Empirically, GIN is a strong default encoder for our tasks; attention (GAT) becomes useful when selective neighbor emphasis is crucial (Sec. 19).

20.4 Edge Features, Self-Loops, and Degree Handling

We always include self-loops in the sum (so v 's own state participates via $(1 + \epsilon)$). If edge features e_{uv} are present, a simple and injective variant augments the message as

$$\sum_{u \in \mathcal{N}(v)} \phi_e(e_{uv}) \odot h_u^{(\ell)} \quad \text{or} \quad \sum_{u \in \mathcal{N}(v)} \text{MLP}_e(h_u^{(\ell)}, e_{uv}),$$

keeping the outer aggregation a sum. Degrees need no explicit normalization (unlike GCN); the MLP can learn degree-aware responses from the summed magnitude and from degree included in x_v .

20.5 Complexity and Kernels

Each layer performs one SPMM-like sum over edges ($O(|E|d_\ell)$) plus an MLP on nodes ($O(|V|d_\ell d_{\ell+1})$). There is no edge-wise softmax, so memory footprint is modest; this is attractive at larger T where backprop-through-time multiplies activation storage.

20.6 Integration in Our Policy

We instantiate Φ_{GNN} as GIN (2–3 layers, width 64–128), producing z_v :

$$(G, X) \xrightarrow{\text{GIN}} \{z_v\}, \quad s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)] \xrightarrow{\text{GRU/Transformer}} \theta_{v,t} \xrightarrow{\Phi} C_v(t).$$

Because z_v is a *count- and multiset-aware* summary of the L -hop neighborhood, the temporal controller can map consistent structural roles (leaf, corner, hub, bridge) to *time-varying* coin patterns that shape interference appropriately.

20.7 Hyperparameters (Defaults for ACPL)

Layers $L \in \{2, 3\}$; hidden width $d \in \{64, 128\}$; $\epsilon^{(\ell)}$ learned (per layer) or fixed at 0; 2-layer MLPs with hidden d and ReLU/GELU; residual connections; GraphNorm/LayerNorm; feature dropout 0.1–0.3; DropEdge optional on large, sparse graphs.

20.8 When to Prefer GIN vs. GCN/SAGE/GAT

- **Prefer GIN** when structure counting and motif distinction matter (irregular graphs; corners/junctions on grids; hypercube subface roles), or when you want a strong, simple baseline with good expressivity and speed.
- **Prefer GAT** when selective, content-based neighbor emphasis is crucial (Sec. 19).
- **Prefer SAGE-mean/GCN** when compute/memory are tight or homophily is strong and fixed normalization suffices (Secs. 18, 17).

20.9 Physics Intuition: Sum as “Amplitude Capacity” Proxy

The raw sum $\sum_{u \in \mathcal{N}(v)} h_u$ correlates with how many channels can carry amplitude out of v and the composition of those channels. The MLP then learns to translate this proxy for *outgoing capacity and pattern* into coin parameters $\theta_{v,t}$ that (i) widen/narrow the spread and (ii) set phases that favor constructive interference along desired routes.

One-line summary. GIN gives a permutation-equivariant, multiset-sensitive encoder with 1-WL expressivity; in ACPL it furnishes robust structural embeddings that a temporal controller converts into time-varying coin parameters, delivering precise, topology-aware interference control with low computational overhead.

21 Principal Neighbourhood Aggregation (PNA): Mechanics, Degree-Scaling, and Role in ACPL

21.1 Motivation and Design

PNA [39] targets graphs with *skewed or diverse degree distributions* where single-aggregator GNNs (sum/mean/max alone) either saturate on hubs or under-represent sparse neighborhoods. PNA combines:

1. multiple **aggregators** to capture complementary neighbourhood statistics; and
2. learnable **degree-scalers** that modulate the aggregated signal based on the target node’s degree.

This yields a richer, degree-aware neighbourhood summary before passing it through an MLP update.

21.2 Layer Mechanics

Let $h_v^{(\ell)}$ be node states and (optionally) e_{uv} edge features. A typical PNA layer proceeds:

$$\textbf{Messages: } m_{u \rightarrow v}^{(\ell)} = \phi_m^{(\ell)}(h_u^{(\ell)}, h_v^{(\ell)}, e_{uv}), \quad (30)$$

$$\textbf{Multi-aggregation: } \forall a \in \mathcal{A}: \text{Agg}_a(\mathcal{N}(v)) = \text{AGG}_a\{m_{u \rightarrow v}^{(\ell)} : u \in \mathcal{N}(v)\}, \\ \text{with } \mathcal{A} \subseteq \{\sum, \text{mean}, \text{max}, \text{min}, \text{std}\}, \quad (31)$$

$$\textbf{Degree scaling: } \forall r \in \mathcal{R}: \widehat{\text{Agg}}_{a,r}(v) = s_r(d_v) \odot \text{Agg}_a(\mathcal{N}(v)), \quad (32)$$

$$\textbf{Concatenate \& update: } \bar{m}_v^{(\ell)} = \parallel_{a \in \mathcal{A}, r \in \mathcal{R}} \widehat{\text{Agg}}_{a,r}(v), \quad h_v^{(\ell+1)} = U^{(\ell)}([h_v^{(\ell)} \parallel \bar{m}_v^{(\ell)}]). \quad (33)$$

Here $s_r(d_v)$ are scalar (or channel-wise) *degree-scalers*; $\odot$ is broadcasted multiply; $\parallel$ is concatenation. The update $U^{(\ell)}$ is typically a 2-layer MLP (with nonlinearity), often wrapped with residual, normalization, and dropout.

Degree-scalers (examples). PNA proposes simple monotone functions calibrated using corpus statistics (e.g., mean degree $\bar{d}$):

$$s_{\text{id}}(d) = 1, \quad s_{\text{amp}}(d) = \log\left(\frac{d+1}{\bar{d}}\right), \quad s_{\text{att}}(d) = \log\left(\frac{\bar{d}}{d+1}\right), \quad s_{\text{inv}}(d) = \frac{\bar{d}}{d+1}.$$

These amplify or attenuate aggregated messages for hubs vs. leaves, stabilizing feature magnitudes across degrees.

21.3 Why Multi-Aggregation Helps

Different aggregators capture different neighbourhood signals:

- **Sum** preserves multiplicity (counts);
- **Mean** removes degree bias;
- **Max/Min** capture extremal (edge-case) neighbours;
- **Std** encodes dispersion/heterogeneity of messages.

Concatenating them allows the MLP to approximate expressive injective maps over multisets with varying cardinalities, while degree-scalers re-normalize for stability.

21.4 Permutation-Equivariance and Expressivity

All components are permutation-invariant wrt. neighbour ordering, and parameters are shared across nodes/edges; hence PNA is permutation-equivariant (as required by our theorem). PNA remains within the MPNN/1-WL envelope theoretically, but empirically attains strong accuracy on graphs with *heterogeneous degrees* or *heterophilous signals* by providing a richer sufficient statistic of the neighbourhood.

21.5 Relation to ACPL (Why PNA is Useful for Coin Control)

In ACPL, coin choices must reflect local *branching structure* and *heterogeneity* to choreograph interference:

- **Degree-aware spreading:** At a hub (large d_v), the coin often needs to *temper* spreading (to avoid amplitude dilution) or set phases to favour a subset of ports; at leaves (small d_v), it can *boost* transmission. PNA’s $s_r(d_v)$ exposes degree to the encoder in a multiplicative, stable way.
- **Heterogeneous neighbourhoods:** Std/max aggregators flag mixed neighbour types (e.g., boundary/corner vs. interior in grids), guiding the controller to distinct phase policies.
- **Irregular graphs:** On ER/small-world/ d -regular mixtures (Sec. 38.2), PNA normalizes message magnitudes across degrees, which improves training stability and sample efficiency for our time-conditioned policy.
- **Regular graphs (grids/hypercubes):** When d_v is constant, degree-scalers collapse to constants; PNA reduces to a multi-aggregator encoder that can still exploit dispersion (std) and extremal signals for finer structural cues (e.g., grid corners/edges).

21.6 Complexity and Kernels

Let $|\mathcal{A}|$ be the number of aggregators and $|\mathcal{R}|$ the number of scalars. The edge-wise compute is still linear in $|E|$:

$$\text{cost per layer} \approx O(|E| d_\ell) \quad (\text{messages}) + O(|V| |\mathcal{A}| |\mathcal{R}| d_\ell) \quad (\text{concat+MLP}).$$

In practice, aggregations are implemented via fused SPMM/segment-reduce kernels; the constant factor grows with $|\mathcal{A}| |\mathcal{R}|$ but remains modest (e.g., $|\mathcal{A}|=4$, $|\mathcal{R}|=3$).

21.7 Integration in Our Policy

We may instantiate Φ_{GNN} as PNA (2–3 layers, hidden 64–128), then proceed with the same controller/coin head:

$$(G, X) \xrightarrow{\text{PNA}} \{z_v\}, \quad s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)] \xrightarrow{\text{GRU/Transformer}} \theta_{v,t} \xrightarrow{\Phi} C_v(t).$$

Because z_v now encodes degree-aware, multi-statistic neighbourhood summaries, the temporal controller can map hubs vs. leaves vs. boundaries to *different time-varying* coin behaviours (e.g., early-time broad spread at interiors; late-time focusing and phase alignment near targets).

21.8 Hyperparameters (Defaults for ACPL)

$\mathcal{A} = \{\text{sum, mean, max, std}\}$; $\mathcal{R} = \{\text{id, amp, att}\}$ (calibrated by $\bar{d}$ on the *training* set only to avoid leakage); $L \in \{2, 3\}$; hidden width 64–128; two-layer MLP per update with ReLU/GELU; residual + GraphNorm/LayerNorm; feature dropout 0.1–0.3; DropEdge optional on very high-degree graphs.

21.9 When to Prefer PNA vs. GIN/GCN/SAGE/GAT

- **Prefer PNA** when degrees vary widely or heterophily is present; degree-scalers stabilize training and improve calibration of z_v across roles (hubs/leaves/bridges).
- **Prefer GIN** for a strong, simple 1-WL-power baseline with multiset sensitivity but no degree-scalers (Sec. 20).
- **Prefer GAT** if selective neighbour emphasis is critical (Sec. 19); PNA and GAT can also be combined (attention in ϕ_m , PNA in aggregation).
- **Prefer GCN/SAGE-mean** under tight compute budgets or highly homophilous graphs where fixed normalization suffices (Secs. 17, 18).

21.10 Physics Intuition: Degree as “Channel Count” Normalizer

The degree d_v approximates the number of *outgoing channels* for amplitude at v . PNA’s scalars act like a physically meaningful normalization of *message energy* per channel, preventing hubs from swamping the update and allowing the controller to choose coins that *either compress or expand* the outgoing superposition in a degree-aware manner.

One-line summary. PNA furnishes a permutation-equivariant, multi-statistic and degree-aware encoder that remains linear in $|E|$ yet captures heterogeneous neighbourhood structure; in ACPL it provides robust z_v embeddings that let the temporal controller emit coin parameters tuned to hubs, leaves, and boundaries, improving interference choreography across irregular graphs.

22 Edge Features and Degrees: Where and How They Enter

In ACPL, *degrees* d_v and *edge features* e_{uv} influence (i) the graph encoder (message passing mechanics), (ii) positional/spectral encodings, and (iii) the control space itself (coin dimension d_v and parameterization). This section makes those pathways explicit.

22.1 Degrees in Graph Structure, Features, and Coins

Combinatorial role. For each vertex v , the degree $d_v = |\mathcal{N}(v)|$ determines the *local coin dimension* and thus the control manifold:

$$C_v(t) \in U(d_v), \quad \text{Degree } 2 \Rightarrow \text{SU}(2) \text{ with 3 parameters,} \quad \text{Degree } d_v > 2 \Rightarrow \exp(K_{v,t}), \quad K_{v,t}^\dagger = -K_{v,t}.$$

Hence d_v directly sets the output size p of the coin head and the block size of $C_t = \bigoplus_v C_v(t)$.

Feature role (node inputs). We include d_v explicitly in the node features x_v :

$$x_v = \left[\underbrace{d_v}_{\text{degree}}, \underbrace{\pi(\text{pos}_v)}_{\text{positional/spectral}}, \underbrace{\text{coords/bitstrings}}_{\text{grids/cubes (if any)}} \right] \in \mathbb{R}^F.$$

Feeding d_v helps the encoder learn degree-aware embeddings z_v , important on irregular graphs.

Spectral role (Laplacian). Degree enters the (normalized) Laplacian $L = D - A$ or $L_{\text{sym}} = I - D^{-1/2}AD^{-1/2}$ used for Laplacian positional encodings (LapPE). Thus, even if d_v were not concatenated into x_v , it would still shape the eigenvectors $\phi_k(v)$ that enter $\pi(\text{pos}_v)$.

22.2 Degrees inside Message Passing

GCN normalization. In GCN-like mixing, the renormalized adjacency $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ with $\tilde{A} = A + I$ and $\tilde{D}_{vv} = d_v + 1$ yields

$$H^{(\ell+1)} = \sigma(\hat{A} H^{(\ell)} W^{(\ell)}), \quad \hat{A}_{uv} = \frac{\tilde{A}_{uv}}{\sqrt{(d_u + 1)(d_v + 1)}}.$$

Thus d_u, d_v scale messages multiplicatively, preventing hubs from dominating and stabilizing variance layer-to-layer.

SAGE-mean. GraphSAGE-mean uses $\bar{m}_v = \frac{1}{d_v} \sum_{u \in \mathcal{N}(v)} h_u$, i.e., explicit division by d_v ; the degree appears as a normalizer in the aggregator (and again implicitly if $d_v \in x_v$).

PNA degree-scalers. PNA introduces *learnable* degree-aware gains $s_r(d_v)$ (Sec. 21):

$$\widehat{\text{Agg}}_{a,r}(v) = s_r(d_v) \odot \text{AGG}_a\{m_{u \rightarrow v}\}, \quad s_r \in \{\text{id}, \log \frac{d_v+1}{d}, \log \frac{\bar{d}}{d_v+1}, \frac{\bar{d}}{d_v+1}\}.$$

This explicitly modulates the magnitude of neighbourhood statistics conditioned on d_v .

GAT with degree priors (optional). Standard GAT does not directly use d_v in attention scores, but we may add a *degree channel* to the key/query features, or apply a degree-based temperature to the softmax (e.g., $\alpha_{uv} \propto \text{softmax}(a^\top [Wh_u \| Wh_v] / \tau(d_v))$), which stabilizes attention on hubs.

Directed graphs (in/out-degree). If the graph is directed, maintain $d_v^{\text{in}}, d_v^{\text{out}}$ and use them in the chosen normalization (e.g., left or right normalization $D^{-1}A$ or AD^{-1}), and (optionally) as two scalar features in x_v .

22.3 Edge Features: Definitions and Insertion Points

What is an edge feature? An edge feature $e_{uv} \in \mathbb{R}^{d_e}$ carries local relational information. In our DTQW context, typical edge features include:

- **Geometric/structural:** edge type (horizontal/vertical on a grid), lattice orientation, distance (for weighted graphs), or port index pairs.
- **Physical parameters:** static edge phases ϕ_{uv} for disorder, time-varying noise marks, or coupling strengths.
- **Learned encodings:** embeddings of relation types on heterogeneous graphs.

Where do they enter? They enter (i) the *message function* and (ii) sometimes the *aggregator*:

$$m_{u \rightarrow v}^{(\ell)} = \phi_m^{(\ell)}(h_u^{(\ell)}, h_v^{(\ell)}, e_{uv}), \quad \bar{m}_v^{(\ell)} = \text{AGG}_{-u} \in \mathcal{N}(v) \ m_{u \rightarrow v}^{(\ell)},$$

with $\phi_m^{(\ell)}$ an MLP (or attention kernel) that can condition on e_{uv} . In attention layers (GAT/ECC), e_{uv} can also modulate the edge weight:

$$\alpha_{uv} = \text{softmax}_{u \in \mathcal{N}(v)} \left(\psi^\top [Wh_u \| Wh_v \| Ee_{uv}] \right), \quad h_v^{(\ell+1)} = \sigma \left(\sum_u \alpha_{uv} Wh_u \right).$$

In **edge-conditioned conv** (ECC), e_{uv} parametrizes the edge-specific kernel:

$$m_{u \rightarrow v}^{(\ell)} = \mathcal{K}_\Theta(e_{uv}) h_u^{(\ell)} \quad \text{with} \quad \mathcal{K}_\Theta : \mathbb{R}^{d_e} \rightarrow \mathbb{R}^{d \times d}.$$

Edge weights vs. features. If e_{uv} is a scalar weight $w_{uv} > 0$, many layers support *weighted* aggregation:

$$\bar{m}_v^{(\ell)} = \sum_u w_{uv} m_{u \rightarrow v}^{(\ell)} \quad \text{or} \quad H^{(\ell+1)} = \sigma(\hat{W} H^{(\ell)} W^{(\ell)}), \quad \hat{W} = D_w^{-1/2} W D_w^{-1/2},$$

where $W = [w_{uv}]$ and D_w is its degree matrix.

22.4 Why Edge Features Matter for ACPL

Encoding physical asymmetries. On grids, ($e_{uv} = \text{dir}$) distinguishes horizontal vs. vertical ports; the encoder can learn different phase tendencies for those directions, which the controller then refines over time.

Disorder-aware embeddings. For robust transport, static phases ϕ_{uv} entering e_{uv} let the encoder/temporal controller *see* the disordered landscape and emit coins that compensate (e.g., constructive paths that avoid strong backscattering).

Directed control policies. If the shift pairing is not symmetric (directed arcs), encoding direction and port identities in e_{uv} helps the policy choose anisotropic local coins that favour forward progress toward targets.

22.5 Putting Degrees and Edge Features Together

Unified message template. We use a single, expressive message form across layers:

$$m_{u \rightarrow v}^{(\ell)} = \text{MLP}_m^{(\ell)} \left(\underbrace{h_u^{(\ell)} - h_v^{(\ell)}}_{\text{relative}}, \underbrace{h_u^{(\ell)}}_{\text{absolute}}, \underbrace{e_{uv}}_{\text{edge}}, \underbrace{\gamma(d_u), \gamma(d_v)}_{\text{degree encodings}} \right),$$

where $\gamma(\cdot)$ is either the raw scalar degree or a learned embedding (e.g., $\gamma(d) = \text{PE}(d)$). Relative terms aid stability; absolute terms allow residual information to pass.

Aggregation with degree-awareness. Depending on the chosen encoder:

$$\bar{m}_v^{(\ell)} = \sum_u m_{u \rightarrow v}^{(\ell)} \quad (\text{GIN/GCN-like, possibly with norm}) \quad \text{or} \quad \bar{m}_v^{(\ell)} = \left\|_{a \in \mathcal{A}, r \in \mathcal{R}} s_r(d_v) \text{AGG}_a \{m_{u \rightarrow v}^{(\ell)}\},$$

for PNA; for attention, $\bar{m}_v^{(\ell)} = \sum_u \alpha_{uv} m_{u \rightarrow v}^{(\ell)}$.

Update & residual.

$$h_v^{(\ell+1)} = \text{MLP}_u^{(\ell)}([h_v^{(\ell)} \| \bar{m}_v^{(\ell)}]),$$

with residual, normalization (GraphNorm/LayerNorm), and dropout.

22.6 Downstream Effects on Coin Policy

From z_v to $\theta_{v,t}$. Because z_v encodes (i) local branching via d_v , (ii) anisotropy/noise via e_{uv} , and (iii) positional context via LapPE, the temporal controller can emit *degree- and edge-aware* coin parameters:

$$\theta_{v,t} = \text{Head}(\text{Controller}([z_v; \tau_t; \pi(\text{pos}_v)])),$$

which Φ lifts to $C_v(t)$ with the correct dimension d_v . Intuitively, hubs get different (often more conservative or phase-selective) coins than leaves; edges with disorder/anisotropy induce compensating phase patterns.

22.7 Implementation Notes (Tensors and Caching)

- **Degrees:** store d_v as int64 (graph) and float32 (features). For directed graphs, keep both $d_v^{\text{in}}, d_v^{\text{out}}$.
- **Edge features:** e_{uv} is a $[|E|, d_e]$ float32 tensor; include categorical parts as learned embeddings (e.g., orientation), real parts as scalars (weights, phases ϕ_{uv}).
- **Normalization:** precompute $\tilde{D}^{-1/2}$ (or $D_w^{-1/2}$) per graph size and cache by topology hash; safe for train/val/test splits with leak-free seeds.
- **Numerics:** if phases ϕ_{uv} are used, keep them in $[-\pi, \pi]$ and expose both $\sin \phi_{uv}$ and $\cos \phi_{uv}$ as features to remove 2π -wrap discontinuities.

22.8 Sanity Checks

1. **Degree conservation:** $\sum_u \tilde{A}_{uv} = d_v + 1$ (post self-loop) and $SS^\dagger = I$ for the shift S constructed from the port map.
2. **Permutation tests:** relabel nodes; d_v reindexes, e_{uv} permutes; the encoder output permutes identically, metrics unchanged.
3. **Ablations:** removing d_v from x_v or disabling degree-scalers in PNA should degrade performance on irregular graphs; removing e_{uv} should reduce robustness under disorder tasks.

One-line summary. Degrees d_v set the *coin dimension* and enter encoder *normalization and scaling*, while edge features e_{uv} carry anisotropy/disorder into the *message* and (optionally) the *attention/weighting*; together they yield structurally faithful embeddings z_v that the time-aware policy maps to physically valid, node-specific unitary coins $C_v(t)$.

23 Positional Encodings (PE) Inside Message Passing

Why PE? Message passing with purely local aggregation cannot distinguish nodes in *globally symmetric* roles (automorphisms): two nodes with isomorphic k -hop neighborhoods may remain indistinguishable to MPNNs (1-WL limitation). *Positional encodings* (PE) inject *global* or *long-range* structure into node features, breaking these symmetries while *preserving permutation-equivariance*: if nodes are relabeled, PE values permute accordingly, so the GNN’s outputs permute in lockstep (cf. our equivariance theorem).

23.1 PE Families We Use

(A) Laplacian Positional Encodings (LapPE). Let A be adjacency, $D = \text{diag}(d_v)$, and $L_{\text{sym}} = I - D^{-1/2}AD^{-1/2}$ the symmetric normalized Laplacian. Compute the K smallest nontrivial eigenpairs

$$L_{\text{sym}} \phi_k = \lambda_k \phi_k, \quad 0 = \lambda_0 < \lambda_1 \leq \dots \leq \lambda_K, \quad \phi_k \in \mathbb{R}^{|V|}.$$

Define the LapPE at vertex v as $\pi_{\text{Lap}}(v) = (\phi_1(v), \dots, \phi_K(v)) \in \mathbb{R}^K$. These are graph analogues of Fourier modes: low-frequency eigenvectors encode coarse geometry; higher frequencies encode finer structure. We *fix eigenvector signs* (Sec. 23.5) to ensure determinism.

(B) Random-Walk Positional Encodings (RWPE). Let $P = D^{-1}A$ be the (row-stochastic) random walk matrix. Define p -step landing distributions from node v :

$$r_p(v) = e_v^\top P^p \in \mathbb{R}^{|V|}, \quad p \in \{1, \dots, K\},$$

and compress them to a low-dimensional feature via moments or projections. A simple, permutation-consistent option is to form *RW moments* $\mu_p(v) = \sum_u (P^p)_{vu}$ over structural bins or to project onto Laplacian eigenvectors: $\tilde{r}_p(v) = (\langle r_p(v), \phi_1 \rangle, \dots, \langle r_p(v), \phi_K \rangle)$, yielding RWPE $\pi_{\text{RW}}(v) = [\tilde{r}_1(v) \parallel \dots \parallel \tilde{r}_{K'}(v)]$.

(C) Shortest-Path / Anchor Distance Encodings (SPD/Anchor). Choose M *anchors* $\mathcal{A} = \{a_1, \dots, a_M\}$ (e.g., highest-degree nodes or k -centers). Define

$$\pi_{\text{SPD}}(v) = (d_{\text{sp}}(v, a_1), \dots, d_{\text{sp}}(v, a_M)) \in \mathbb{R}^M,$$

optionally normalized/embedded via RBFs: $\text{RBF}_\ell(v) = \exp\{-\gamma(d_{\text{sp}}(v, a_\ell) - \mu_\ell)^2\}$.

(D) Geometry/Bitstring Coordinates (structured graphs). On grids, use normalized integer coordinates $\pi_{\text{coord}}(x, y) = (x/L, y/L)$; on hypercubes, use bitstrings $b \in \{0, 1\}^n$ and Hamming weight $|b|$ (optionally a learned embedding of bits).

(E) Heat/Wave Kernel Signatures (HK/WKS; optional). Form heat features $h_t(v) = [e^{-tL}]_{vv}$ for t in a log-spaced set, or spectral wavelets $\psi_s(v) = [g(sL)\delta_v](v)$ with $g(\cdot)$ a band-pass. These are robust to small topological perturbations.

In practice, we adopt: LapPE (irregulars), coordinates (grids), bitstrings (cubes), with optional RWPE/anchor features on highly symmetric graphs to break residual symmetries.

23.2 How PE Enters Message Passing

Let x_v be the raw node features (degree, etc.). We form the *augmented* feature

$$\tilde{x}_v = [x_v ; \underbrace{\pi_{\text{PE}}(v)}_{\text{absolute PE}}] \in \mathbb{R}^{F+K_{\text{PE}}}.$$

This vector is used as the *layer-0 state* $h_v^{(0)} = \tilde{x}_v$ and is threaded through all layers. We employ three complementary injection points:

(1) Concatenation at the node state. At every layer update,

$$h_v^{(\ell+1)} = U^{(\ell)}\left([h_v^{(\ell)} \parallel \pi_{\text{PE}}(v)], \bar{m}_v^{(\ell)}\right),$$

so that PE is always available to the update MLP. This preserves permutation-equivariance: a node relabeling permutes $(h_v^{(\ell)}, \pi_{\text{PE}}(v))$ jointly.

(2) Edge-aware messages via PE differences. We enrich messages with *relative* PE:

$$m_{u \rightarrow v}^{(\ell)} = \phi_m^{(\ell)}\left(h_u^{(\ell)}, h_v^{(\ell)}, e_{uv}, \underbrace{\pi_{\text{PE}}(u) - \pi_{\text{PE}}(v)}_{\text{relative PE}}\right).$$

Relative differences are translation-invariant (for coordinate-like PEs) and help disambiguate directions (e.g., “to the right” vs. “to the left” on grids).

(3) **Attention biasing (GAT/PNA-Attn).** For attention layers, we inject PE into the *logit*:

$$\alpha_{uv} = \text{softmax}_{u \in \mathcal{N}(v)} \left(a^\top [Wh_u \| Wh_v] + b^\top [\Delta\pi_{uv}] \right), \quad \Delta\pi_{uv} = \pi_{\text{PE}}(u) - \pi_{\text{PE}}(v).$$

This implements a *directional prior*: neighbors whose relative position matches the task’s flow (e.g., toward a target) get higher attention weights.

23.3 Why PE Helps ACPL (Control-Theoretic View)

- **Breaking symmetries:** In highly regular graphs (grids, hypercubes), many nodes are automorphic. Without PE, identical messages/updates yield identical embeddings, so the policy cannot emit differentiated local coins near, say, the target j^* . PE provides a *reference frame* so the controller can break symmetry to focus amplitude.
- **Global phase choreography:** The learned coin schedule must coordinate interference across long distances. LapPE/RWPE encode global geometry/spectral modes that correlate with constructive paths, helping the policy time constructive interference (e.g., aligning phase fronts toward j^*).
- **Generalization across sizes:** Normalized coordinates (grids) and low-frequency Laplace eigenfunctions vary smoothly with graph size. Using $\tau_t = t/T$ and scale-stable PE lets schedules transfer across T and $|V|$.

23.4 PE for Different Encoders (GCN/SAGE/GAT/GIN/PNA)

GCN.

$$H^{(\ell+1)} = \sigma(\hat{A} [H^{(\ell)} \| \Pi] W^{(\ell)}), \quad \Pi = [\pi_{\text{PE}}(v)]_{v \in V}.$$

Here PE is concatenated to node states before mixing by $\hat{A}$. Relative PE can be injected by precomputing B with $B_{uv} = b^\top (\pi(u) - \pi(v))$ and adding it to the logit prior to normalization (i.e., biased GCN).

GraphSAGE (mean).

$$\bar{m}_v = \frac{1}{d_v} \sum_u \phi_m(h_u, [\pi(u)]), \quad h_v^{(\ell+1)} = \phi_u([h_v, \pi(v)], \bar{m}_v).$$

PE enters both the neighbor transform and the self-update; relative PE can be included via $\phi_m(h_u, [\pi(u) - \pi(v)])$.

GAT.

$$\alpha_{uv} = \text{softmax}_u \left(a^\top [Wh_u \| Wh_v] + b^\top [\pi(u) - \pi(v)] \right), \quad h_v^{(\ell+1)} = \sigma \left(\sum_u \alpha_{uv} Wh_u \right).$$

This creates *direction-sensitive attention* while preserving permutation-equivariance (the subtraction uses aligned coordinates/eigenfunctions).

GIN.

$$h_v^{(\ell+1)} = \text{MLP} \left((1 + \epsilon) [h_v \| \pi(v)] + \sum_u [h_u \| \pi(u)] \right).$$

Concatenating PE preserves the strong injectivity properties of GIN and allows sum-aggregation to leverage absolute/global cues.

PNA.

$$\widehat{\text{Agg}} = \left\|_{a,r} s_r(d_v) \cdot \text{AGG}_a(\psi([m_{u \rightarrow v}, \Delta\pi_{uv}])) \right\|, \quad h_v^{(\ell+1)} = \phi_u([h_v, \pi(v)], \widehat{\text{Agg}}).$$

PE difference $\Delta\pi_{uv}$ enters the message transform ψ ; degree-scalers $s_r(d_v)$ retain robustness to heterogeneity.

23.5 Implementation Details & Pitfalls

Sign ambiguity (LapPE). Eigenvectors are defined up to a global sign. We *fix signs deterministically*: for each ϕ_k , find index $i^* = \arg \max_v |\phi_k(v)|$ and enforce $\phi_k(i^*) > 0$; alternatively, choose sign so that $\sum_v \phi_k(v) \geq 0$.

Multiplicity and subspace alignment. If λ_k has multiplicity $m > 1$, any orthonormal basis of the eigenspace is valid. Across re-samplings/sizes, basis vectors may rotate. Remedies: (i) use only *low multiplicity* bands (small K); (ii) apply a Procrustes alignment to a *reference* PE (same family/size) before concatenation; (iii) prefer RWPE/anchor PEs when multiplicities are severe (grids, hypercubes).

Normalization and scaling. We standardize each PE channel over nodes: zero mean, unit variance. For coordinates, use $[0, 1]$ normalization. For RW moments, log/Box–Cox transforms stabilize heavy tails.

Stability to perturbations. Laplacian eigenvectors may be unstable to small topology changes at high frequencies. Restrict to low K (e.g., $K \in [8, 16]$) and complement with RWPE/anchor encodings.

Complexity. LapPE: K smallest eigenvectors via Lanczos in $O(K|E|)$ per graph; cache per (family, size). RWPE: repeated sparse matvecs P^p ($\#steps \times |E|$), cacheable. Anchor distances: multi-source BFS/SSSP in $O(M|E|)$.

23.6 How PE Interacts with the Temporal Controller and Coins

At time t , the controller input is

$$s_{v,t} = [z_v ; \tau_t ; \pi_{\text{PE}}(v)].$$

PE provides a *global phase map* the controller uses to decide *when to spread* (Fourier-like coins early) and *when to focus* (rotation phases that phase-align paths near j^* late). In grids, coordinate PE enables directionally distinct scheduling (e.g., different ZYZ angles for x vs. y ports). In irregulars, LapPE identifies bottlenecks/communities that the policy can *open* (larger R_y) or *damp* (smaller rotations) to shape interference.

23.7 Sanity Checks and Ablations

1. **Permutation check:** relabel nodes π ; verify $\pi(\pi_{\text{PE}})$ and embeddings Z permute identically; metrics unchanged.
2. **No-PE ablation:** remove PE; expect degraded performance on symmetric graphs (grids/cubes), poorer transfer/search.
3. **Relative-only ablation:** use only $\Delta\pi_{uv}$ (drop absolute $\pi(v)$); expect less gain for tasks requiring absolute targets but still improvements in directed attention.
4. **PE-alignment stress:** perturb topology slightly; LapPE-only may wobble; LapPE+RWPE stays stable.

One-line summary. PE furnishes a permutation-consistent global reference frame (spectral, random-walk, or geometric) that flows into messages, attention logits, and updates; this lets the GNN encode *where* a node sits in the graph so the time-aware policy can emit *physically coherent, location-aware* coin schedules $C_v(t)$ that choreograph interference toward the task objective.

24 Stabilizers (Depth, Smoothing, Squashing)

Our policy encoder (GNN + temporal controller) must train stably at moderate depth and across horizons T . Three intertwined phenomena govern stability: *depth* (vanishing/exploding signals through stacked layers/time), *over-smoothing* (node embeddings become indistinguishable), and *over-squashing* (exponentially many distant signals are compressed through few edges). We summarize mechanisms, diagnostics, and remedies we *actually use* in ACPL.

24.1 Depth: gradient flow across layers and time

Mechanism. Stacked message passing composes linear-like mixing with nonlinear updates: $H^{(\ell+1)} = \sigma(\text{Mix}^{(\ell)}(H^{(\ell)})W^{(\ell)})$. Backprop multiplies Jacobians layer-by-layer; poor conditioning and repeated contractions cause vanishing/exploding gradients. In our time-conditioned policy, a GRU/Transformer introduces a second chain in t .

Stabilizers for depth.

- **Residual/skip connections.** For each layer,

$$\tilde{H}^{(\ell+1)} = \text{Norm}(\text{Mix}(H^{(\ell)})W^{(\ell)}), \quad H^{(\ell+1)} = \tilde{H}^{(\ell+1)} + H^{(\ell)}.$$

Residuals give Jacobians close to identity, easing gradient flow (empirically crucial $L \in \{2, 3, 4\}$).

- **Normalization layers.** BatchNorm/LayerNorm/GraphNorm on node states: $H^{(\ell)} \leftarrow \text{Norm}(H^{(\ell)})$. GraphNorm re-centers per-graph, reducing distribution shift across sampled episodes.

- **Activation choice.** Use GELU/PReLU (leak mitigates dead neurons). We avoid very sharp activations in the encoder.
- **Weight init and widths.** Xavier/KAIMING, widths $H \in [64, 128]$. Deeper $\Rightarrow$ narrower is avoided; instead we keep L small and H moderate.
- **Temporal controller gating.** The GRU’s gates regulate information flow in t . We clip controller gradients (ℓ_2 clip, e.g., 1.0–2.0) and use $\tau_t = t/T$ so time scaling is normalized across horizons.

24.2 Over-smoothing: why embeddings collapse and how we prevent it

Mechanism (spectral view). GCN-like propagation can be written $H^{(\ell+1)} = \sigma(\hat{A}H^{(\ell)}W^{(\ell)})$. Ignoring nonlinearity and weights, repeated multiplication by $\hat{A}$ yields $\hat{A}^L X \rightarrow$ projection onto the low-frequency eigenspace of $\hat{A}$; as L grows, node embeddings align and *become indistinguishable*.

Diagnostics. (i) *Embedding contrast* drops: $\frac{1}{|V|(|V|-1)} \sum_{u \neq v} \|h_u^{(L)} - h_v^{(L)}\|_2$, (ii) *Dirichlet energy* $E(H) = \sum_{(u,v) \in E} \|h_u - h_v\|_2^2$ collapses toward 0, (iii) downstream linear probe on $H^{(L)}$ cannot recover structural distinctions present in X .

Stabilizers against over-smoothing.

- **Shallow stacks + residuals.** Use $L \in \{2, 3\}$ with residuals. Early layers retain high-frequency content; residuals preserve identity.
- **Jumping Knowledge (JK).** Read out from a *concatenation* (or max) of all $H^{(\ell)}$: $z_v = \text{JK}(h_v^{(0)}, \dots, h_v^{(L)})$ so that high-frequency features survive to the head.
- **Decoupled propagation (APPNP/PPNP-style).** Compute transformed features once, then propagate with a personalized PageRank (PPR) mixer,

$$H^{(\text{prop})} = (1 - \alpha) \sum_{k=0}^K \alpha (I - \alpha \hat{L})^k H^{(0)} W,$$

reducing repeated nonlinearity-induced contractions.

- **Normalization targeted to graphs.** PairNorm/GraphNorm constrain global shrinkage and prevent the trivial constant solution.
- **DropEdge / stochastic depth.** Randomly drop a fraction of edges per layer or skip layers at train-time. This injects noise that counters deterministic contraction and improves generalization.
- **PE retention.** Concatenate absolute PE $\pi(v)$ at *every* layer input: $[h_v^{(\ell)} \parallel \pi(v)]$. Absolute signals anchor embeddings and resist collapse.

24.3 Over-squashing: why long-range signals get bottlenecked

Mechanism (combinatorial view). The number of distinct k -hop sources influencing a node grows roughly like $O(\bar{d}^k)$, while only $O(\bar{d})$ messages can pass through each neighborhood per layer. Thus exponentially many constraints (distant signals) are *squashed* through few edges. This is severe in graphs with bottlenecks (high effective resistance, bad expansion).

Diagnostics. (i) Performance degrades sharply when the task requires multi-hop coordination (e.g., transfer across a bridge), (ii) Sensitivity to small topological rewires near bottlenecks, (iii) Attention saturates on a few edges with high variance across seeds.

Stabilizers against over-squashing.

- **Topology rewiring (lightweight).** Add a *virtual global node* connected to all nodes with small weight; or add k long-range edges based on PE proximity (kNN in PE space). This increases message channels across bottlenecks without violating permutation-equivariance.
- **Attention with PE bias.** In GAT/PNA-attn, bias logits with relative PE $b^\top(\pi(u) - \pi(v))$ so information preferentially flows along geometry-aligned directions, reducing competition among unrelated paths.
- **Multi-aggregator + degree scalers (PNA).** Use sum/mean/max (and variance) aggregators with degree-based scalers $s_r(d_v)$, increasing per-hop capacity and reducing saturation on hubs.
- **APPNP-style longer but *linear* propagation.** After a few nonlinear layers, use K linear propagation steps with teleportation; this expands receptive field with less squashing than stacking more nonlinear layers.
- **Controller-assisted disambiguation.** Time-aware controller uses global τ_t and PE to *stage* information: early steps spread (Fourier-like coins), later steps focus near the target. Temporal staging reduces the need to push all long-range info in one layer.

24.4 Regularizers and losses we add (what we actually optimize)

Dirichlet and smoothing penalties (encoder). We penalize over-smoothing while keeping structure:

$$\mathcal{R}_{\text{Dir}} = \lambda_{\text{Dir}} \sum_{\ell=0}^L \sum_{(u,v) \in E} \|h_u^{(\ell)} - h_v^{(\ell)}\|_2^2, \quad \mathcal{R}_{\text{Var}} = \lambda_{\text{Var}} \sum_{\ell} \sum_v \|h_v^{(\ell)} - \bar{h}^{(\ell)}\|_2^2.$$

Small λ discourage collapse; we turn them off if they hurt expressivity.

Temporal smoothness on coin parameters (controller/head). To avoid high-frequency jitter in $C_v(t)$ which destabilizes interference:

$$\mathcal{R}_{\text{time}} = \lambda_t \sum_{v=1}^{|V|} \sum_{t=1}^{T-1} \|K_{v,t} - K_{v,t-1}\|_F^2,$$

and a magnitude control $\mathcal{R}_{\text{mag}} = \lambda_m \sum_{v,t} \|K_{v,t}\|_F^2$. These promote smooth, physically plausible schedules.

Attention entropy (optional). For GAT-like layers, encourage spread (avoid single-edge choke): $\mathcal{R}_{\text{ent}} = -\lambda_e \sum_{(u \rightarrow v)} \alpha_{uv} \log \alpha_{uv}$.

24.5 Numerical stabilizers and dtypes

- **Complex vs real.** We keep the simulator in `complex128` for phase fidelity; the GNN in `float32`.
- **Gradient clipping.** Clip global norm (e.g., 1–2); prevents rare spikes from the matrix exponential/Cayley Jacobians.
- **Cayley vs exponential.** The Cayley retraction $C = (I - K)(I + K)^{-1}$ has cheap, stable gradients; we use it when $d_v > 2$ unless experiments prefer exp.
- **DropEdge / feature dropout.** $p \in [0.05, 0.2]$ on edges and features during training reduces co-adaptation and combats smoothing.

24.6 How these stabilizers manifest in ACPL

Encoder depth. We use $L = 2$ (grids/cubes) and $L = 3$ (irregulars) with residuals + GraphNorm. JK is enabled when the task needs fine local detail (e.g., near targets).

Propagation strategy. For tasks needing wider receptive fields without depth, we switch to a decoupled head: two GIN layers $\Rightarrow$ APPNP propagation with $K = 5$ and teleport $\alpha = 0.1$.

Topology aids. On severe bottlenecks we add a *virtual node* (weight $w \approx 10^{-2}$) and/or $k = 2$ PE-kNN edges per node during training; these edges are not part of the physical walk—only the encoder graph.

Controller regularization. We always include $\mathcal{R}_{\text{time}}$ and $\mathcal{R}_{\text{mag}}$ for $K_{v,t}$ (or ZYZ angles) and apply ℓ_2 weight decay to the head.

24.7 Ablations and sanity checks

1. **Remove residuals/Norm:** training becomes unstable; validation curve oscillates; contrast collapses earlier.
2. **No JK/APPNP:** long-range tasks degrade (transfer over long lines).
3. **No $\mathcal{R}_{\text{time}}$:** learned coin angles jitter rapidly in t ; performance drops under small horizon changes.
4. **No PE bias in attention:** GAT heads over-focus on hubs; squashing reappears.

One-line summary. We keep the encoder *shallow + residualized*, preserve high-frequency signal with JK/PE, expand receptive fields via *decoupled propagation* rather than depth, relieve bottlenecks by *rewiring/attention bias*, and regularize the controller to produce *smooth, physically coherent* coin schedules—together preventing vanishing gradients, over-smoothing, and over-squashing.

25 Why This Matters for ACPL (How z_v Drives Coin Control)

Role of z_v (big picture). The encoder output $z_v = \Phi_{\text{GNN}}(G, X)_v \in \mathbb{R}^H$ is a *compressed, permutation-equivariant summary* of the local graph context around vertex v (degree, motifs, spectral/positional cues). At time t , the policy forms

$$s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)], \quad h_{v,t}^{\text{tmp}} = \text{Controller}(s_{v,t}, \eta_t), \quad \theta_{v,t} = Wh_{v,t}^{\text{tmp}} + b,$$

then lifts $\theta_{v,t}$ to a *physical* coin $C_v(t) = \Phi(\theta_{v,t}) \in U(d_v)$. Thus z_v is the *only* node-local, structure-aware signal the controller sees when choosing how to mix amplitudes at v at time t . Changing z_v *changes* $\theta_{v,t}$, hence $C_v(t)$, hence the interference pattern after applying $U_t = SC_t$.

25.1 How z_v influences the quantum dynamics (mathematical chain)

At step t :

$$\psi_{t+1} = U_t \psi_t = S \left(\bigoplus_{u \in V} \Phi(\theta_{u,t}) \right) \psi_t, \quad \theta_{u,t} = \Theta(z_u, \tau_t, \pi(\text{pos}_u), \eta_t),$$

where $\Theta(\cdot)$ denotes the controller+head mapping. For a task loss $L = -f(\psi_T)$,

$$\frac{\partial L}{\partial z_v} = \sum_{t=0}^{T-1} \underbrace{\frac{\partial L}{\partial C_t}}_{\text{adjoint through } U_t} : \underbrace{\frac{\partial C_t}{\partial \theta_{v,t}}}_{\text{unitary lift}} \cdot \underbrace{\frac{\partial \theta_{v,t}}{\partial z_v}}_{\text{controller+head}} \in \mathbb{R}^H,$$

where $A : B = \text{Re Tr}(A^\dagger B)$ is the Frobenius pairing. Concretely, using the adjoint recursion $\lambda_t = U_t^\dagger \lambda_{t+1}$,

$$\frac{\partial L}{\partial \theta_{v,t}} = 2 \text{Re} \left\langle \lambda_{t+1}, S \frac{\partial C_t}{\partial \theta_{v,t}} \psi_t \right\rangle, \quad \frac{\partial \theta_{v,t}}{\partial z_v} = W \frac{\partial \text{Controller}(s_{v,t}, \eta_t)}{\partial s_{v,t}} [I \ 0 \ 0],$$

so any improvement in L attributed to changing $C_v(t)$ induces a credit assignment signal that propagates *directly* back into z_v and, via the GNN, into *all* message-passing weights that produced z_v .

25.2 What z_v must encode to control interference (task by task)

State transfer (lines/cycles, $d=2$). To route amplitude ballistically from source to a distant target by step T , the local coin at each vertex must: (i) preserve the preferred chirality early to sustain spread, (ii) apply phase corrections near turning points/target to constructively interfere at j^* . Hence z_v must encode local *direction-of-travel* cues and distance/positional context (from PE) so that $\theta_{v,t} = (\alpha, \beta, \gamma)$ switches from “spread” (Fourier-like) to “focus” (phase alignment) as $t \rightarrow T$.

Marked-node search (grids/cubes). The oracle modifies dynamics at marked vertices. Effective coins should amplify return amplitude near M (Grover-like reflections) while maintaining global exploration. Thus z_v must distinguish *marked vs. unmarked neighborhoods*, recognize local degree/motif symmetries, and exploit PE to avoid symmetric dead-ends. The controller then emits $\theta_{v,t}$ that increasingly biases mixing toward the marked basin.

Fast mixing (lattices). To minimize total-variation to uniform, the schedule must cancel standing-wave patterns and flatten eigenmode imbalances. Therefore z_v should summarize local spectral bias (via LapPE) so the emitted coins damp low-frequency dominance and inject phases that accelerate mode blending.

Robust transport (disorder). Disorder randomizes phases/edges; robust schedules avoid phase-locking to fragile paths. z_v aggregates neighborhood variability; the controller uses it to *smooth* $\theta_{v,t}$ (via our temporal regularizer) and prefer coins that spread over multiple redundant routes (risk-sensitive control).

25.3 Local structure $\Rightarrow$ local unitary: sufficiency and symmetry

Sufficiency (informal). Let $\mathcal{N}_k(v)$ be the k -hop rooted subgraph around v with node/edge features. A depth- L message-passing GNN with sum aggregation defines $z_v = g(\mathcal{N}_L(v))$ that is invariant to permutations of $\mathcal{N}_L(v)$ and stable to small feature noise. Because the coin $C_v(t)$ acts *locally* on the degree- d_v port space, the information needed to choose $C_v(t)$ at time t is contained in (i) $\mathcal{N}_L(v)$ (structural affordances), (ii) τ_t (temporal phase of the schedule), and (iii) $\pi(\text{pos}_v)$ (global alignment). Hence z_v is a sufficient statistic for local control, up to the chosen receptive field L .

Permutation-equivariance (consistency). If two nodes v, w are related by a graph automorphism that preserves features and PE, then $z_w = z_v$ and the controller emits identical $\theta_{w,t} = \theta_{v,t}$, i.e., identical local unitaries. This ensures policies respect symmetries, a necessary condition for cross-graph generalization.

25.4 Interpretability: reading what z_v learned

We probe linear functionals of z_v : (i) *degree heads* $d^\top z_v \approx d_v$, (ii) *motif probes* that predict counts of wedges/4-cycles in $\mathcal{N}_2(v)$, (iii) *PE alignment* $\langle z_v, \text{PE}(v) \rangle$. High probe accuracy correlates with better control performance, indicating z_v captures the structural factors coins need.

25.5 Sensitivity of control to z_v (local Jacobians)

Define the *control sensitivity* at (v, t) by

$$\mathcal{S}_{v,t} = \left\| \frac{\partial \theta_{v,t}}{\partial z_v} \right\|_2 \quad \text{and} \quad \mathcal{G}_{v,t} = \left\| \frac{\partial L}{\partial \theta_{v,t}} \right\|_2.$$

Large $\mathcal{S}_{v,t}\mathcal{G}_{v,t}$ indicates that improving L is most responsive to adjusting z_v at time t ; we observe peaks near the target vicinity late in episodes (transfer/search), and early peaks during “spread” phases on mixing tasks. This matches the intuition that z_v steers where and when to mix strongly.

25.6 Why z_v (and not raw x_v) is necessary

Raw features x_v (degree, coordinates, LapPE) are *first-order* and do not summarize multi-hop constraints (e.g., bottlenecks two hops away, marked-neighbor configurations). Message passing composes these into z_v that reflects *effective controllability*: whether outgoing ports lead into cul-de-sacs, symmetries, or high-conductance regions. Policies trained to output coins from x_v alone underperform markedly on irregular graphs; replacing x_v with z_v restores generalization.

25.7 Concrete micro-cases (intuition builders)

Line transfer (degree 2). Early t : z_v indicates “midline” vs. “near boundary”; controller outputs nearly Fourier-like ZYZ angles to maintain ballistic spread. Late t : z_v near j^* triggers a phase-correcting coin (adjusting α, γ) to align chiral components for constructive interference at j^* .

Grid search (degree 4). In the unmarked bulk, z_v resembles a generic 4-regular lattice node; coins promote wide spread. Near a marked node, the oracle-perturbed neighborhood changes messages, z_v shifts, and the controller outputs a more “reflective” local unitary (Grover-like component) that increases local return probability.

Mixing. z_v correlates with low-frequency Laplacian modes; controller emits coins that de-emphasize persistent modes (phase-scrambling angles), flattening P_T toward uniform.

25.8 Generalization and identifiability

Generalization across sizes/topologies. Because z_v depends on *relative* neighborhood statistics (degree-normalized aggregations, PE), the same policy weights induce comparable $\theta_{v,t}$ patterns on larger graphs (e.g., lines 128 $\rightarrow$ 256), enabling zero-shot transfer.

Gauge and port ordering. We fix a canonical port map for S ; residual local gauge (per-port phases) is absorbed into the unitary lift Φ . Since z_v is computed from gauge-invariant inputs (degree, PE, adjacency) and the head is port-order agnostic, emitted coins are physically well-defined.

25.9 Ablation evidence (causal link from z_v to control)

- **No-PE:** reduces the information in z_v about global alignment; transfer/search degrade significantly.
- **Global coin:** forcing $\theta_{v,t} \equiv \theta_t$ removes $z_v \rightarrow \theta$ dependence; performance drops on irregulars and under disorder.
- **Frozen time:** removing τ_t collapses phase scheduling; z_v cannot induce the necessary temporal choreography.

One-line summary. z_v is the *decision state* for local coin control: it distills the L -hop structural context at v into a permutation-equivariant code that, together with time and position, determines $\theta_{v,t}$ and thus the unitary $C_v(t)$. Gradients from task-level interference objectives flow back through $\theta_{v,t}$ into z_v and the GNN, aligning the embedding with exactly the structural information required to choreograph the walk.

26 Permutation-Equivariance: Why Sum/Mean/Max and Softmax Attention Work

Setup and group action. Let $G = (V, E)$ with $|V| = n$. Stack node representations as $H^{(\ell)} \in \mathbb{R}^{n \times d_\ell}$, and let $A \in \{0, 1\}^{n \times n}$ be (row-)adjacency. A vertex permutation is represented by a permutation matrix $P \in \{0, 1\}^{n \times n}$. The relabeled graph has $A' = PAP^\top$ and features $H'^{(\ell)} = PH^{(\ell)}$. A layer $\mathcal{F}$ is *node-permutation-equivariant* iff

$$\mathcal{F}(PAP^\top, PH^{(\ell)}) = P\mathcal{F}(A, H^{(\ell)}) \quad \text{for all } P.$$

26.1 Sum/Mean/Max Aggregation (Message Passing/GCN/GraphSAGE/GIN)

Vectorized sum case (GCN-style). Consider $H^{(\ell+1)} = \sigma(\hat{A}H^{(\ell)}W)$ with shared weight W and any pointwise nonlinearity σ , where $\hat{A}$ is any function of A that respects conjugation by P (e.g., $\hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ with $\tilde{A} = A + I$, $\tilde{D} = \text{diag}(\tilde{A}\mathbf{1})$). Then

$$\begin{aligned} H'^{(\ell+1)} &= \sigma\left(\underbrace{\hat{A}'}_{P\hat{A}P^\top} \underbrace{H'^{(\ell)}}_{PH^{(\ell)}} W\right) = \sigma(P\hat{A}P^\top PH^{(\ell)}W) = \sigma(P(\hat{A}H^{(\ell)}W)) \\ &= P\sigma(\hat{A}H^{(\ell)}W) = PH^{(\ell+1)}, \end{aligned}$$

since σ is applied rowwise. Thus GCN mixing via (normalized) SPMM is equivariant.

Local message passing (sum). Write the layer in edge form:

$$m_{u \rightarrow v} = M(h_u, h_v, e_{uv}), \quad \bar{m}_v = \sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}, \quad h'_v = U(h_v, \bar{m}_v),$$

with *shared* M, U for all nodes/edges. Under permutation P , neighborhoods map bijectively: $\mathcal{N}'(Pv) = \{Pu : u \in \mathcal{N}(v)\}$. Because features permute as $h'_{Pv} = h_v$ and (optionally) edge features as $e'_{Pu, Pv} = e_{u,v}$, the multiset $\{m'_{Pu \rightarrow Pv}\}_{Pu \in \mathcal{N}'(Pv)}$ equals the permuted multiset $\{m_{u \rightarrow v}\}_{u \in \mathcal{N}(v)}$. Since *sum* is permutation-invariant over a multiset, $\bar{m}'_{Pv} = \bar{m}_v$, hence $h'_{Pv} = U(h'_{Pv}, \bar{m}'_{Pv}) = U(h_v, \bar{m}_v) = h'_v$. In matrix form this is exactly the property used in the GCN proof.

Mean and max. Replace sum by mean or max: both are symmetric functions of a multiset, so the same argument holds. (For mean, degrees permute as $D' = PDP^\top$, so normalizations remain consistent.)

GraphSAGE/GIN. GraphSAGE (mean/pool) and GIN ($h'_v = \text{MLP}((1 + \epsilon)h_v + \sum_{u \in \mathcal{N}(v)} h_u)$) are special cases of the above: they use symmetric aggregation with shared parameters, so they inherit equivariance.

26.2 Softmax Attention Aggregation (GAT and variants)

Per-edge scores and normalized weights. Let the attention score be

$$e_{u \rightarrow v} = \phi(W_q h_v, W_k h_u, \gamma(e_{uv})),$$

where W_q, W_k are shared linear maps and ϕ is a shared scalar MLP (or bilinear form). Define

$$\alpha_{u \rightarrow v} = \frac{\exp(e_{u \rightarrow v})}{\sum_{w \in \mathcal{N}(v)} \exp(e_{w \rightarrow v})}, \quad h'_v = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{u \rightarrow v} W_v h_u\right),$$

with shared W_v (value map). Under permutation P , neighbors relabel as before and features permute: $h'_{Pv} = h'_v$, $h'_{Pu} = h_u$, $e'_{Pu, Pv} = e_{u, v}$. Since ϕ, W_q, W_k are *shared* and applied pointwise per edge, we get a *permuted multiset* of scores: $e'_{Pu \rightarrow Pv} = e_{u \rightarrow v}$. The softmax denominator at Pv runs over the same multiset $\{e_{u \rightarrow v}\}$, hence

$$\alpha'_{Pu \rightarrow Pv} = \alpha_{u \rightarrow v}.$$

Therefore the weighted sum at Pv is just the permuted version of the one at v :

$$\sum_{Pu \in \mathcal{N}'(Pv)} \alpha'_{Pu \rightarrow Pv} W_v h'_{Pu} = \sum_{u \in \mathcal{N}(v)} \alpha_{u \rightarrow v} W_v h_u,$$

implying $h'_{Pv} = h'_v$ after σ . Thus attention with per-target softmax is permutation-equivariant.

Multi-head attention and edge features. With K heads, each head k has its own shared $(W_q^{(k)}, W_k^{(k)}, W_v^{(k)}, \phi^{(k)})$; the argument applies headwise, and concatenation/sum over heads is again rowwise, preserving equivariance. Edge features enter via a shared map $\gamma(e_{uv})$; under permutation $e'_{Pu, Pv} = e_{uv}$ so γ preserves the multiset of inputs.

26.3 Conditions required (and how they are satisfied in ACPL)

1. **Shared parameters.** M, U (or W_q, W_k, W_v, ϕ) do not depend on absolute node IDs; they are tied across all nodes/edges. (True in our policy.)
2. **Symmetric aggregation.** The aggregator over $\mathcal{N}(v)$ is a symmetric function of a multiset (sum/mean/max or softmax-weighted sum with per-target normalization). (True in our layers.)
3. **Consistent relabeling of inputs.** Under P , we use $A' = PAP^\top$, $H' = PH$, and edge features relabel as $e'_{Pu, Pv} = e_{uv}$. (Built into our dataloader.)
4. **PEs transform consistently.** Positional encodings $\pi(\text{pos}_v)$ are stored as node features and permute as $P\pi(\text{pos})$; Laplacian PEs are eigenvectors and we fix eigenvector signs deterministically, so P acts by a signed permutation on columns—the layer sees a permuted input. (Section on PE.)

26.4 Failure modes (when equivariance breaks)

- **Index leakage.** Injecting absolute IDs or hash-based buckets into x_v breaks symmetry.
- **Non-shared parameters.** Using node-specific weights W_v (not values map but different parameters per node) breaks equivariance.
- **Cross-target normalization.** Normalizing attention across *all* edges globally (not per target v) mixes rows and breaks the rowwise permutation structure.
- **Asymmetric pre-processing.** Computing features with a non-equivariant routine (e.g., ordering-sensitive Laplacian eigensolver without sign fixing) introduces spurious asymmetry.

26.5 Matrix proof template (sum and attention as SPMM)

Let $S \in \{0, 1\}^{n \times m}$ be an incidence-like selector with one column per edge ($u \rightarrow v$) and rows indexed by sources; let $T \in \{0, 1\}^{n \times m}$ select targets. Then a generic sum-aggregation MPNN can be written as

$$H' = \sigma(T \text{Msg}(S^\top H, T^\top H, E) W),$$

where E stacks edge features and Msg is applied rowwise (per edge). Under any node permutation P , we have $S' = PS$, $T' = PT$, $H' = PH$, and $E' = E$ up to column permutation. Because Msg and W are shared and rowwise, $\text{Msg}(S'^\top PH, T'^\top PH, E')$ equals the column-permuted version of $\text{Msg}(S^\top H, T^\top H, E)$, and multiplication by T' pulls out P on the left:

$$T' \text{Msg}(\cdot) W = PT \text{Msg}(\cdot) W.$$

Thus $H' = PH^{(\ell+1)}$. For attention, replace the fixed edge weights by $\text{softmax}_{\text{per target}}$ of shared per-edge scores; per-target softmax is invariant to permutations of the incoming-edge columns of $T^\top$, so the same argument follows.

26.6 Consequences for ACPL

Correct symmetry. Because our encoder layers satisfy the above, node embeddings $Z = \Phi_{\text{GNN}}(G, X)$ obey $Z' = PZ$. The temporal controller + head act *rowwise*, so $\theta'_{Pv,t} = \theta_{v,t}$, yielding $C'_v(t) = C_{P^{-1}v}(t)$. Therefore π_θ satisfies the project’s required equivariance:

$$\pi_\theta(P \cdot G, P \cdot X, t) = P \pi_\theta(G, X, t).$$

This is essential for coin policies that transfer across relabelings and across isomorphic substructures.

Practical note. During batching and sampling we never impose an order on $\mathcal{N}(v)$ inside the model; aggregations are implemented with segment-reduce (sum/mean/max) or with per-target softmax, both of which are insensitive to neighbor permutations. This guarantees that training signals cannot “cheat” by memorizing an arbitrary ordering.

27 Computational Form: Sparse Kernels and Efficient Implementations

Why sparse? All core operations in ACPL are naturally sparse or block-structured: (i) graph message passing only communicates along edges, and (ii) the shift S is a permutation (extremely sparse), while (iii) $C_t = \bigoplus_v C_v(t)$ is block-diagonal with small dense blocks of size $d_v \times d_v$. Exploiting sparsity yields $O(|E| \cdot H)$ or $O(|V| \cdot \bar{d}^2)$ kernels with high arithmetic intensity and low memory.

27.1 Graph kernels: SPMM, SDDMM, and segment-reduce

Data layouts. We represent the directed edge set by a COO index $E = \{(u_e \rightarrow v_e)\}_{e=1}^{|E^\rightarrow|}$ as two integer tensors: **src** $= (u_1, \dots, u_E)$, **dst** $= (v_1, \dots, v_E)$, each **int64**. Node features are $H \in \mathbb{R}^{|V| \times d}$, row-major.

SPMM (sparse-matrix $\times$ dense-matrix). GCN-like layers compute $H' = \hat{A}HW$ where $\hat{A}$ is (normalized) adjacency. This decomposes into: (i) *SPMM* $\hat{A}H$ implemented by

$$(\hat{A}H)_v = \sum_{u \in \mathcal{N}(v)} \hat{a}_{vu} H_u,$$

(ii) dense GEMM $(\cdot)W$. On GPU, SPMM is realized as a *gather* (load H_u per edge), a *multiply* by the (optional) scalar $\hat{a}_{vu}$, and a *scatter-add* into row v . In PyTorch Geometric/DGL this is exposed as **spmm** / **gspmm**.

Segment-reduce (sum/mean/max). Generic MPNNs compute per-edge messages m_e then aggregate: $\bar{m}_v = \sum_{e: \text{dst}_e=v} m_e$ (sum) or mean/max. This is a *segmented reduction* over edges grouped by identical **dst**. Efficient kernels:

- **Edge-parallel:** one thread (or warp) per edge computes m_e ; atomics accumulate into $\bar{m}_{\text{dst}_e}$.
- **CSR/CSC grouping:** reorder edges so that all incoming edges of a node are contiguous; then perform warp-level reductions without atomics.

Mean uses sum plus a degree-based scale; max keeps argmax for backward.

SDDMM (sampled dense-dense matrix multiply). Attention scores (GAT) compute $e_{u \rightarrow v} = \phi(W_q h_v, W_k h_u, \gamma(e_{uv}))$ *only* for edges $(u \rightarrow v)$. This is SDDMM: take two dense matrices ($Q = HW_q$, $K = HW_k$) and evaluate pairwise inner products or tiny MLPs *sampled* at the sparse edge list. Under the hood: *gather* Q_v and K_u , evaluate score per edge.

Softmax per target (edge_softmax). Attention weights $\alpha_{u \rightarrow v} = \exp(e_{u \rightarrow v}) / \sum_{w \in \mathcal{N}(v)} \exp(e_{w \rightarrow v})$ require a numerically-stable *per-target* softmax:

$$\alpha_{u \rightarrow v} = \frac{\exp(e_{u \rightarrow v} - m_v)}{\sum_{w \in \mathcal{N}(v)} \exp(e_{w \rightarrow v} - m_v)}, \quad m_v = \max_{w \in \mathcal{N}(v)} e_{w \rightarrow v}.$$

This is implemented via two segmented reductions over incoming edges of v : one for m_v (max), one for the normalizer (sum of exp-shifted scores). Backward reuses the saved α and the per-target sums.

Putting it together (GAT example).

1. Dense GEMMs: $Q = HW_q$, $K = HW_k$, $V = HW_v$.
2. SDDMM: per-edge score $e_e = \phi(Q_{v_e}, K_{u_e}, \gamma(e_e))$.
3. Edge softmax (segmented max+sum): α_e .
4. SPMM with edge weights: $H' = \sigma(\sum_{e: \text{dst}_e=v} \alpha_e V_{u_e})$.

Complexity: $O(|V|d^2)$ for GEMMs + $O(|E|d)$ for SDDMM/weighted-SPMM; memory $O(|E|)$ for edge buffers.

Autograd. All kernels are differentiable:

- **SPMM/segment-reduce:** backward is another SPMM/segment-reduce with transposed incidence (gradients scatter back to sources).
- **SDDMM:** elementwise chain rule per edge; gradients w.r.t. Q and K accumulate via edge-parallel scatter-add.
- **Softmax:** standard per-target Jacobian $\partial \alpha = \alpha(\delta - \alpha^\top)$ with segmented reductions.

Normalization and degree scalers. GCN uses $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$. $\tilde{D}$ is computed via a *rowwise* segment-sum over **dst** (or **src**, depending on convention). PNA adds degree-dependent scalers: precompute d_v via segment-sum and apply elementwise affine maps during aggregation.

27.2 DTQW kernels: permutation shift and block-diagonal coins

State layout (arc basis). We store ψ_t as a complex vector indexed by *arcs* (ports): either flat $[|V|\bar{d}]$ or 2D $[|V|, \bar{d}]$ if we pad irregular degrees to $\bar{d} = \max_v d_v$ with masked entries (never touched). Coin application is per-node small dense matmul; shift is a permutation of arc indices.

Shift S as a pure permutation. We precompute a `port_map` tensor:

$$\text{port_map}[v, a] = (u, a') \iff S|v, a\rangle = |u, a'\rangle.$$

Applying S is a *gather/scatter* on ψ :

$$\psi'_{u, a'} = \psi_{v, a} \quad \text{for all } (u, a') = \text{port_map}[v, a].$$

On GPU this is a single custom kernel with one thread per arc (or per warp for vectorized loads), fully coalesced if we lay out arcs of a node contiguously. No arithmetic, $O(|V|\bar{d})$ memory traffic; $SS^\dagger = I$ holds bitwise.

Block-diagonal coin $C_t = \bigoplus_v C_v(t)$. We never materialize C_t . Instead we run *batched small GEMMs*: for each vertex v , multiply its local slice $\psi_t[v, :] \in \mathbb{C}^{1 \times d_v}$ by the local coin matrix $C_v(t) \in \mathbb{C}^{d_v \times d_v}$:

$$\tilde{\psi}_t[v, :] = \psi_t[v, :] C_v(t).$$

Implementation options:

- **Degree-homogeneous (grids, $d_v = 4$):** fuse into one batched GEMM of shape $[|V|, d, d]$ (cublasStrided-BatchedGemm).
- **Irregular degrees:** group vertices by degree and launch one batched GEMM per degree class; masked layout also works if padding overhead is acceptable.

Cost per step: $O(\sum_v d_v^2) = O(|V| \bar{d}^2)$ flops; memory $O(|V| \bar{d})$.

Unitary maps (per-node). When $d_v = 2$, $C_v(t) = R_z(\alpha)R_y(\beta)R_z(\gamma)$ is built from 2×2 complex sines/cosines (cheap). For $d_v > 2$, we form $K_{v,t} = i \sum_k \theta_{v,t}^{(k)} G_k$ and compute either (i) $C_v(t) = \exp(K_{v,t})$ via small dense `expm` (Pade + scaling/squaring), or (ii) the Cayley map $(I - K)(I + K)^{-1}$ via small dense linear solves (batched LU). Both are run as *batched small-matrix* routines per degree class; gradients are provided by autodiff (Fréchet derivative for exp; implicit-function rule for Cayley).

Partial trace over coin. To obtain position probabilities $P_t(v)$ without measuring, reshape ψ_t as $[|V|, d_v]$ and compute

$$P_t(v) = \sum_{a=1}^{d_v} |\psi_t(v, a)|^2.$$

This is a reduction over the coin axis (no need to form ρ_t or any Kronecker). If we do need $\rho_t^{(P)}$, we compute diagonal entries only; off-diagonals require $O(|V|^2)$ and are avoided unless explicitly analyzed.

One-step kernel schedule. At time t :

1. **Coin build (per node):** map $\theta_{v,t} \rightarrow C_v(t)$ (batched 2×2 or $d \times d$).
2. **Apply coins (batched GEMM):** $\tilde{\psi}_t[v, :] = \psi_t[v, :] C_v(t)$.
3. **Apply shift (permutation):** $\psi_{t+1} = S \tilde{\psi}_t$ via gather/scatter with `port_map`.

All three are embarrassingly parallel across vertices or arcs; step (ii) is compute-bound, (iii) is bandwidth-bound but cache-friendly (pure permutation).

27.3 Fusions, batching, and memory traffic

Kernel fusion (GNN). Libraries fuse *gather* $\rightarrow$ *edge-MLP* $\rightarrow$ *scatter-add* into a single CUDA kernel to reduce round-trips to global memory. Attention often fuses SDDMM and edge-softmax passes when feasible.

Batched multi-graph training. For many small graphs, we concatenate them into a block-diagonal mega-graph: edge indices are offset, and a `batch` vector tags membership. Segment-reduce kernels respect batch boundaries automatically because `dst` indices are disjoint across graphs.

Precision and dtypes. We use `float32` for GNN tensors and `complex128` for ψ to maintain unitarity (norm drift $\approx 10^{-12}$). Mixed precision (FP16/BF16) works for GNN but not recommended for ψ unless we add renormalization after each step.

Memory footprint. Node features: $O(|V|H)$; edge buffers: $O(|E|d_m)$ for messages or scores; state: $O(|V|\bar{d})$ complex; coin blocks: $O(|V|\bar{d}^2)$ if cached (we usually *stream*, not *cache*). Backward stores minimal intermediates: attention α_e and possibly normalized degrees.

27.4 Backprop-through-time (BPTT) as sparse adjoints

Adjoint recurrence with sparsity. Backward through the DTQW uses $\lambda_t = U_t^\dagger \lambda_{t+1}$; applying $U_t^\dagger$ is the same sequence in reverse: (i) inverse shift (same permutation kernel), then (ii) apply $C_t^\dagger$ (batched GEMM with conjugate-transpose). Gradients w.r.t. $\theta_{v,t}$ come from local Jacobians $\partial C_v / \partial \theta_{v,t}$ (tiny dense derivatives), accumulated per node/time.

Gradient accumulation.

$$\frac{\partial L}{\partial \theta_{v,t}} = 2 \operatorname{Re} \left\langle \lambda_{t+1}, S \frac{\partial C_t}{\partial \theta_{v,t}} \psi_t \right\rangle = 2 \operatorname{Re} \left\langle S^\dagger \lambda_{t+1}, \frac{\partial C_t}{\partial \theta_{v,t}} \psi_t \right\rangle.$$

We first unshift the adjoint $\tilde{\lambda}_t = S^\dagger \lambda_{t+1}$ (permutation), then perform *per-node* small complex bilinear forms; this is parallelized across nodes and time steps.

27.5 Complexity recap and bottlenecks

Per time step.

$$\begin{aligned} \text{GNN message passing: } & O(|E| \cdot H) \text{ (SPMM/segment-reduce)} \\ \text{Coin application: } & O(|V| \cdot \bar{d}^2) \text{ (batched small GEMM)} \\ \text{Shift (permute): } & O(|V| \cdot \bar{d}) \text{ (pure memory move)} \end{aligned}$$

Total per step: $O(|E|H + |V|\bar{d}^2)$; over horizon T : $O(T(|E|H + |V|\bar{d}^2))$.

Bandwidth vs compute. Message passing and shift are bandwidth-bound; coin application is compute-bound but tiny-block (use strided batched GEMM to keep high occupancy). Attentional SDDMM adds a moderate compute term $O(|E| \cdot H)$.

27.6 Implementation checklist (to hit peak throughput)

- **Edge ordering:** group edges by `dst` (for in-aggregation) to enable contiguous segment-reduce and fewer atomics.
- **Degree bucketing:** group vertices by degree for coin batched GEMMs; use `cublasStridedBatchedGemm`.
- **Avoid giant block-diagonals:** never materialize C_t or U_t ; operate on slices and permutations.
- **Coalesced permutations:** layout arcs so that $(v, 1:d_v)$ are contiguous; `port_map` fits in L2.
- **Mixed precision cautiously:** keep ψ in complex128; GNN may use AMP (autocast) safely.
- **Caching:** memoize `port_map` and LapPE per size; preallocate coin parameter/work buffers to avoid allocator overhead.
- **Checkpointing:** for long T , use gradient checkpointing on GNN layers or store only ψ_t every k steps and recompute segments in backward.

Takeaway. All heavy lifting reduces to three families of kernels: (i) *SPMM/segment-reduce* for the GNN, (ii) *batched tiny GEMMs* for coin application, and (iii) *index permutations* for the shift. This decomposition both explains and achieves the linear-in- T runtime $O(T(|E|H + |V|\bar{d}^2))$ claimed in Section 11.

28 Backpropagation: How Gradients Reach the Coins

This section expands the gradient pathway from the scalar objective to the *per-node*, *per-time* coin parameters, covering (i) adjoint dynamics through the DTQW, (ii) local Jacobians for the unitary maps (SU(2) Euler, matrix exponential, Cayley), (iii) chaining through the policy head, temporal controller, and GNN encoder, and (iv) numerics, batching, and stability.

28.1 Setup and notation

Let the terminal objective be $L(\omega) = -f(\psi_T(\omega))$, real-valued. The walk evolves

$$\psi_{t+1} = U_t(\omega) \psi_t, \quad U_t(\omega) = S C_t(\omega), \quad C_t(\omega) = \bigoplus_{v \in V} C_v(\varphi_{v,t}),$$

where the policy π_ω outputs *coin parameters* $\varphi_{v,t} \in \mathbb{R}^{p_v}$ and a fixed *unitary lift* Φ maps them to physical coins: $C_v = \Phi(\varphi_{v,t}) \in U(d_v)$. For clarity we distinguish:

$$\underbrace{\omega}_{\text{policy weights}} \xrightarrow{\pi_\omega} \underbrace{\varphi_{v,t}}_{\text{coin parameters}} \xrightarrow{\Phi} \underbrace{C_v(t)}_{\text{unitary coin}} \xrightarrow{\text{DTQW}} \psi_T \xrightarrow{f} L.$$

28.2 Adjoint recursion through the DTQW

Define the adjoint (costate) $\lambda_t = \partial_{\psi_t} L \in \mathbb{C}^{|V|\bar{d}}$ using complex *Wirtinger* calculus (for real L , λ_t is the gradient w.r.t. ψ_t^*). Given $L = L(\psi_T)$, the terminal condition is $\lambda_T = \partial_{\psi_T} L$. One step backward:

$$\psi_{t+1} = U_t \psi_t \implies \lambda_t = U_t^\dagger \lambda_{t+1} = C_t^\dagger S^\dagger \lambda_{t+1}.$$

Thus the backward pass mirrors the forward pass: inverse shift (a permutation) and per-node $C_v^\dagger$ (small batched GEMMs).

Sensitivity w.r.t. C_t . The total variation of L under a perturbation δC_t (holding ψ_t fixed) is

$$\delta L = 2 \operatorname{Re} \left\langle \lambda_{t+1}, S \delta C_t \psi_t \right\rangle = 2 \operatorname{Re} \sum_{v \in V} \left\langle \tilde{\lambda}_t^{(v)}, \delta C_v(t) \psi_t^{(v)} \right\rangle, \quad \tilde{\lambda}_t \equiv S^\dagger \lambda_{t+1},$$

where $\psi_t^{(v)} \in \mathbb{C}^{d_v}$ and $\tilde{\lambda}_t^{(v)} \in \mathbb{C}^{d_v}$ are the *local* coin-space slices at node v . Therefore the *local* gradient w.r.t. a coin matrix is

$$\boxed{\frac{\partial L}{\partial C_v(t)} = \tilde{\lambda}_t^{(v)} (\psi_t^{(v)})^\dagger \in \mathbb{C}^{d_v \times d_v}.$$

This is a rank-1 outer product per node/time (and is exactly what autodiff computes when contracting the bilinear forms).

28.3 From coin matrices to coin parameters (local Jacobians)

Coins are *not* free matrices; they are constrained to be unitary via Φ . We thus need $\frac{\partial L}{\partial \varphi_{v,t}}$ given $\frac{\partial L}{\partial C_v(t)}$. By chain rule:

$$\boxed{\frac{\partial L}{\partial \varphi_{v,t}} = \left\langle \left\langle \frac{\partial L}{\partial C_v(t)}, \frac{\partial C_v(t)}{\partial \varphi_{v,t}} \right\rangle \right\rangle = \sum_{k=1}^{p_v} \operatorname{Re} \operatorname{Tr} \left(\left(\frac{\partial L}{\partial C_v(t)} \right)^\dagger \frac{\partial C_v(t)}{\partial \varphi_{v,t}^{(k)}} \right) e_k,$$

where $\langle\langle A, B \rangle\rangle = \operatorname{Re} \operatorname{Tr}(A^\dagger B)$ is the real Frobenius pairing and e_k the k -th basis vector.

Case A: $d_v = 2$ (Euler ZYZ for SU(2))

With $\varphi_{v,t} = (\alpha, \beta, \gamma)$ and $C_v = R_z(\alpha) R_y(\beta) R_z(\gamma)$, where

$$R_z(\theta) = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{bmatrix},$$

the partials are

$$\frac{\partial C_v}{\partial \alpha} = \left(\frac{\partial R_z(\alpha)}{\partial \alpha} \right) R_y(\beta) R_z(\gamma), \quad \frac{\partial C_v}{\partial \beta} = R_z(\alpha) \left(\frac{\partial R_y(\beta)}{\partial \beta} \right) R_z(\gamma), \quad \frac{\partial C_v}{\partial \gamma} = R_z(\alpha) R_y(\beta) \left(\frac{\partial R_z(\gamma)}{\partial \gamma} \right),$$

with $\frac{\partial R_z(\theta)}{\partial \theta} = (-i/2) \sigma_z R_z(\theta)$ and $\frac{\partial R_y(\theta)}{\partial \theta} = (1/2) R_y(\theta) (-i\sigma_y)$ (using the usual Pauli matrices). Plugging these into the Frobenius pairing with $\partial L / \partial C_v$ yields closed-form 3-vector gradients. *Numerical note:* Euler angles suffer from local singularities near $\beta \approx 0, \pi$; we mitigate with (i) angle wrapping to $(-\pi, \pi]$, (ii) temporal smoothness on (α, β, γ) , and (iii) optionally switching to a 2-parameter geodesic update (see §24).

Case B: $d_v > 2$ via matrix exponential $C_v = \exp(K_v)$

We parametrize

$$K_v(\varphi_{v,t}) = i \sum_{k=1}^q \varphi_{v,t}^{(k)} G_k, \quad G_k^\dagger = G_k, \quad C_v = \exp(K_v) \in U(d_v).$$

The differential of the exponential is the Fréchet map

$$\mathrm{d} \exp_K[H] = \int_0^1 e^{(1-s)K} H e^{sK} ds.$$

Let $G = \partial L / \partial C_v$ (the local “coin matrix” gradient). Then

$$\delta L = \langle\langle G, \mathrm{d} \exp_K[\delta K] \rangle\rangle = \left\langle\left\langle \underbrace{\int_0^1 e^{sK^\dagger} G e^{(1-s)K^\dagger} ds}_{\mathcal{A}(K, G)}, \delta K \right\rangle\right\rangle,$$

so the adjoint of the Fréchet map is the *Liouville* integral $\mathcal{A}(K, G)$. By chain rule with $\delta K = i \sum_k \delta \varphi^{(k)} G_k$,

$$\boxed{\frac{\partial L}{\partial \varphi_{v,t}^{(k)}} = \operatorname{Re} \operatorname{Tr} \left(\mathcal{A}(K_v, G)^\dagger \cdot i G_k \right) = -\operatorname{Im} \operatorname{Tr} \left(\mathcal{A}(K_v, G) G_k \right).}$$

In practice, autodiff frameworks compute this through stable `expm` implementations (Pade + scaling/squaring) with built-in VJP (vector-Jacobian product), which is equivalent to evaluating $\mathcal{A}$ without forming the integral explicitly.

Spectral shortcut (optional). If $K = Q \Lambda Q^{-1}$ is diagonalizable, then $\mathrm{d} \exp_K[H] = Q (F \odot (Q^{-1} H Q)) Q^{-1}$ with $F_{ij} = \frac{e^{\lambda_i} - e^{\lambda_j}}{\lambda_i - \lambda_j}$ (and $F_{ii} = e^{\lambda_i}$). This gives a closed form for the Jacobian action and its adjoint; we rely on autodiff unless profiling shows a hotspot.

Case C: Cayley map $C_v = (I - K_v)(I + K_v)^{-1}$

Define $C = (I - K)(I + K)^{-1}$ with $K^\dagger = -K$. Differentiating:

$$\delta C = \left(-(I + K)^{-1} \delta K \right) (I + K)^{-1} (I - K) - (I - K)(I + K)^{-1} \delta K (I + K)^{-1}.$$

Given $G = \partial L / \partial C$, the adjoint rule (using $\langle\langle A, B \rangle\rangle = \operatorname{Re} \operatorname{Tr}(A^\dagger B)$) yields

$$\frac{\partial L}{\partial K} = -(I + K)^{-\dagger} G (I + K)^{-1} (I - K) - (I + K)^{-\dagger} (I - K)^\dagger G (I + K)^{-1}.$$

Then project onto the parameter basis:

$$\boxed{\frac{\partial L}{\partial \varphi^{(k)}} = \operatorname{Re} \operatorname{Tr} \left(\left(\partial L / \partial K \right)^\dagger \cdot i G_k \right) = -\operatorname{Im} \operatorname{Tr} \left(\left(\partial L / \partial K \right) G_k \right).}$$

Autodiff handles this via batched triangular solves (for $(I + K)^{-1}$) with stable VJPs.

28.4 From coin parameters back to the policy

Once we have $\frac{\partial L}{\partial \varphi_{v,t}}$ for all (v, t) , gradients to ω follow by standard backpropagation through the *head* (linear), *temporal controller* (GRU/Transformer), the *policy input* $s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)]$, and finally the *GNN encoder* that produced z_v .

Head (linear). If $\varphi_{v,t} = W_{\text{head}} h_{v,t} + b$, then

$$\frac{\partial L}{\partial W_{\text{head}}} = \sum_{v,t} \frac{\partial L}{\partial \varphi_{v,t}} h_{v,t}^\top, \quad \frac{\partial L}{\partial b} = \sum_{v,t} \frac{\partial L}{\partial \varphi_{v,t}}, \quad \frac{\partial L}{\partial h_{v,t}} = W_{\text{head}}^\top \frac{\partial L}{\partial \varphi_{v,t}}.$$

Temporal controller (GRU example). For a GRU, $h_{v,t}, \eta_{t+1} = \text{GRU}(s_{v,t}, \eta_t)$, the adjoint runs backward in t with the usual gate-wise derivatives (update/reset/candidate). Because all nodes share controller weights (either per-node state or a global state η_t), per-node gradients $\partial L / \partial h_{v,t}$ *sum* into shared parameter gradients. Time conditioning via τ_t contributes $\partial L / \partial \tau_t = \sum_v (\partial L / \partial s_{v,t})_{[\tau]}$; we typically do not train τ but this term chains back into temporal embeddings if present.

Policy input and GNN. $s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)]$ splits the gradient:

$$\frac{\partial L}{\partial z_v} = \sum_t \left(\frac{\partial L}{\partial s_{v,t}} \right)_{[1:H]}, \quad \frac{\partial L}{\partial \pi(\text{pos}_v)} = \sum_t \left(\frac{\partial L}{\partial s_{v,t}} \right)_{[H+2:]},$$

which then backpropagates through the message-passing stack. For a generic MPNN layer

$$m_{u \rightarrow v}^{(\ell)} = M^{(\ell)}(h_u^{(\ell)}, h_v^{(\ell)}, e_{uv}), \quad \bar{m}_v^{(\ell)} = \sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}^{(\ell)}, \quad h_v^{(\ell+1)} = U^{(\ell)}(h_v^{(\ell)}, \bar{m}_v^{(\ell)}),$$

adjoints mirror the forward segmented reductions: edge-wise VJPs through $M^{(\ell)}$, a segmented *sum* to distribute $\partial L / \partial \bar{m}_v^{(\ell)}$ back to edges ($u \rightarrow v$), and node-wise VJPs through $U^{(\ell)}$. The sparsity pattern (edges) determines where gradients flow—identical to forward neighborhoods (permutation-equivariant by construction).

28.5 Efficient batched computation of coin gradients

For each time t :

1. **Unshift the adjoint:** $\tilde{\lambda}_t = S^\dagger \lambda_{t+1}$ via an index permutation kernel.
2. **Per node outer product:** $G_v = \tilde{\lambda}_t^{(v)} (\psi_t^{(v)})^\dagger$ (complex $d_v \times d_v$). Implemented as batched rank-1 GEMMs grouped by degree.
3. **Local VJP through Φ :** For $d_v = 2$, contract G_v with $\partial C / \partial (\alpha, \beta, \gamma)$. For $d_v > 2$, call the batched VJP of **expm** or Cayley to get $\partial L / \partial \varphi_{v,t}$ directly.
4. **Accumulate into head/controller:** stack $\partial L / \partial \varphi_{v,t}$ into a $[|V|, p]$ tensor and continue standard dense backprop.

This pipeline is embarrassingly parallel across nodes (and across graphs in a batch). Memory overhead is dominated by storing ψ_t and λ_{t+1} slices during BPTT or by recomputation if checkpointing is used.

28.6 Complex variables, reality of L , and gradient correctness

Because $L \in \mathbb{R}$ and the state is complex, we use the identity

$$\delta L = 2 \text{Re} \langle g, \delta \psi \rangle \iff g = \partial_{\psi^*} L,$$

so inner products in all formulas are taken with complex conjugation on the left factor. All matrix pairings use the real Frobenius product $\text{Re Tr}(A^\dagger B)$, ensuring the resulting gradients w.r.t. *real* parameters $\varphi_{v,t}$ are real.

28.7 Stability aids and regularization

- **Temporal smoothness:** add $\lambda_{\text{TV}} \sum_{v,t} \|K_{v,t} - K_{v,t-1}\|_F^2$ (or angle differences for $\text{SU}(2)$). Its gradient adds local finite-difference terms to $\partial L / \partial \varphi_{v,t}$, improving conditioning.
- **Magnitude control:** $\lambda_{\text{mag}} \sum_{v,t} \|K_{v,t}\|_F^2$ discourages large exponents that can amplify **expm** sensitivity.
- **Angle wrapping & clipping:** keep (α, β, γ) in $(-\pi, \pi]$ and clip per-step changes to avoid jumping across Euler singularities.
- **Checkpointing:** recompute ψ_t segments during backward to reduce memory without affecting gradient correctness.

28.8 Complexity of backprop to coins

Per step, the dominant costs are: (i) one inverse shift $O(|V|\bar{d})$, (ii) per-node outer products $O(|V|d_v^2)$, (iii) VJP through Φ (batched tiny **expm**/solves), and (iv) standard dense backprop through the policy. Aggregating over T steps recovers the overall $O(T(|E|H + |V|\bar{d}^2))$ scaling claimed in §11.

Takeaway. Gradients reach the coins via a *local* bilinear form at each node/time, $G_v = \tilde{\lambda}_t^{(v)}(\psi_t^{(v)})^\dagger$, then pass through the *unitary lift* Φ (Euler, exp, or Cayley) to yield $\partial L / \partial \varphi_{v,t}$. These parameter gradients propagate through the head, temporal controller, and GNN encoder in the usual way, with sparse segment-reduce adjoints mirroring message passing. The entire pathway is batched, permutation-consistent, and numerically stable with the listed regularizers.

29 Choice per Task: Why These Variants

This section justifies the concrete architectural choices (GNN layer, positional encodings, temporal controller, coin parameterization, and regularizers) *per task*. The guiding principle is: each task induces a *target interference pattern* (concentration, exploration, uniform mixing, or robustness), which in turn dictates (i) how much and what kind of neighborhood information a node should aggregate (GNN variant), (ii) how strongly the policy should choreograph time (controller), and (iii) how expressive the local coin needs to be (coin map).

29.1 Common signals that drive all choices

Interference intent. *State transfer* demands phase alignment at a target; *search* demands selective amplification around marked nodes while still exploring; *fast mixing* aims to wash out structure to approach uniformity; *robust transport* builds transfer that survives noise/disorder.

Graph regularity and degree spread. Lines, cycles, grids, and hypercubes are highly regular (d_v constant). Irregular graphs (ER, small-world, d -regular mixtures) exhibit degree skew and motif diversity. Degree heterogeneity and motif richness favor *degree-aware* and *multi-aggregator* GNNs (PNA), while perfectly regular families are well served by lighter GCN/GraphSAGE.

Temporal choreography complexity. If the target interference requires pulses, phase ramps, or two-stage explore-then-focus schedules, we prefer a controller with memory (GRU) and, if needed, short-range attention (tiny Transformer).

Coin expressivity vs. stability. On $d_v = 2$ (lines/cycles), $C \in \text{SU}(2)$ (Euler) is both *complete* and *stable*. For $d_v > 2$, $\exp(K)$ spans $U(d_v)$ (or a chosen subspace); Cayley provides a cheaper, well-conditioned retraction. We regularize coins to avoid rapid oscillations that create brittle interference.

29.2 State transfer (line/cycle, grid)

Goal shape. A narrow, high-probability lobe at j^* at time T , obtained by *ballistic* spreading followed by *phase focusing*. Early time: spread; late time: align phases and reflect to cancel sidelobes.

Recommended stack.

- **GNN:** *GCN* or *GraphSAGE-mean* with $L \in \{2, 3\}$, $H \in [64, 128]$. On regular lines/grids, the inductive bias of GCN (renormalized Laplacian mixing) already tracks low-frequency, smoothly varying controls that match phase ramps; GraphSAGE-mean adds mild nonlinearity without overfitting to node IDs.
- **PE:** coordinates (lines/grids) and degrees. LapPE not required on perfectly regular lattices, but harmless.
- **Controller:** GRU (1–2 layers). GRU memory captures two-phase schedules: (i) *spread* (Hadamard/Fourier-like coins), (ii) *focus* (target-facing rotations).
- **Coin map:** $\text{SU}(2)$ Euler (α, β, γ) for $d = 2$; on grids ($d = 4$) use *Cayley* first (cheap, stable) and switch to $\exp(K)$ for final accuracy if needed.
- **Reg.:** temporal smoothness on angles or K (prevents ringing), small magnitude penalty (prevents over-rotation).

Why this works. GCN’s low-pass bias complements the smooth, slowly varying coin schedules that implement phase ramps; GRU supplies the time asymmetry (spread→focus). Euler $\text{SU}(2)$ gives exactly the right 3 DoF to steer left/right *and* phase, avoiding over-parameterization.

29.3 Marked-node search (2D grid, hypercube)

Goal shape. Amplify amplitude at a *small marked set* M from a broad initial spread, akin to Grover-like dynamics but *spatial* and *distributed*. We need *selectivity*: react strongly near marks, weakly far away, with constructive returns.

Recommended stack.

- **GNN:** *GAT* (1–2 heads) or *GIN* (*small MLP*) with $L = 2$. If marks are encoded as a binary node feature, **GAT** lets nodes *attend* to marked neighbors (and neighbors-of-neighbors) with learned importance; **GIN** (sum+MLP) increases expressivity for subtle mark patterns without attention overhead.
- **PE:** coordinates (grid) or bitstrings (hypercube) *plus* Hamming weight; include a *mark bit* feature (oracle).
- **Controller:** GRU or tiny Transformer. Tiny Transformer helps if the policy learns periodic “query pulses” (alternating emphasis between expand and refocus).
- **Coin map:** for $d > 2$ (grid/hypercube), start with *Cayley*; escalate to $\exp(K)$ when approaching the Grover-like reflection subspace near marks. For hypercube, $q \ll d^2$ (restricted generator set) often suffices due to symmetry.
- **Reg.:** moderate temporal smoothness (avoid jitter near marks), optional spatial TV penalty on $\{K_{v,t}\}$ to prefer coherent regions rather than spiky, brittle controls.

Why this works. Search needs *local heterogeneity*: near marks behave differently than far away. Attention (GAT) naturally modulates message importance based on the mark bit and local structure; GIN achieves similar selectivity via a stronger sum+MLP update. The controller orchestrates alternating explore/focus phases; the coin’s extra DoF on $d > 2$ supports Grover-like local reflections.

29.4 Fast mixing (lattices and irregular graphs)

Goal shape. Suppress structure-induced bias and reach a *nearly uniform* position distribution by T . Requires *balanced* scattering that washes out degree/motif effects.

Recommended stack.

- **GNN:** *PNA* or *APPNP/PPNP*. **PNA** aggregates (sum/mean/max) with degree scalars, explicitly compensating degree skew (critical on irregulars). **APPNP** (predict-then-propagate) decouples transformation and propagation with personalized PageRank, emulating a long-range diffusion that promotes mixing.
- **PE:** Laplacian PE (irregulars) to expose large-scale modes that need flattening; coordinates on grids optional.
- **Controller:** light GRU (one layer). Mixing benefits from *stationary* or slowly varying coins; complex temporal patterns are unnecessary.
- **Coin map:** Cayley or $\exp(K)$ with *strong magnitude regularization* to prevent over-focusing; small, steady rotations improve isotropy.
- **Reg.:** (i) temporal smoothness with higher weight, (ii) spectral flattening penalty $\sum_v \|K_{v,t}\|_F^2$, (iii) optional entropy encouragement on P_T during training.

Why this works. PNA’s degree-aware scaling is designed to *neutralize degree bias*, exactly the artifact that slows mixing. APPNP offers long-range, diffusion-like propagation with low depth, accelerating homogenization. A calm controller plus gentle, regularized coins create stable, isotropic scattering.

29.5 Robust transport under disorder (1D lines and irregulars)

Goal shape. Deliver amplitude to a target despite *static edge phases* or *dephasing*. Needs *redundant* paths and *phase-insensitive* schedules (minimize reliance on a single fragile resonance).

Recommended stack.

- **GNN:** *PNA* (irregulars) or *GraphSAGE-mean* (lines) with $L = 3$. PNA can ingest local disorder statistics (e.g., phase variance as an edge feature) and output embeddings that steer coins towards redundancy; SAGE-mean gives robust, noise-averaged summaries on simple topologies.
- **PE:** LapPE (irregulars) to detect bottlenecks/bridges; coordinates on lines.
- **Controller:** GRU with *strong temporal smoothness* (low-pass). Smooth schedules reduce susceptibility to noise-induced destructive interference.
- **Coin map:** $\exp(K)$ with a *restricted generator set* ($q \ll d^2$) that avoids highly sensitive directions; Cayley for additional numerical damping.
- **Reg.:** (i) CVaR-style objective on $P_T(j^*)$, (ii) temporal smoothness, (iii) magnitude penalty, (iv) augment with stochastic noise during training (domain randomization).

Why this works. Robustness comes from *distributional training* and *model bias* toward smooth, moderate rotations. PNA can *learn* to avoid high-variance regions or to distribute flow across multiple edges; restricted-generator $\exp(K)$ reduces the control space to well-conditioned directions.

29.6 Hypercube specifics (symmetry-aware)

Structure. Degree $d = n$ with high symmetry; nodes are n -bit strings; natural features are bitstrings and Hamming weight.

Stack.

- **GNN:** *GCN* or *GIN* (*shallow*). Bitstrings already encode a rich, nearly-sufficient embedding; deep models risk symmetry breaking without gains.
- **PE:** Hamming weight (global role), optional LapPE (rarely needed).
- **Controller:** GRU (1 layer). Schedules often mirror Grover-like phases with mild time variation.
- **Coin map:** $\exp(K)$ with a *symmetry-respecting basis* (e.g., generators that act uniformly across coordinate directions), so $q \ll d^2$.

Why this works. The cube’s symmetry means much of the “structure” is already in the features; the GNN should *not* overfit. A small controller plus symmetry-respecting generators delivers the needed reflections without fragile anisotropy.

29.7 Controller choice: GRU vs. tiny Transformer

GRU default. GRUs are parameter-efficient, causal, and excel at smooth ramps and two-stage schedules typical in transfer and robust transport.

When to use a tiny Transformer. If validation reveals *multi-pulse* or *phase-locked* patterns (e.g., alternating expand/focus in search), a 2–4 layer Transformer with short context (e.g., 8–16 steps) can attend across recent history to compose pulses. We keep heads small (1–2) to avoid overfitting.

29.8 Coin map choice: Euler ($d = 2$), Cayley vs. $\exp(K)$ ($d > 2$)

$d = 2$. $SU(2)$ Euler (3 DoF) is complete and minimal, with closed-form Jacobians, stable training, and a clear physical interpretation (phase, mixing angle, phase).

$d > 2$. Start with *Cayley* during early training for speed and conditioning; switch to $\exp(K)$ when nearing convergence to access the full unitary manifold (or a carefully chosen subspace). In robust/irregular settings, constrain to a low-dimensional generator set to reduce sensitivity.

29.9 Regularization and curriculum per task

- **Transfer:** moderate smoothness; weak magnitude penalty; curriculum in T (short→long).
- **Search:** weak–moderate smoothness; optional spatial TV; noise-free training first, then light oracle noise.
- **Mixing:** *strong* smoothness; stronger magnitude penalty; entropy encouragement.
- **Robust:** strong smoothness; CVaR objective; noise augmentation; restricted generator set.

29.10 Compute budget considerations

Lightest stack for lines/grids. GCN/SAGE + GRU + Euler/Cayley keeps cost near the theoretical floor while meeting transfer/search needs.

Irregulars require PNA cost. For mixing/robust transport on irregular graphs, PNA’s extra aggregators/scalers add a small constant factor but pay off in degree-normalized behavior and stability.

Attention is targeted. Use GAT only when marks/heterogeneity demand selective focus; otherwise, its softmax and per-edge scoring add overhead without consistent gains.

29.11 Summary table (at a glance)

Task	GNN	Controller	Coin map	Notes
Transfer (line/grid)	GCN/SAGE	GRU (1–2)	SU(2) (d=2), Cayley→exp (d>2)	Smooth ramps, focus late
Search (grid/cube)	GAT or GIN	GRU or tiny Tx	Cayley→exp (restricted)	Mark-aware attention/pulses
Fast mixing (irregular)	PNA or APPNP	GRU (light)	Cayley/exp (strong reg.)	Degree-normalized scattering
Robust transport	PNA / SAGE	GRU (smooth)	exp (restricted) or Cayley	CVaR, noise aug., smooth coins
Hypercube	GCN/GIN (shallow)	GRU (1)	exp (symm. basis)	Use bitstrings, low q

One-line takeaway. We match the inductive biases of GNN variants and coin maps to the *physics of the target interference*: smooth, low-pass mixing (GCN/APPNP) for transfer/mixing on regular graphs; selective, mark-aware sensitivity (GAT/GIN) for search; degree-aware robustness (PNA) for irregulars; and controlled, smooth temporal policies (GRU) in all cases, with coin parameterizations chosen for minimal yet sufficient expressivity and numerical stability.

30 Complexity (Per Layer and End-to-End)

We decompose the cost of one training *episode* into (i) GNN encoding, (ii) temporal control & coin parameter head, (iii) unitary coin assembly and application, (iv) shift, and (v) readout & loss. Let $|V|$ nodes, $|E|$ directed edges (for undirected graphs we include both directions), hidden width H , coin dimensions $\{d_v\}$ and typical/maximum degree $\bar{d}/d_{\max}$. State lives in the arc basis of size $D = \sum_v d_v$ (for undirected graphs $D = 2|E|$).

Notation for costs. We use T for the episode horizon (time steps), L for GNN depth (message-passing layers), K for Laplacian PE dimension, and Q for the number of PNA aggregators. Per-step forward cost is denoted C_{fwd} and the backward (autodiff) cost is $\approx \kappa C_{\text{fwd}}$ with $\kappa \in [1.5, 3]$ depending on fusion and checkpointing (attention kernels are nearer the upper end).

30.1 (A) GNN Encoder (per layer)

Each message-passing layer performs (gather → message → segmented reduce → update). The dominant FLOPs are linear in edges for gather/reduce and linear in $|V|$ for per-node updates.

GCN (normalized SPMM). One layer is essentially $\hat{A}H^{(\ell)}W^{(\ell)}$:

$$C_{\text{GCN}}^{(\ell)} = \underbrace{O(|E|H)}_{\text{SPMM: } \hat{A}H^{(\ell)}} + \underbrace{O(|V|H^2)}_{\text{dense } W^{(\ell)}}$$

Often $|E|H \gg |V|H^2$ unless H is very large; on sparse graphs the SPMM term dominates.

GraphSAGE (mean). Messages are $h_u^{(\ell)}$ with mean aggregation and an MLP on $[h_v^{(\ell)} \parallel \bar{m}_v^{(\ell)}]$:

$$C_{\text{SAGE}}^{(\ell)} = O(|E|H) + O(|V|H^2)$$

(same scaling as GCN; slightly larger constant due to concatenation/MLP).

GAT (multi-head attention). Per edge compute scores α_{uv} and a weighted sum; with H split into n_h heads of size $h = H/n_h$:

$$C_{\text{GAT}}^{(\ell)} = \underbrace{O(|E|n_h h)}_{\text{score } a^\top [Wh_u \parallel Wh_v]} + \underbrace{O(|E|n_h h)}_{\text{softmax + weighted sum}} + \underbrace{O(|V|H^2)}_{\text{head concat/projection (if used)}} \approx O(|E|H) + O(|V|H^2)$$

with a larger constant than GCN/SAGE because of edge-wise scores and softmax.

GIN. Sum aggregation plus a 2–3 layer MLP on $(1 + \epsilon)h_v + \sum_{u \in \mathcal{N}(v)} h_u$:

$$C_{\text{GIN}}^{(\ell)} = O(|E|H) + O(|V|H^2)$$

(heavier MLP constant than SAGE).

PNA. Combine Q aggregators (sum/mean/max/var) with degree-scalers:

$$C_{\text{PNA}}^{(\ell)} = O(Q|E|H) + O(|V|H^2)$$

Max/var require additional passes or fused kernels; typical $Q \in \{3, 4\}$.

Edge features. If messages use edge features $e_{uv} \in \mathbb{R}^{d_e}$ via a linear/MLP, add $O(|E|(Hd_e + H^2))$; with small d_e this is a constant-factor overhead.

PE concatenation. Concatenating Laplacian PE of size K increases input width once: replace H by $H + K$ in the first layer only (minor). Computing LapPE via K -eigenpair Lanczos is $O(K|E|)$ per *graph build* (cached per size/topology), not per time step.

Per-layer memory. Activations: $O(|V|H)$; edge buffers for messages/scores: $O(|E|h)$ (GAT) or $O(|E|)$ (GCN/SAGE/GIN/PNA with fused SPMM). Backprop keeps a subset (checkpointing reduces memory $\downarrow$ with recompute $\uparrow$).

30.2 (B) Temporal Controller + Head (per time step)

GRU (per-node). Input $s_{v,t} \in \mathbb{R}^{H+1+K}$, hidden H_{tmp} :

$$C_{\text{GRU}} = O(|V| H_{\text{tmp}} (H + 1 + K + H_{\text{tmp}}))$$

For $H_{\text{tmp}} \approx H$ and small K , this is $O(|V|H^2)$, typically smaller than the GNN edge term.

Tiny Transformer (optional). Local temporal attention over a context W_t (8–16 steps) with n_h heads:

$$C_{\text{Tx}} = O(|V| W_t H^2) \quad (\text{per step})$$

We use it sparingly (search pulses), keeping W_t and n_h small.

Linear head. Map $h_{v,t}^{\text{tmp}} \in \mathbb{R}^{H_{\text{tmp}}}$ to $\theta_{v,t} \in \mathbb{R}^{p_v}$:

$$C_{\text{head}} = O\left(\sum_v H_{\text{tmp}} p_v\right), \quad p_v = \begin{cases} 3, & d_v = 2 \\ q_v \leq d_v^2, & d_v > 2 \end{cases}$$

Usually negligible compared to GNN/SPMM.

30.3 (C) Coin map and application (per time step)

Parameter $\rightarrow$ unitary per node.

- $d_v = 2$ (**SU(2) ZYZ**): closed-form rotations $R_z(\alpha)R_y(\beta)R_z(\gamma)$: $O(1)$ FLOPs per node (tiny constant).
- $d_v > 2$, **Cayley**: $C = (I - K)(I + K)^{-1}$ with K skew-Hermitian; per node a small dense solve/inversion: $O(d_v^3)$.
- $d_v > 2$, **exponential**: $C = \exp(K)$ via scaling-squaring/Padé: $O(d_v^3)$.

Summed over all nodes: $O(\sum_v d_v^3)$. For bounded degree families (lines/cycles/grids/hypercubes with small n) this is $O(|V|)$ (since $d_v \leq d_{\max}$ is a small constant). For variable degrees, $\sum_v d_v^3 \leq d_{\max} \sum_v d_v^2$.

Apply block-diagonal coin to the state. State ψ_t has arc dimension $D = \sum_v d_v$. Multiplying by block-diagonal C_t touches each vertex-local block once:

$$C_{\text{apply-coin}} = O\left(\sum_v d_v^2\right)$$

For $d_v = d$ constant, this is $O(|V|d^2)$; for undirected graphs with $d=2$ (lines/cycles) it is $O(|V|)$.

Shift S (flip-flop permutation). S is an index permutation on D arcs; application is a gather/scatter:

$$C_{\text{shift}} = O(D) = O(\sum_v d_v) = O(|E|)$$

Pure memory traffic, typically bandwidth-bound.

30.4 (D) Readout & loss (per episode)

Partial trace over coin. Compute $P_T(v) = \langle v | \rho_T^{(P)} | v \rangle = \sum_{a=1}^{d_v} |\psi_T(v, a)|^2$:

$$C_{\text{trace}} = O(D) = O(|E|)$$

This is a per-vertex reduction of arc-probabilities.

Loss evaluation. All task losses (transfer/search/mixing/robust) are $O(|V|)$ aggregates over P_T ; robust objectives with M disorder draws multiply the *episode* cost by M (or by a smaller *minibatch* of draws if we use stochastic estimates).

30.5 (E) End-to-end per-time-step and per-episode cost

Per time step (forward). Dominant terms:

$$C_{\text{step,fwd}} = \underbrace{O(|E|H)}_{\text{GNN message passing}} + \underbrace{O(|V|H^2)}_{\text{node updates + controller}} + \underbrace{O(\sum_v d_v^2)}_{\text{apply coin}} + \underbrace{O(|E|)}_{\text{shift}} + \underbrace{O(\sum_v d_v^3)}_{\text{coin map (Cayley/exp), often cached}}$$

In practice we *cache* $C_v(t)$ within the step (we have to produce it once anyway); if we also cache $C_v(t)$ across micro-batches for the same t this term is amortized.

Per episode (forward). Multiply by T and add a small $O(|E|)$ at the end for the partial trace:

$$C_{\text{episode,fwd}} = O\left(T(|E|H + |V|H^2 + \sum_v d_v^2 + |E|) + T \sum_v d_v^3\right)$$

For bounded degrees ($d_v \leq d_{\max}$) and H not too large, this simplifies to

$$C_{\text{episode,fwd}} = O\left(T(|E|H + |V|H^2 + |V|d_{\max}^2)\right)$$

and since on sparse graphs $|E| = O(|V|)$, one often quotes

$$C_{\text{episode,fwd}} = O\left(T|E|H + T|V|\bar{d}^2\right)$$

matching the synthesis in the main text.

Backward (autodiff). Backprop through message passing roughly doubles/triples the edge work (saved activations + gradient SPMM), and backprop through the coin map adds the Fréchet derivative cost for $\exp(K)$ (implemented efficiently). A conservative multiplier:

$$C_{\text{episode,bwd}} \approx \kappa C_{\text{episode,fwd}}, \quad \kappa \in [1.5, 3]$$

Attention layers (GAT) push κ toward 3 due to softmax and score gradients.

30.6 (F) Memory and bandwidth

State & coins. State vector size is $D = \sum_v d_v$ complex numbers (complex64/128). Coins per node are $d_v \times d_v$ (tiny for $d_v \in \{2, 4\}$). We avoid materializing the full block-diagonal C_t ; we apply per-node blocks in a fused kernel to the corresponding arc slices.

GNN activations. Store L node embeddings ($|V|H$ each); optionally re-compute intermediate messages with checkpointing. Edge buffers (scores/messages) are $O(|E|h)$ for attention, else $O(|E|)$.

Bandwidth. Two hot spots: (i) SPMM/segment-reduce in GNN layers (irregular memory access over edges), (ii) permutation for S (pure gather/scatter). We rely on vendor-optimized sparse kernels (coalesced edge ordering) and fuse gathers with reductions where possible.

30.7 (G) Special cases and regimes

Regular, low degree (lines/cycles/grids). d_v is small constant $\Rightarrow \sum_v d_v^2 = O(|V|)$, $\sum_v d_v^3 = O(|V|)$. The GNN edge term $O(|E|H)$ dominates; coin map/application are near-linear.

Hypercube of dimension n . $d = n$ grows with n while $|V| = 2^n$. The coin terms scale as $O(|V|n^2)$ (apply) and $O(|V|n^3)$ (map). For $n \leq 10$ these are still small compared to the $O(|E|H) = O(n2^n H)$ GNN edge term.

Irregular graphs with skewed degrees. Use PNA: constant factor Q on edge term; degree scalers avoid over-amplifying hubs. If a few nodes have very large d_v , cap the generator subspace size q_v for their coins to curb d_v^3 cost in the map.

30.8 (H) One-line summary

Per time step, forward cost is $O(|E|H)$ (message passing) + $O(|V|H^2)$ (updates/controller) + $O(\sum_v d_v^2)$ (apply coin) + $O(|E|)$ (shift), with a one-off $O(\sum_v d_v^3)$ to map parameters to unitaries (Cayley/exp). Over T steps, this yields the end-to-end scaling

$$C_{\text{episode}} = \tilde{O}\left(T(|E|H + |V|H^2 + |V|\bar{d}^2)\right),$$

and in practice the $|E|H$ term dominates on sparse graphs with bounded degree, matching our empirical profile and the synthesis used in the main text.

30.9 What z_v Captures for Coin Control (Extreme Detail)

Big picture. The node embedding $z_v \in \mathbb{R}^H$ is a *structural control code* distilled by the GNN from the L -hop neighborhood $N_L(v)$. It serves as a *local sufficient statistic* that the temporal controller uses, together with time τ_t and positional encodings, to choose the coin parameters $\theta_{v,t}$. Intuitively, z_v tells the controller how to *mix*, *phase*, or *route* amplitude at v so that global interference patterns at time T optimize the task objective.

Information-theoretic view. Formally, $z_v = \Phi_{\text{GNN}}(G, X)_v$ is trained to maximize the mutual information with the optimal per-node schedule (conditioned on the problem instance):

$$\text{maximize } I(z_v; \{\theta_{v,t}^*\}_{t=0}^{T-1} \mid G, X),$$

subject to permutation-equivariance and locality (receptive field L). In practice, gradients flowing from $f(\psi_T)$ through $\theta_{v,t}$ back into the GNN make z_v predictive of *control-relevant* structure.

What “control-relevant structure” means. z_v aggregates the following ingredients, each directly tied to coin design:

1. **Branching geometry & degree cues.** Degree d_v and the pattern of d_u in $N_L(v)$ determine how strongly a Grover-/Fourier-like coin should spread vs. reflect. High branching (hubs) often calls for tempered spreading (avoid over-dispersion), whereas degree-2 chains benefit from nearly-Hadamard early-time spreading.
2. **Routing asymmetry (port-level anisotropy).** Although coin spaces are local and symmetric, *graph asymmetry* around v (e.g., a corridor vs. a junction) creates preferred directions. Via message functions and positional encodings, z_v encodes anisotropy so the controller can bias phases/angles to route amplitude along “good” exits and suppress leakage.
3. **Spectral/positional context.** Laplacian PEs (LapPE) or grid/hypercube coordinates inject coarse eigenstructure and metric signals (distance-to-target, parity classes, community boundaries). These inform *phase timing* (e.g., when to switch from ballistic spread to focusing) and *sign conventions* (avoid destructive cancellation from PE sign flips).
4. **Motifs & bottlenecks (impedance matching).** Subgraph motifs (chains, cycles, bridges) behave like impedance elements for waves. z_v encodes whether v is near a cut/bridge or deep inside a cluster. The optimal coin then *impedance-matches* at v : increase transmission through bottlenecks when driving transport; increase reflection to trap/protect probability in robust scenarios.
5. **Target/mark proximity signatures.** For transfer/search, z_v learns monotone surrogates of geodesic or spectral distance to j^* or to the marked set M . Near the target, coins should *focus and phase-align* (e.g., shrink R_y angle in $\text{SU}(2)$); far away, coins should *spread* (larger mixing angles, Grover-/Fourier-like).
6. **Topological constraints & symmetry classes.** In grids/hypercubes, z_v captures equivalence classes (e.g. Hamming weight shells in the hypercube). The controller can therefore output the *same* parameters for symmetry-equivalent nodes, preserving equivariance while reducing variance.
7. **Noise/disorder sensitivity.** With randomized training (edge phases, dephasing), z_v becomes sensitive to regions where interference is fragile. The controller can then favor robust coins (e.g., milder phases, less aggressive focusing) in those regions, improving CVaR/quantile metrics.
8. **Temporal readiness.** Although z_v is time-agnostic, it shapes how the *time-conditioned* controller responds: early-time schedules typically *spread*, mid-time schedules *steer*, and late-time schedules *focus*. z_v provides the static situational awareness; τ_t injects the choreography.

Concrete task-specific manifestations.

- **Line/cycle (degree 2; $\text{SU}(2)$ coins).** z_v captures (i) position/phase class (parity), (ii) relative distance to the target, (iii) boundary proximity (line ends). The controller maps z_v to ZYZ angles: large β early (Hadamard-like spread), then decreasing β and tuned z -phases to phase-lock peaks near j^* .
- **Grid (degree 4).** z_v encodes corridors vs. junctions, obstacles/marks layout, and anisotropy. The controller uses selected generators in $K_{v,t}$ to bias mixing across N/S/E/W ports—e.g., increase transmission along geodesics to the target, damp orthogonal bleed, then apply late-time phase tweaks to concentrate amplitude.

- **Hypercube (degree n).** z_v learns Hamming weight and directional bias to marked vertices. Near-symmetric shells favor Grover-like reflections; shells closer to marks get progressively more focal coins (smaller off-diagonal weights, targeted phases) to compress probability mass into the marked neighborhood on schedule.
- **Robust transport (disorder).** z_v highlights fragile cuts and highly disordered regions; the controller responds with smoother temporal variation $\theta_{v,t}$ (penalized by our regularizer), smaller generator norms $\|K_{v,t}\|$, and phase choices that reduce destructive interference variance.

Control-theoretic intuition. Viewed as a distributed controller, each node selects a *local scattering matrix* $C_v(t)$. z_v is the local estimate of (i) impedance toward each outgoing port, (ii) desired local reflectance vs. transmittance, and (iii) phase shifts needed to align multi-path arrivals at downstream foci. The global behavior emerges from these locally-informed, time-choreographed choices.

How training sculpts z_v . Backprop supplies node- and time-resolved sensitivity:

$$\frac{\partial \mathcal{L}}{\partial z_v} = \sum_t \underbrace{\frac{\partial \mathcal{L}}{\partial \theta_{v,t}}}_{\text{adjoint through } \Phi, U_t} \cdot \underbrace{\frac{\partial \theta_{v,t}}{\partial h_{v,t}} \frac{\partial h_{v,t}}{\partial s_{v,t}}}_{\text{head + controller}} \cdot \underbrace{\frac{\partial s_{v,t}}{\partial z_v}}_{\text{identity on } z_v \text{ block}}.$$

These gradients (aggregated over episodes) push the encoder to produce embeddings whose dimensions align with actionable control knobs in $\theta_{v,t}$ (e.g., one direction of z_v correlates with “mix more along the corridor”, another with “advance late-time phase”).

Diagnostics (how we know z_v encodes the right stuff).

1. **Ablate PE or degree features** $\Rightarrow$ drops in transfer/search/mixing show z_v used them.
2. **UMAP/PCA of $\{z_v\}$** clusters by structural roles (hubs, bridges, shells) and by distance-to-target.
3. **Saliency $\|\partial f / \partial z_v\|$** highlights nodes where local coin decisions are most critical (usually near bottlenecks/targets).
4. **Counterfactual swaps $z_v \leftrightarrow z_u$** within a graph class: if metrics degrade, z encodes nontrivial local role info.

Failure modes and mitigations.

- *Over-smoothing:* z_v loses contrast across large regions $\Rightarrow$ use residuals, Graph/LayerNorm, Jumping Knowledge, APPNP.
- *Over-squashing:* distant signals collapse into narrow cuts $\Rightarrow$ limited depth L , PNA (multi-aggregators), structural rewiring if needed.
- *Gauge ambiguity:* arbitrary port phases can confuse orientation $\Rightarrow$ fix canonical port ordering and phase conventions; feed consistent positional cues.

One-line summary. z_v is a compact, permutation-equivariant encoding of the *local physics of interference* around v —branching, bottlenecks, spectral position, symmetry class, and robustness cues—precisely the information a time-aware controller needs to turn into coin parameters $\theta_{v,t}$ that choreograph constructive interference where and when the task requires.

31 How the Model Learns (Signals, Objectives, “Knowing it is Right”)

Differentiable tasks (backprop). We define $L = -f(\psi_T)$ and *differentiate through the unitaries*. With $U_t = SC_t(\theta)$:

$$\psi_{t+1} = U_t \psi_t, \quad \lambda_T = \partial_{\psi_T} L, \quad \lambda_t = U_t^\dagger \lambda_{t+1}, \quad \frac{\partial L}{\partial \theta} = 2 \operatorname{Re} \sum_{t=0}^{T-1} \left\langle \lambda_{t+1}, S \frac{\partial C_t}{\partial \theta} \psi_t \right\rangle.$$

Autodiff (JAX/PyTorch) computes these gradients efficiently; for $C = \exp(K)$ we rely on the Fréchet derivative of $\exp$ (implemented by autodiff).

Non-differentiable pieces (RL). If an oracle/measurement makes gradients awkward, we treat the simulator as an episodic environment with terminal reward $r_T = f(\psi_T)$ and learn with PPO/A2C. Reward shaping (e.g., intermediate sparsity penalties or proximity bonuses) can accelerate learning.

How it knows it’s right. *Training objectives* match task goals, and *validation metrics* check generalization:

- **State transfer:** maximize $P_{j^*}(T) = \langle j^* | \rho_T | j^* \rangle$; report success vs. T and distance.
- **Search:** maximize $\sum_{m \in M} \langle m | \rho_T | m \rangle$; compare to SKW/AKR.
- **Fast mixing:** minimize $\|\rho_T - \rho^*\|_{\text{TV}}$; plot TV vs. T .
- **Robust transport:** maximize $\mathbb{E}_\xi[P_{j^*}(T; \xi)]$ and report quantiles/CVaR under disorder ξ .

Early stopping on validation curves; ablations (global coin vs. per-node; no PE vs. PE) verify that learned structure is causal.

31.1 How the Model Knows It Is “Right”

What “right” means (task-level optimality). Let $\mathcal{D}$ be a distribution over episodes $(G, X, T, \psi_0, \text{task}, \text{noise law})$. For a policy π_ω and task-specific return $f(\psi_T)$, define the *true generalization value*

$$\mathcal{J}(\omega) = \mathbb{E}_{\text{episode} \sim \mathcal{D}}[f(\psi_T(\text{episode}; \pi_\omega))].$$

We say the model is *right* at tolerance ϵ if it is ϵ -*optimal* relative to a baseline family $\mathcal{B}$:

$$\mathcal{J}(\omega) \geq \max_{\beta \in \mathcal{B}} \mathcal{J}(\beta) + \epsilon,$$

with statistical confidence $\geq 1 - \alpha$ (e.g., $\alpha = 0.05$). Baselines include fixed Hadamard/Grover coins, global time-varying coins, and task-analytical schedules (SKW/AKR for search).

Empirical estimate and confidence. We approximate $\mathcal{J}(\omega)$ on held-out *validation/test episode sets* $\{\varepsilon_i\}_{i=1}^N$ (see splits in §38.5):

$$\hat{\mathcal{J}}(\omega) = \frac{1}{N} \sum_{i=1}^N f(\psi_T(\varepsilon_i; \pi_\omega)), \quad \widehat{\text{SE}}(\omega) = \sqrt{\frac{1}{N(N-1)} \sum_i (f_i - \hat{\mathcal{J}})^2}.$$

Across S random seeds we report mean $\pm$ stderr and a 95% CI via t -statistics. We declare *right* if $\hat{\mathcal{J}}(\omega)$ exceeds the *best baseline’s* CI upper bound by at least a pre-registered margin ϵ .

Task metrics (what f is). We align f with each task’s physics:

- **State transfer:** $f = P_T(j^*) = \langle j^* | \rho_T | j^* \rangle$. Success if $P_T(j^*) \geq \tau$ (e.g., $\tau = 0.9$); we also plot $P_T(j^*)$ vs. graph distance and vs. T .
- **Search:** $f = \sum_{m \in M} \langle m | \rho_T | m \rangle$; compare to SKW/AKR on matched instances.
- **Fast mixing:** $f = 1 - \text{TV}(\rho_T, \rho^*)$, where $\text{TV}(\rho_T, \rho^*) = \frac{1}{2} \sum_v |P_T(v) - 1/|V||$; we also report mixing curves TV vs. t .
- **Robust transport:** $f = \mathbb{E}_{\xi \sim \mathcal{D}}[P_T(j^*; \xi)]$ with ξ the disorder; we additionally report *quantiles* and *CVaR*:

$$\text{CVaR}_\alpha(P) = \frac{1}{\alpha} \int_0^\alpha Q_u(P) du,$$

where Q_u is the u -quantile of $P_T(j^*; \xi)$.

Monte Carlo accuracy for disorder. For robust transport we approximate $\mathbb{E}_\xi$ with M i.i.d. disorder draws: $\hat{f} = \frac{1}{M} \sum_{m=1}^M P_T(j^*; \xi_m)$. By Hoeffding, with probability $\geq 1 - \alpha$,

$$|\hat{f} - \mathbb{E}_\xi[f]| \leq \sqrt{\frac{\ln(2/\alpha)}{2M}}.$$

We pick M so this Monte Carlo error is ≤ 0.01 absolute (typ. $M \in [50, 200]$ depending on variance).

Stopping criteria (preventing overfit). We stop when (i) validation $\hat{\mathcal{J}}(\omega)$ has not improved for P evaluations (patience), *and* (ii) the training–validation gap is below a pre-set tolerance, *or* (iii) the CI for improvement over baselines crosses zero. This guards against chasing noise.

Causality checks (the policy, not artifacts, drives wins). We run *counterfactual ablations*:

1. **No-positional-encodings (No-PE):** remove LapPE/coords; large drops on irregular graphs indicate the model truly uses structure.
2. **Global coin only:** force $\varphi_{v,t} \equiv \varphi_t$; drops show per-node control is essential.
3. **Time-frozen coins:** fix $C_v(t) \equiv C_v$; drops show temporal choreography matters.
4. **Permutation test:** randomly relabel nodes; because Φ_{GNN} is equivariant, *predictions permute but metrics stay invariant*. Any metric drift flags leakage.

Invariance and unitary sanity checks. At every step: (i) *Unitarity*: $\|\psi_{t+1}\|_2 - \|\psi_t\|_2 \approx 0$ (machine precision); (ii) *Gauge invariance*: rephase ports by diagonal unitaries—metrics must be unchanged; (iii) *Shift consistency*: S must be a permutation unitary (check $SS^\dagger = I$). Failures indicate simulation bugs, not model learning.

Generalization (IID and OOD). We evaluate on *held-out sizes and topologies* (see §38.5). Report:

- **IID:** same family/size ranges as training but disjoint seeds.
- **OOD-size:** larger N, L, n (e.g., line $N = 256$, grid $L = 96$, cube $n = 10$).
- **OOD-topology:** families unseen in training (e.g., small-world if omitted), small fraction only.

We declare “right” only if gains persist *and* confidence holds in OOD.

Calibration and sensitivity. We assess sensitivity to horizon T (sweep T) and to small perturbations of X (jitter coords/PE signs). Smooth degradation indicates robustness; sharp cliffs prompt regularization (temporal smoothness on $K_{v,t}$) or curriculum adjustments.

Pre-registered thresholds (per task). We predefine success criteria:

- **Transfer:** $\mathbb{E}[P_T(j^*)] \geq 0.90$ (line), ≥ 0.80 (grid) *and* $\geq$ best baseline +0.05 absolute.
- **Search:** improvement over SKW/AKR median by ≥ 0.03 absolute at matched T or matched time budget.
- **Mixing:** $\text{TV} \leq 0.05$ by $T = 128$ on grids; monotone TV decrease over time on average.
- **Robust:** mean success ≥ 0.75 with $\text{CVaR}_{0.1} \geq 0.60$; Monte Carlo CI ≤ 0.01 .

Meeting these thresholds *with 95% CI* constitutes “right” for the corresponding benchmark table.

Validation loop (executable pseudocode).

Listing 2: Validation protocol with seeds, CIs, and ablations

```

scores = []; seeds = fixed_val_episode_seeds()
for s in seeds:
    ep = regenerate_episode(s)          # deterministic from manifest
    f = run_episode_and_score(ep, pi)   # returns f(psi_T)
    scores.append(f)
J_hat = np.mean(scores); se = scores.std(ddof=1) / np.sqrt(len(scores))
ci95 = (J_hat - 1.96*se, J_hat + 1.96*se)

# Compare to best baseline
Jb, seb = baseline_stats(ep_list, baselines)
impr = J_hat - Jb; se_impr = np.sqrt(se**2 + seb**2)
ci95_impr = (impr - 1.96*se_impr, impr + 1.96*se_impr)
assert ci95_impr[0] > epsilon # declare "right" if passes margin

```

```
# Causality checks
for ablation in ["NoPE", "GlobalCoin", "TimeFrozen", "PermuteNodes"]:
    J_ablt = evaluate_ablation(ep_list, pi, ablation)
    log_gap(ablation, J_hat - J_ablt)
```

Reporting. All tables show mean $\pm$ stderr over S seeds; boldface indicates CI-improvement over best baseline by at least ϵ . Plots include: $P_T(j^*)$ vs distance (transfer), success vs T (search), TV vs t (mixing), and robustness curves (mean/quantiles/CVaR vs disorder level).

One-line summary. The model is “right” when (i) its *task-aligned* objective exceeds strong baselines by a *pre-registered margin*, (ii) the improvement is *statistically significant* with controlled Monte Carlo error, (iii) ablations and invariance tests attribute gains to the learned *structure-aware, time-conditioned policy*, and (iv) the effect *generalizes* to held-out sizes/topologies.

32 Using π_θ Inside the Coin Operators (End-to-End)

At each time t :

$$\theta_{v,t} = \pi_\theta(v, t; G, X), \quad C_v(t) = \Phi(\theta_{v,t}), \quad C_t = \bigoplus_{v \in V} C_v(t), \quad U_t = S C_t, \quad \psi_{t+1} = U_t \psi_t.$$

Degree 2 (lines/cycles). $\theta_{v,t} = (\alpha, \beta, \gamma)$ and $C_v(t) = R_z(\alpha)R_y(\beta)R_z(\gamma) \in \text{SU}(2)$.

Degree $d > 2$. $\theta_{v,t}$ parameterizes a skew-Hermitian $K_{v,t}$; use $C_v(t) = \exp(K_{v,t})$ (unitary by construction). For speed/stability, one can use the Cayley retraction $C_v(t) = (I - K_{v,t})(I + K_{v,t})^{-1}$.

Forward pass (one episode, pseudocode).

```
psi = psi0
h = zeros_like(GRU_state)
for t in range(T):
    # Policy: per-node parameters
    theta_t, h = policy(G, X, pos_enc, t/T, h) # -> [V, p]
    # Map to unitary coins
    C_blocks = coin_param.to_unitary(theta_t) # list of C_v(t)
    # One QW step
    psi = sim.step(psi, C_blocks) # psi <- S @ blkdiag(C) @ psi
rho_T = partial_trace_coin(psi)
loss = task.loss(psi, rho_T) # backprop or RL update
```

This *explicitly* shows how π_θ plugs into $C_v(t)$ and thus into the global step $U_t = S C_t$ that drives the walk.

33 Embedding Spaces: Classical (Policy) vs. Quantum (State)

33.1 Classical embedding spaces used by the policy

Node features and GNN node embeddings. Each node has features $x_v \in \mathbb{R}^F$ (e.g., degree, Laplacian positional encodings), and the GNN produces permutation-equivariant node embeddings

$$Z = \{z_v\}_{v \in V} = \Phi_{\text{GNN}}(G, X), \quad z_v \in \mathbb{R}^H.$$

Thus the *node embedding space* is $\mathbb{R}^H$ (shared for all v). By construction, relabeling nodes permutes Z accordingly (equivariance).

Node-time embeddings. For control at time t we augment with time and position encodings:

$$s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)] \in \mathbb{R}^{H+1+K_{\text{PE}}}, \quad \tau_t := t/T,$$

giving the *node-time embedding space* $\mathbb{R}^{H+1+K_{\text{PE}}}$.

Temporal controller state. A GRU/Transformer maps $s_{v,t}$ to a hidden state $h_{v,t} \in \mathbb{R}^{H_{\text{tmp}}}$ (per-node) or maintains a global controller state $\eta_t \in \mathbb{R}^{H_{\text{tmp}}}$ (shared), depending on implementation. Hence the *controller embedding space* is $\mathbb{R}^{H_{\text{tmp}}}$.

Coin-parameter embedding spaces. The policy head outputs coin parameters $\theta_{v,t}$ that live in a Euclidean parameter space and are then mapped to the unitary manifold:

$$\theta_{v,t} \in \begin{cases} \mathbb{R}^3, & d_v = 2 \text{ (SU(2) Euler angles } \alpha, \beta, \gamma), \\ \mathbb{R}^q, & d_v > 2 \text{ (coefficients for a skew-Hermitian } K_{v,t} \in \mathfrak{u}(d_v)), \end{cases}$$

$$C_v(t) = \Phi(\theta_{v,t}) = \begin{cases} R_z(\alpha)R_y(\beta)R_z(\gamma) \in \text{SU}(2), & d_v = 2, \\ \exp(K_{v,t}) \in U(d_v), \quad K_{v,t}^\dagger = -K_{v,t}, & d_v > 2. \end{cases}$$

For $d_v > 2$, a convenient basis is $K_{v,t} = i \sum_{k=1}^q \theta_{v,t}^{(k)} G_k$ with $\{G_k\}$ an orthonormal Hermitian basis (e.g., generalized Gell-Mann); using a truncated $q \ll d_v^2$ yields a *low-dimensional control embedding*.

Summary (classical side). The policy operates in three Euclidean embedding spaces:

$$\underbrace{\mathbb{R}^H}_{\text{node}} \rightarrow \underbrace{\mathbb{R}^{H+1+K_{\text{PE}}}}_{\text{node-time}} \rightarrow \underbrace{\mathbb{R}^p}_{\text{coin params}},$$

which are then mapped onto the *unitary manifold* $U(d_v)$ at each node/time.

33.2 Quantum state space and “final” embeddings

Hilbert and projective spaces. The walk lives in $\mathcal{H} = \mathcal{H}_P \otimes \mathcal{H}_C \cong \mathbb{C}^{|V| \bar{d}}$, where $\bar{d}$ is the coin dimension (degree). States are rays in the projective space $\mathbb{CP}^{|V|\bar{d}-1}$. The control (coin schedule) defines a trajectory

$$\psi_{t+1} = U_t \psi_t, \quad U_t = S\left(\bigoplus_v C_v(t)\right), \quad t = 0, \dots, T-1.$$

Measurement-level (position) embedding. For evaluation, we use the reduced position density $\rho_T = \text{Tr}_C(|\psi_T\rangle\langle\psi_T|)$ and its diagonal $P_T \in \Delta^{|V|-1}$:

$$P_T(v) = \langle v | \rho_T | v \rangle, \quad \sum_v P_T(v) = 1.$$

Thus a natural *final embedding* of the episode is the probability vector P_T in the simplex $\Delta^{|V|-1}$. Tasks consume P_T directly (state transfer/search) or via distances (mixing).

Density-matrix embedding (optional). If phase information matters, one may take the full $\rho_T \in \mathbb{C}^{|V| \times |V|}$ (PSD, $\text{Tr} = 1$) as the final embedding. This is richer than P_T but heavier; most of our objectives depend only on P_T .

Graph-level classical embedding (optional, for analysis/meta-learning). A compact summary of the *input instance* is

$$\bar{z} = \frac{1}{|V|} \sum_{v \in V} z_v \in \mathbb{R}^H,$$

and a compact summary of the *control schedule* is, e.g.,

$$\bar{K} = \frac{1}{T|V|} \sum_{t,v} \text{vec}(\text{skewHerm}(\log C_v(t))) \in \mathbb{R}^{d_{\text{sched}}},$$

when the matrix log is available (or use the raw parameter mean $\frac{1}{T|V|} \sum_{t,v} \theta_{v,t}$). These are not required for training but are useful for diagnostics and meta-learning.

33.3 Putting it together (one-line view)

$$(G, X) \xrightarrow{\text{GNN}} \{z_v\} \rightarrow \{s_{v,t}\} \xrightarrow{\text{temporal controller}} \{\theta_{v,t}\} \xrightarrow{\Phi} \{C_v(t)\} \xrightarrow{\text{DTQW}} \psi_T \rightsquigarrow \underbrace{P_T \in \Delta^{|V|-1}}_{\text{final embedding for tasks}},$$

with optional ρ_T if phase/coherence matters.

34 Implementation Plan (Doing All Four with GNNs)

Common stack. Batched DTQW simulator on lines/cycles ($d = 2$), 2D grid ($d = 4$), and hypercube ($d = n$). Coins: $SU(2)$ for $d = 2$; $C = \exp(K)$ or Cayley for $d > 2$; flip-flop shift; efficient sparse ops [3, 29, 28].

Core A: State transfer. GNN+GRU ACPL; compare to (i) static coin, (ii) global time-varying, (iii) coord-MLP baseline on lines.

Core B: Marked-node search. GNN+GRU on grid/hypercube with oracle; Grover/AKR/SKW baselines. Add PPO/A2C if non-diff oracles/constraints emerge [4, 5, 6, 22].

Extension C: Fast mixing. Optimize terminal TV at fixed T ; scaling vs T ; compare to mixing analyses [8, 10].

Extension D: Robust transport. Static/dynamic disorder; robustness/phase diagrams [17, 18, 19].

Meta-generalization. Train across graph families/sizes with LapPE; test distribution shift.

Paper-quality figures. Final tables/plots; visualize learned per-node coin schedules.

35 End-to-End GNN Processing in ACPL (Step-by-Step)

This section details exactly how the GNN policy processes a graph input, produces per-node, per-time coin parameters, and how gradients propagate back through the discrete-time quantum walk (DTQW) dynamics to optimize the policy.

35.1 Data Ingestion and Preprocessing

Graph tensors (episode-level). We represent a graph $G = (V, E)$ with sparse tensors:

- **edge_index** $\in \mathbb{N}^{2 \times |E|}$: COO edges ($u \rightarrow v$) (for undirected graphs include both directions).
- **deg** $\in \mathbb{N}^{|V|}$: degrees d_v .
- **port_map** $[v, a] = (u, a')$: flip-flop pairing to implement the shift S (precomputed).
- **X** $\in \mathbb{R}^{|V| \times F}$: node features (degree, Laplacian positional encodings, coordinates on grids, role encodings).
- $\psi_0 \in \mathbb{C}^{(|V|\bar{d}) \times 1}$: initial DTQW state in the arc basis ($\bar{d}$ is a representative degree).

Task metadata. Horizon T , task specification (target j^* , marked set M , target distribution ρ^*), and (optionally) disorder law $\mathcal{D}$ for robust transport.

35.2 Permutation-Equivariant Node Embeddings (GNN Encoder)

We compute L layers of message passing to obtain $z_v \in \mathbb{R}^H$:

$$h_v^{(0)} = x_v, \tag{34}$$

$$m_{u \rightarrow v}^{(k)} = M^{(k)}(h_u^{(k)}, h_v^{(k)}, e_{uv}), \tag{35}$$

$$\bar{m}_v^{(k)} = \sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}^{(k)}, \tag{36}$$

$$h_v^{(k+1)} = U^{(k)}(h_v^{(k)}, \bar{m}_v^{(k)}), \quad k = 0, \dots, L-1, \tag{37}$$

$$z_v = h_v^{(L)}. \tag{38}$$

Choices for $(M^{(k)}, U^{(k)}, \text{AGG})$ instantiate GCN/GraphSAGE/GAT/GIN/PNA layers. Sum (or attention-weighted sum) aggregation ensures permutation-equivariance (see Theorem in the policy section).

35.3 Node–Time Conditioning (Build Policy Inputs)

At time $t \in \{0, \dots, T-1\}$:

$$\tau_t = t/T, \quad s_{v,t} = [z_v; \tau_t; \pi(\text{pos}_v)] \in \mathbb{R}^{H+1+K_{\text{PE}}}.$$

$\pi(\text{pos}_v)$ are positional encodings (Laplacian PE for irregular graphs; normalized coordinates on grids).

35.4 Temporal Controller and Coin Parameter Head

A GRU/Transformer reads $s_{v,t}$ and outputs coin parameters:

$$h_{v,t}^{\text{tmp}}, \eta_{t+1} = \text{Controller}(s_{v,t}, \eta_t), \quad (39)$$

$$\theta_{v,t} = Wh_{v,t}^{\text{tmp}} + b. \quad (40)$$

$$\theta_{v,t} \in \begin{cases} \mathbb{R}^3, & d_v = 2 \text{ (SU(2) ZYZ angles),} \\ \mathbb{R}^q, & d_v > 2 \text{ (coeffs for skew-Hermitian } K_{v,t}). \end{cases}$$

35.5 Lift Parameters to Unitaries (Coin Map)

$$C_v(t) = \Phi(\theta_{v,t}) = \begin{cases} R_z(\alpha)R_y(\beta)R_z(\gamma) \in \text{SU}(2), & d_v = 2, \\ \exp(K_{v,t}(\theta_{v,t})) \in U(d_v) \text{ or } (I - K_{v,t})(I + K_{v,t})^{-1}, & d_v > 2, \end{cases}$$

and assemble the block-diagonal coin $C_t = \bigoplus_v C_v(t)$.

35.6 DTQW Evolution (Coin Then Shift)

Apply coin then shift:

$$\tilde{\psi}_t = C_t \psi_t, \quad \psi_{t+1} = S \tilde{\psi}_t,$$

where S is realized as an index permutation using `port_map`. Repeat for $t = 0, \dots, T-1$.

35.7 Task Readout and Losses

Reduce coin to position and compute the task loss:

$$\rho_T = \text{Tr}_C(|\psi_T\rangle\langle\psi_T|), \quad P_T(v) = \langle v | \rho_T | v \rangle,$$

$$\mathcal{L} = \begin{cases} -P_T(j^*), & \text{state transfer,} \\ -\sum_{m \in M} P_T(m), & \text{search,} \\ \frac{1}{2} \sum_v |P_T(v) - 1/|V||, & \text{fast mixing,} \\ -\mathbb{E}_{\xi \sim \mathcal{D}} [P_T(j^*; \xi)] \text{ (or CVaR)}, & \text{robust transport.} \end{cases}$$

35.8 Backpropagation (Gradients to the GNN)

For differentiable settings we minimize $L(\theta) = -f(\psi_T(\theta))$ and backpropagate through time:

$$\psi_{t+1} = U_t(\theta) \psi_t, \quad U_t = S \left(\bigoplus_v C_v(\theta_{v,t}) \right), \quad \lambda_T = \partial_{\psi_T} L, \quad \lambda_t = U_t^\dagger \lambda_{t+1},$$

$$\frac{\partial L}{\partial \theta} = 2 \text{Re} \sum_{t=0}^{T-1} \langle \lambda_{t+1}, S(\partial C_t / \partial \theta) \psi_t \rangle.$$

Autodiff provides $\partial C_t / \partial \theta$: for $C = \exp(K)$ it uses the Fréchet derivative $d \exp_K[H] = \int_0^1 e^{(1-s)K} H e^{sK} ds$; for Cayley, closed-form Jacobians are used. Chain rule delivers gradients to the head $\rightarrow$ controller $\rightarrow s_{v,t} \rightarrow z_v \rightarrow$ message functions $M^{(k)}, U^{(k)}$.

35.9 RL Option (Non-Differentiable Oracles)

When oracles/measurements make gradients awkward, we treat $\theta_{v,t}$ as the action of a stochastic policy and apply PPO/A2C with terminal reward $r_T = f(\psi_T)$; the environment transition is the DTQW step with $C_v(t) = \Phi(\theta_{v,t})$.

35.10 Complexity and Stabilization

Per step: $O(|E|H)$ for message passing + $O(|V|\bar{d}^2)$ to apply C_t ; total $O(T(|E|H + |V|\bar{d}^2))$. We use residual connections, GraphNorm/LayerNorm, DropEdge/feature dropout, gradient clipping, and temporal regularizers $\sum_{v,t} \|K_{v,t} - K_{v,t-1}\|_F^2$.

35.11 Tensor Shape & DType Reference (for implementation)

Name	Symbol	Shape	DType
Node feats	X	$[V , F]$	float32
Edges (COO)	<code>edge_index</code>	$[2, E]$	int64
Degrees	d_v	$[V]$	int32/int64
Port map	<code>port_map</code>	$[V , \bar{d}, 2]$	int64
Node emb.	z_v	$[V , H]$	float32
Time feature	τ_t	$[1]$ or $[V , 1]$	float32
Policy input	$s_{v,t}$	$[V , H+1+K_{PE}]$	float32
Controller state	η_t	$[1, H_{\text{tmp}}]$ or $[V , H_{\text{tmp}}]$	float32
Coin params	$\theta_{v,t}$	$[V , p]$	float32
Local coin	$C_v(t)$	$[\bar{d}, \bar{d}]$ (batched)	complex64/complex128
Block coin	C_t	$[V \bar{d}, V \bar{d}]$ (implicit)	complex
State	ψ_t	$[V \bar{d}, 1]$	complex64/complex128
Position probs	P_T	$[V]$	float64

35.12 Micro-Example (1D Line, $d = 2$)

Features $X = [\text{deg}, \text{pos}/L]$; 2-layer GIN $\Rightarrow z_v \in \mathbb{R}^{64}$; GRU reads $[z_v, t/T]$ and outputs (α, β, γ) ; build $C_v(t) = R_z(\alpha)R_y(\beta)R_z(\gamma)$; shift by `port_map`; loss $= -P_T(j^*)$; backprop tunes GNN+GRU to phase-align interference at j^* by T .

Summary. The GNN encodes local graph structure into z_v , the temporal controller turns $[z_v, t/T, \pi(\text{pos}_v)]$ into compact parameters $\theta_{v,t}$, the coin map lifts them to unitaries $C_v(t) \in U(d_v)$, and the DTQW dynamics push ψ_t so as to maximize the task objective $f(\psi_T)$; gradients flow back through time and across edges to update π_θ .

35.13 Fast Mixing (Expanded): Objectives, Regimes, and Monte Carlo

Goal. Given a graph $G = (V, E)$, we want the position marginals of the DTQW to approach the uniform distribution $\rho^*(v) = 1/|V|$. Let $P_t \in \Delta^{|V|-1}$ denote the position probabilities extracted from the quantum state ψ_t via the reduced density $\rho_t^{(P)} = \text{Tr}_C(|\psi_t\rangle\langle\psi_t|)$, i.e., $P_t(v) = \langle v | \rho_t^{(P)} | v \rangle$. Our loss is the terminal total-variation (TV) distance

$$\mathcal{L}_{\text{mix}}(T) = \text{TV}(P_T, \rho^*) = \frac{1}{2} \sum_{v \in V} |P_T(v) - \frac{1}{|V|}|.$$

We also report *mixing curves* $t \mapsto \text{TV}(P_t, \rho^*)$, as in [8, 10, 11].

Three operational regimes. Unlike a classical random walk, a fully coherent DTQW is not Markovian on positions; interference couples P_t to off-diagonal coin/position coherences. We therefore distinguish three regimes used in analysis and training:

1. **Coherent (noiseless):** P_{t+1} is *not* a linear function of P_t ; however the *Cesàro average* $\bar{P}_T = \frac{1}{T} \sum_{t=1}^T P_t$ often converges to a distribution close to uniform on many graphs (cf. [8, 10]). We may minimize $\text{TV}(\bar{P}_T, \rho^*)$ in addition to $\text{TV}(P_T, \rho^*)$.
2. **Time/coin randomization (expected dynamics):** if at each step we inject small coin-phase noise or sample coins from a symmetric law, the *expectation* $\mathbb{E}[P_{t+1} | P_t]$ obeys a *linear* recursion $P_{t+1} = \underbrace{M_t}_{\text{bistochastic}} P_t$, where M_t is doubly-stochastic (defined below). This gives a bona fide (possibly time-inhomogeneous) Markov chain on V .
3. **Projective/weak measurements (CPTP mixing):** inserting weak position or coin dephasing each step yields a completely-positive trace-preserving (CPTP) map whose *marginal* on positions is again a linear map $P_{t+1} = M P_t$ with M doubly-stochastic (DS). The uniform distribution is then a fixed point.

Doubly-stochastic (DS) matrices and uniform stationarity. A matrix $M \in \mathbb{R}^{|V| \times |V|}$ is *doubly-stochastic* if $M \geq 0$, $M\mathbf{1} = \mathbf{1}$ (row-stochastic), and $\mathbf{1}^\top M = \mathbf{1}^\top$ (column sums are 1). For any DS M , the uniform distribution ρ^* is stationary: $M\rho^* = \rho^*$. Moreover, DS kernels are *Schur-contractive* in TV distance:

$$\text{TV}(MP, \rho^*) \leq \sigma(M) \text{TV}(P, \rho^*),$$

where $\sigma(M) = \max\{|\lambda| : \lambda \in \text{spec}(M), \lambda \neq 1\}$ bounds the one-step contraction; for time-homogeneous M , the mixing time scales like $O(\log(1/\varepsilon)/(1 - \lambda_2))$ with λ_2 the second-largest eigenvalue modulus (SLEM).

Why (and when) the induced kernel is DS. Consider one coherent step $\psi \mapsto S(\oplus_v C_v)\psi$ followed by (i) averaging over i.i.d. coin-phase noise with a law invariant under local phase permutations, or (ii) complete dephasing in the coin basis (tracing coin coherences each step). In either case, the expected population transfer from a node u to node v is the sum of squared moduli of certain unitary amplitudes and therefore:

- *Nonnegativity*: transitions are probabilities of mutually exclusive events;
- *Row sums*: total probability leaving u equals 1 (unitarity preserves norm);
- *Column sums*: by the same unitarity and flip-flop symmetry of S , the total inflow to v equals 1.

Thus the (expected) one-step map on P_t is $P_{t+1} = M_t P_t$ with M_t DS. On d -regular graphs with symmetric coins, M_t often reduces to the classical lazy random walk or a closely related kernel.

What “Monte Carlo” means here. We use Monte Carlo in two senses:

1. **Expectation over noise/coin randomness**: when training/evaluating the robust or randomized-coin variant, we estimate $\mathbb{E}[P_T]$ (hence $\mathbb{E}[\text{TV}(P_T, \rho^*)]$) by averaging over M sampled noise realizations. By Hoeffding, the absolute error decays like $O(1/\sqrt{M})$ (we set M so the CI width is ≤ 0.01).
2. **Time averaging (Cesàro)**: in the coherent regime, we approximate $\bar{P}_T$ via the empirical average $\frac{1}{T} \sum_{t=1}^T P_t$; this is not stochastic sampling but the term “Monte Carlo time average” is sometimes used informally.

Our fast-mixing objectives (three options). We support three interchangeable losses; the manifest decides which one a run uses:

$$\text{Terminal (coherent): } \mathcal{L}_{\text{TV}} = \frac{1}{2} \sum_v |P_T(v) - \frac{1}{|V|}|. \quad (41)$$

$$\text{Cesàro (coherent): } \bar{P}_T = \frac{1}{T} \sum_{t=1}^T P_t, \quad \mathcal{L}_{\text{Cesàro}} = \frac{1}{2} \sum_v |\bar{P}_T(v) - \frac{1}{|V|}|. \quad (42)$$

$$\text{Expected (DS kernel): } \hat{P}_{t+1} = \mathbb{E}[P_{t+1} | P_t] = M_t P_t, \quad \mathcal{L}_{\text{DS}} = \frac{1}{2} \sum_v |(\hat{P}_T)_v - \frac{1}{|V|}|. \quad (43)$$

Differentiation is straightforward in all cases because P_t is obtained by a differentiable reshape/trace of ψ_t , and in the DS case M_t is a differentiable function of the coin parameters through the (noisy) unitary channels.

How ACPL drives mixing. The policy π_ω learns coin schedules $C_v(t)$ that:

1. **Increase spectral gap of the effective kernel.** In the randomized/noisy regime, the induced M_t (or its time average) has a larger gap $1 - \lambda_2$, implying a faster contraction to ρ^* .
2. **Destructure interference bottlenecks.** On grids and irregular graphs, early-time coins encourage *dispersive* dynamics (Fourier-like spread), while mid/late-time coins damp residual spatial bias.
3. **Exploit degree information.** On irregular graphs, degree-aware phases approximate Grover diffusion locally, promoting near-uniform outflow from high-degree nodes.

Diagnostics and guarantees. We verify (i) *bistochasticity* in the DS setting by checking $M_t \mathbf{1} = \mathbf{1}$ and $\mathbf{1}^\top M_t = \mathbf{1}^\top$ numerically, (ii) *unitarity* and *norm conservation* at the state level, and (iii) *monotone TV curves* on average. For time-homogeneous DS kernels, we estimate the empirical SLEM from the learned $\bar{M} = \frac{1}{T} \sum_t M_t$ and report an implied mixing time bound $\tilde{\tau}_\varepsilon \approx \log(1/\varepsilon)/(1 - \lambda_2(\bar{M}))$ (heuristic but informative).

Practical training details. We add a small *spatial roughness penalty* $\sum_{(u,v) \in E} (P_T(u) - P_T(v))^2$ to suppress checkerboard artifacts on bipartite graphs, and a *temporal smoothness* penalty on coin parameters to avoid oscillatory schedules that impede convergence. For grids, we commonly adopt GCN/GIN with LapPE; for irregular graphs we prefer PNA or GAT to better normalize degree effects (cf. §??).

Connections to prior work. Our TV objectives mirror the theoretical and empirical mixing lenses in [8, 11, 10]: (i) coherent DTQW mixing via time averaging (Cesàro), (ii) improvement from decoherence/randomization which effectively induces Markovian bistochastic evolution on positions, preserving uniform as a fixed point while enhancing convergence speed.

36 Dataset and Task Suite

Episodes as data. Each training instance is an *episode* $(G, X, T, \psi_0, \text{task}, \text{noise})$, where G is a graph (stored sparsely), X are node features, T the horizon, ψ_0 the initial state, and the task/noise specify objectives and disorder models. The simulator supplies learning signals (losses/rewards), thus no external labels are required.

Graph families. We meta-train on lines/cycles ($N = 64 \dots 192$), grids ($L \times L$, $L = 32 \dots 80$), and hypercubes ($n = 6 \dots 9$), with a light mixture of irregular graphs (random d -regular, ER, small-world) to encourage robustness. Meta-test holds out larger sizes (line $N = 256$, grid $L = 96$, cube $n = 10$) to evaluate generalization.

Features. Node features include degree and positional encodings: Laplacian eigenvectors (top $K = 8\text{--}16$) for irregular graphs; normalized coordinates $(x/L, y/L)$ for grids; bitstrings for hypercubes.

Tasks. We uniformly sample among: *state transfer* (target j^* at large graph distance), *search* (single or few marked vertices on grids/cubes), *fast mixing* (TV distance to uniform), and *robust transport* (edge-phase disorder and/or coin dephasing). Horizons $T \in \{64, 96, 128\}$.

Noise/disorder. For robust transport we draw static edge phases $\phi_e \sim \mathcal{U}[-\sigma, \sigma]$ with $\sigma \in \{0.05, 0.1, 0.2\}$ and/or coin dephasing $p \in \{0.01, 0.02\}$; the seed is stored per episode for exact reproducibility.

Splits & curriculum. Meta-train uses the above size ranges; meta-val holds out sizes within range; meta-test uses larger, unseen sizes. We apply a curriculum in horizon ($T = 64 \rightarrow 96 \rightarrow 128$) and introduce robustness in the final phase.

Baselines. We ship procedures (not labels) for Hadamard/Grover fixed coins, a global time-varying coin (MLP in t/T), and analytical SKW/AKR settings on search tasks, all reading the same episode spec.

Scale and storage. We train with 5k–10k sampled episodes initially (on-the-fly generation), scaling to 50k–100k for final results. Episodes can be serialized to .npz/HDF5 with fields: graph (edge list, degrees), features X , task parameters, T , noise seed, and ψ_0 spec.

37 What Exactly Is Our Dataset?

TL;DR. Our “dataset” is a *procedurally generated suite of episodes*. Each episode is a small, self-contained world plus a goal:

$$\underbrace{(G, X, T, \psi_0, \text{task spec}, \text{noise})}_{\text{one episode}} \implies \text{run DTQW} \Rightarrow \psi_T \Rightarrow f(\psi_T) \text{ (training signal)}.$$

There are no external labels; the simulator and objective f create the supervision.

37.1 The Atomic Data Point: One Episode

An **episode** contains:

1. **Graph** $G = (V, E)$: sparse edges (COO), degrees $\{d_v\}$, and a flip-flop port map for the shift S .
2. **Node features** X : degree, positional encodings (Laplacian PE; top $K \in [8, 16]$), or simple coordinates/bitstrings (grids/hypercubes).
3. **Horizon** $T \in \{64, 96, 128\}$.

4. **Initial state** ψ_0 (arc basis), e.g., localized at a node/chirality.

5. **Task specification** (what success means):

- *State transfer*: target node j^* .
- *Search*: marked set $M \subseteq V$.
- *Fast mixing*: target distribution is uniform on V .
- *Robust transport*: as transfer, but under disorder.

6. **Noise/disorder** (optional): e.g., static edge phases $\phi_e \sim \mathcal{U}[-\sigma, \sigma]$, coin dephasing with probability p .

37.2 What is the “Label”? The Simulator Provides It

After evolving $\psi_{t+1} = S C_t \psi_t$ for $t = 0, \dots, T-1$, we compute the reduced position density ρ_T and the position distribution $P_T(v) = \langle v | \rho_T | v \rangle$. The objective $f(\psi_T)$ yields the training signal:

$$\text{State transfer: } \mathcal{L} = -P_T(j^*).$$

$$\text{Search: } \mathcal{L} = - \sum_{m \in M} P_T(m).$$

$$\text{Fast mixing: } \mathcal{L} = \frac{1}{2} \sum_{v \in V} |P_T(v) - 1/|V||.$$

$$\text{Robust transport: } \mathcal{L} = -\mathbb{E}_{\xi \sim \mathcal{D}} [P_T(j^{\xi})] \quad (\text{or CVaR}).$$

Thus, the *walk + objective* replaces external annotations.

37.3 Why On-the-Fly Generation?

It provides (i) unlimited diversity and domain randomization, (ii) exact reproducibility from seeds, (iii) no storage bloat, and (iv) easy curricula (we adjust the sampling distribution across training phases).

37.4 Train/Val/Test Without Leakage

We define *splits* by size/topology:

- **Meta-train**: lines $N = 64..192$, grids $L = 32..80$, hypercubes $n = 6..9$ (+ small fraction of irregular graphs).
- **Meta-val**: held-out sizes within those bands (e.g., specific L or N not used in train).
- **Meta-test (OOD)**: larger/unseen sizes (line $N = 256$, grid $L = 96$, cube $n = 10$) and unseen irregulars.

We store *episode tuples or seeds* for val/test so they can be regenerated exactly; the same configuration never appears in more than one split.

37.5 Concrete Mini-Examples (What an Episode Looks Like)

(A) State transfer on a line. G : line $N = 128$; X : degree + $x/127$; $T = 96$; ψ_0 : localized at node 0, chirality R; task: target $j^* = 100$; noise: none; $\mathcal{L} = -P_T(100)$.

(B) Search on a 64×64 grid. G : grid $L = 64$; X : degree + coords $(x/63, y/63)$; $T = 128$; mark $M = \{(10, 50)\}$; $\mathcal{L} = -\sum_{m \in M} P_T(m)$.

(C) Robust transport on a line with disorder. G : line $N = 160$; X : degree + pos; $T = 128$; target $j^* = 140$; noise: edge phases $\phi_e \sim \mathcal{U}[-0.1, 0.1]$; $\mathcal{L} = -\mathbb{E}_{\phi} [P_T(140; \phi)]$.

37.6 What Do We Actually Store? A Manifest + Seeds

We version a small *manifest* (YAML/JSON) that records: graph families and size ranges per split, PE method and K , task proportions and horizon set, noise distributions, curriculum schedule, and a master seed. Validation/test are lists of episode tuples or their 64-bit seeds. Re-running the generator with these seeds exactly reproduces the episodes.

37.7 How a Batch Flows Through Training

1. Sample a batch of episode configurations (each deterministically maps to a seed).
2. Build G, X, T, ψ_0 , task, noise for each episode.
3. Policy computes $\theta_{v,t} = \pi_\theta(v, t; G, X)$ and coins $C_v(t) = \Phi(\theta_{v,t})$.
4. Run DTQW for T steps to get ψ_T and P_T .
5. Compute the episode loss $\mathcal{L}$ and backpropagate to update π_θ .

Why this is a “dataset.” It is a *procedural dataset*: a fixed generator + a sampling protocol + a public manifest + fixed seeds for evaluation. This is standard in RL/control and fully reproducible.

38 Dataset and Task Suite (On-the-Fly Generation)

Rationale. Because ACPL is a *control/optimization* problem with a simulator-defined objective, we do not require labeled datasets. Instead, we train on an open-ended *task suite* generated *on the fly* with fixed random seeds, which provides (i) unlimited data via domain randomization, (ii) exact reproducibility via deterministic seeding, (iii) storage efficiency, and (iv) easy curricula.

38.1 Episodes as Data Points

A single training instance is an *episode*

$$(G, X, T, \psi_0, \text{task}, \text{noise}),$$

consumed by the policy to produce a coin schedule $\{C_v(t)\}_{v,t}$ and a terminal return $f(\psi_T)$.

- $G = (V, E)$: sparse graph (edge list, degrees, port map for S).
- X : node features (degree, positional encodings, coordinates/bitstrings).
- T : horizon.
- ψ_0 : initial quantum state (arc basis).
- *task*: state transfer / search / fast mixing / robust transport.
- *noise*: disorder law (edge phases, coin dephasing) with seed.

38.2 Graph Families and Sizes

Core. Lines/cycles ($N \in \{64, 96, 128, 160, 192, 256\}$), 2D grids ($L \in \{32, 48, 64, 80, 96\}$), hypercubes ($n \in \{6, 7, 8, 9, 10\}$).
Optional diversity. Small proportions of random d -regular ($d \in \{3, 4, 5\}$), Erdős–Rényi $G(N, p)$ with $p \approx d/N$, and small-world (Watts–Strogatz) with rewiring 0.05–0.2 to encourage robustness.

38.3 Features

Degree d_v ; Laplacian positional encodings (top $K=8\text{--}16$ eigenvectors) with deterministic sign-fixing; grid coordinates ($x/L, y/L$); hypercube bitstrings (and Hamming weight). Edge weights default to 1.

38.4 Tasks and Horizons

Episodes uniformly sample among:

- **State transfer:** target j^* at large graph distance; $T \in \{64, 96, 128\}$.
- **Search:** single or few marks on grids/cubes; T chosen near $\tilde{O}(\sqrt{|V|})$ ranges.
- **Fast mixing:** minimize TV to uniform at horizon $T \in \{64, 96, 128\}$.
- **Robust transport:** as transfer, with disorder: static edge phases $\phi_e \sim \mathcal{U}[-\sigma, \sigma]$ ($\sigma \in \{0.05, 0.1, 0.2\}$) and/or coin dephasing $p \in \{0.01, 0.02\}$.

38.5 Splits and Generalization Protocol

Meta-train: lines $N = 64..192$, grids $L = 32..80$, cubes $n = 6..9$ (+ light irregulars). **Meta-val:** held-out sizes within those bands. **Meta-test (OOD):** larger sizes (line $N = 256$, grid $L = 96$, cube $n = 10$) and unseen irregulars. This probes zero-shot scaling.

38.6 Curriculum

Phase A: lines & grids, $T = 64$, no noise. Phase B: add search (grid/cube), $T \in \{64, 96\}$. Phase C: add robustness & OOD sizes, $T \in \{96, 128\}$.

38.7 Reproducibility via Deterministic Seeding

Each episode is specified by a tuple

(split, family, size, task, T , noise, init, episode_id),

which is hashed to a 64-bit seed. All stochastic choices (graph generation, LapPE, targets, noise, ψ_0) use only this seed, yielding exact regeneration without storing raw samples. Validation/test sets are *lists of tuples/seeds*.

38.8 Batching and Performance

For many small graphs, we concatenate into block-diagonal batches; for single large graphs, we batch multiple episodes at the Python level (8–16 per step). Heavy components (topology, LapPE) are memoized via an LRU cache keyed by (family, size) and (edge-hash, K), respectively. Use complex128 for ψ , float32 for GNN; clip gradients; enable DropEdge/feature dropout; fix port ordering (gauge choice).

38.9 Baselines (Shipped as Procedures)

We provide reference procedures that consume the same episode spec: fixed Hadamard/Grover coins, a global time-varying coin (MLP in t/T), and analytical SKW/AKR settings for search. These return the same metrics (success/mixing/robustness).

38.10 Scale

Initial runs: 5k–10k episodes; final tables: 50k–100k+ with early stopping. Because samples are synthetic, scale is compute-bound rather than data-bound.

38.11 On-the-Fly Generator: Design Blueprint

Deterministic episode IDs $\rightarrow$ seeds. Hash the episode tuple (plus a master seed) to a 64-bit value; create a local RNG per episode.

Pure graph samplers. Return (edge_index, deg, coords/bitstrings, port_map); resample until connected if desired.

Feature builders. Degree + LapPE (Lanczos; per-eigenvector sign fixing) + coords/bitstrings.

Task samplers. Deterministically choose targets/marks/ T from the episode RNG.

Initial state. Localized node/chirality (lines) or simple superpositions; deterministic from RNG.

Noise. Static: sample once per episode; dynamic: pre-sample per-step arrays from sub-seeds (e.g., $seed \oplus t$).

Caching. LRU for (family, size) graph topology and for LapPE.

Leak-free splits. Reserve size bands and/or route base seeds to {train, val, test} via a hash rule.

38.12 Implementation Skeleton (Deterministic and Minimal)

Listing 3: On-the-fly episode generator with deterministic seeding

```
1 from dataclasses import dataclass
2 import numpy as np
3
4 @dataclass(frozen=True)
5 class EpisodeCfg:
6     split: str          # "train" / "val" / "test"
```

```

7     family: str          # "line" / "grid" / "cube" / "dreg" / "er" / "ws"
8     size: dict          # {"N":128} or {"L":64} or {"n":8}
9     task: str           # "transfer" / "search" / "mixing" / "robust"
10    T: int
11    noise: dict          # {"type":"edge_phase","sigma":0.1} etc.
12    init: dict           # {"type":"localized","node":"auto","chirality":"R"}
13    episode_id: int      # unique counter (epoch*1e6 + iter, etc.)
14
15    def cfg_seed(cfg: EpisodeCfg, master_seed: int=0) -> int:
16        import hashlib, struct
17        s = str(cfg).encode()
18        # keyed hash -> 64-bit deterministic seed
19        h = hashlib.blake2b(s, digest_size=8, key=master_seed.to_bytes(8,'little')).digest()
20        return struct.unpack("<Q", h)[0]
21
22    def rng_from_seed(seed: int):
23        return np.random.default_rng(seed)
24
25    # ----- Graph builders (pure) -----
26    def build_line(N, rng):
27        # nodes: 0..N-1; undirected -> add both directions
28        src = np.concatenate([np.arange(N-1), np.arange(1, N)])
29        dst = np.concatenate([np.arange(1, N), np.arange(N-1)])
30        edge_index = np.stack([np.concatenate([src, dst]), np.concatenate([dst, src])], axis
31                               =0)
32        deg = np.bincount(edge_index[0], minlength=N) # out-degree
33        coords = (np.arange(N) / max(1, N-1)).reshape(N,1) # x/L
34        port_map = precompute_port_map_line(N) # flip-flop indexing for S
35        return {"edge_index": edge_index, "deg": deg, "coords": coords, "port_map": port_map}
36
37    def build_grid(L, rng):
38        # 4-neighborhood on LxL lattice; coords in [0,1]^2
39        # ... construct edge_index, deg, coords=(x/L,y/L), port_map ...
40        ...
41
42    def build_cube(n, rng):
43        # hypercube: nodes are bitstrings of length n; edges at Hamming distance 1
44        # ... construct edge_index, deg, bitstrings (as features), port_map ...
45        ...
46
47    # ----- Features (deterministic given graph) -----
48    def lap_pe(edge_index, num_nodes, k, rng):
49        # compute top-k eigenvectors of normalized Laplacian via Lanczos/power-iter
50        # fix sign of each eigenvector by making the largest-magnitude entry positive
51        ...
52
53    def make_features(family, graph, pe_k, rng):
54        deg = graph["deg"][:, None].astype(np.float32)
55        feats = [deg]
56        if family == "grid":
57            feats.append(graph["coords"].astype(np.float32))
58        if family == "cube":
59            feats.append(graph["bitstrings"].astype(np.float32))
60        pe = lap_pe(graph["edge_index"], len(deg), pe_k, rng) # for irregular graphs
61        if pe is not None: feats.append(pe.astype(np.float32))
62        X = np.concatenate(feats, axis=1)
63        return X
64
65    # ----- Task sampler (deterministic) -----
66    def sample_task(family, graph, rng):
67        # choose among {"transfer","search","mixing","robust"} with configured probs
68        # sample targets/marks/horizon and noise law; ensure separation for |M|=2
69        ...

```

```

69
70 def init_state(init_cfg, graph, rng):
71     # e.g., localized at node 0 or far end; pick chirality for d=2
72     ...
73
74 def make_episode(cfg: EpisodeCfg, master_seed=0):
75     seed = cfg_seed(cfg, master_seed)
76     rng = rng_from_seed(seed)
77
78     # 1) Graph (pure)
79     if cfg.family == "line": graph = build_line(cfg.size["N"], rng)
80     elif cfg.family == "grid": graph = build_grid(cfg.size["L"], rng)
81     elif cfg.family == "cube": graph = build_cube(cfg.size["n"], rng)
82     else: graph = build_other(cfg, rng)
83
84     # 2) Features (deterministic)
85     X = make_features(cfg.family, graph, pe_k=16, rng=rng)
86
87     # 3) Task & noise & init
88     task_spec = cfg.task if cfg.task != "auto" else sample_task(cfg.family, graph, rng)
89     noise = cfg.noise
90     psi0 = init_state(cfg.init, graph, rng)
91
92     return {"G": graph, "X": X, "T": cfg.T, "task": task_spec,
93           "noise": noise, "psi0": psi0, "seed": seed}

```

38.13 Validation/Test Regeneration and Manifests

We store only the *episode tuples* (or their 64-bit seeds) for validation/test. A *dataset manifest* (YAML/JSON) records families, size ranges per split, PE method and K , task proportions, horizon set, noise distributions, curriculum schedule, and master seed. Re-running `make_episode(cfg)` exactly reproduces the episodes.

38.14 Common Pitfalls and Remedies

Leakage. Prevent the same (family, size, seed) appearing in train and val; reserve size bands or use hash-based split routing.

PE sign ambiguity. Fix eigenvector signs deterministically.

Slow eigen solves. Cache LapPE per size; use Lanczos (top- K only; $K = 8\text{--}16$).

Temporal RNG drift. For dynamic noise, derive sub-seeds per time step (e.g., $\text{seed} \oplus t$) or pre-sample arrays.

Numerics. Use complex128 for ψ , float32 for GNN; gradient clip; DropEdge and feature dropout in the encoder.

Summary. Our dataset is a *procedural task suite*: episodes are generated on demand from deterministic seeds, with graph families in §38.2, features in §38.3, tasks in §38.4, and leak-free splits in §38.5. The code skeleton in §38.12 provides a minimal, reproducible implementation ready to plug into the ACPL training loop.

39 API & Pseudocode

Core Classes (Python-like)

Listing 4: Minimal API with a GNN policy

```

1 @dataclass
2 class GraphSpec:
3     name: str
4     n: int
5     degree: int
6     orient_map: np.ndarray # [n, degree] -> neighbor index
7     back_index: np.ndarray # [n, degree] -> return-arc index
8

```

```

9 class DTQWSim:
10     def __init__(self, graph: GraphSpec, T: int, backend="torch", dtype=torch.cdouble):
11         ...
12     def step(self, psi, C_blocks):
13         # returns  $S @ \text{block\_diag}(C\_blocks) @ \psi$ 
14         ...
15     def run(self, psi0, coin_schedule): # coin_schedule[v, t] -> params
16         ...
17
18 class CoinParam:
19     def __init__(self, kind: str): # "su2", "exp", "cayley"
20         ...
21     def to_unitary(self, theta): # vector -> unitary
22
23 class GNNCoinPolicy(nn.Module):
24     def __init__(self, d_coin=2, hidden=128, L=3, param_kind="su2"):
25         super().__init__()
26         self.gnn = GNN(layers=L, hidden=hidden) # GCN/GAT/MPNN
27         self.temporal = GRU(input_size=hidden+1+PEdim, hidden_size=hidden)
28         p = (3 if param_kind=="su2" else d_coin*d_coin)
29         self.head = nn.Linear(hidden, p)
30         self.param_kind = param_kind
31     def forward(self, G, node_feats, pos_enc, tfrac, h_t):
32         z = self.gnn(G, node_feats) # [V, hidden]
33         s = torch.cat([z, pos_enc, tfrac.expand_as(z[:, :1])], dim=1)
34         y, h_next = self.temporal(s.unsqueeze(0), h_t)
35         params = self.head(y.squeeze(0)) # [V, p]
36         return params, h_next
37
38 class BackpropTrainer:
39     def fit(self, policy, sim, task, psi0, epochs): ...
40
41 class RLTrainer: # e.g., PPO over actions = coin parameters
42     def fit(self, policy, sim_env, task_reward, steps): ...

```

Training Loops (Sketch)

Listing 5: Backprop vs. PPO training

```

1 # Backprop (differentiable tasks)
2 for epoch in range(E):
3     psi, h = psi0.clone(), torch.zeros(1,1,H) # GRU state
4     for t in range(T):
5         params_t, h = policy(G, node_feats, pos_enc, torch.tensor([t/T]), h)
6         C_blocks = coin_param.to_unitary(params_t) # unitary per node
7         psi = sim.step(psi, C_blocks)
8         rho_T = partial_trace_coin(psi)
9         loss = task.loss(psi, rho_T, extras)
10        loss.backward(); opt.step(); opt.zero_grad()
11
12 # PPO (non-diff tasks/oracles)
13 for rollout in range(R):
14     traj, psi, h = [], psi0.clone(), torch.zeros(1,1,H)
15     for t in range(T):
16         action, logp, h = policy.sample(G, node_feats, pos_enc, torch.tensor([t/T]), h)
17         psi = sim.step(psi, coin_param.to_unitary(action))
18         traj.append((obs_t, action, logp))
19     R_T = task.reward(psi) # terminal
20     ppo.update(traj, R_T)

```

40 Practical Considerations

Stability. Prefer $C = \exp(K)$ with small norms; gradient clipping; curriculum in T . **Efficiency.** Sparse S ; batched unitaries; cap T (e.g., 50–200). **Regularization.** Penalize rapid temporal variation. **Initialization.** Warm-start from Hadamard/Grover coins [3]. **Complexity.** Per step: $O(|E|H)$ for GNN message passing + $O(|V|d^2)$ for coins; total $O(T(|E|H + |V|d^2))$.

41 Novelty & Publishability

The literature includes coin optimization and learned coins (QWNN) [21] and many quantum-control ML successes [22, 24, 23], but not a unified, *GNN-based adaptive policy* that generalizes across graphs/tasks with explicit robustness and mixing targets. ACPL fills this gap, with clear baselines and ablations.

42 Related Work (Pointers)

Surveys and foundations [1, 2]; SU(2) coin parameterization [3]; search algorithms [4, 5, 6, 7]; mixing [8, 9, 10, 11]; time/position-dependent coins and circuit realizations [12, 15, 13, 16, 14]; disorder/localization [17, 18, 19]; ML/differentiable programming/manifolds [21, 25, 26, 27, 28, 29]; GNN foundations and PEs [30, 31, 32, 33].

43 Experimental Protocols (Standard Operating Procedures)

This section specifies *pre-registered*, reproducible procedures for generating data (episodes), training, evaluation, and reporting. All randomness is controlled by deterministic seeds; validation/test manifests are fixed prior to any training.

43.1 Experiment Manifest (Pre-Registration)

Each experiment is declared by a YAML manifest committed before training. It fixes:

1. **Goal & task:** `transfer` | `search` | `mixing` | `robust`.
2. **Graph family & sizes:** e.g., lines $N \in [64, 192]$ for train; $N = 256$ for OOD test.
3. **Horizons:** $T \in \{64, 96, 128\}$ (curriculum permitted).
4. **Policy:** GNN kind (GCN/GAT/GIN/PNA), controller (GRU/Transformer), coin map (SU2 / exp / Cayley), depths/widths.
5. **Features:** degree, positional encodings (LapPE with K), grid coordinates or hypercube bitstrings.
6. **Optimization:** LR, weight decay, batch size (#episodes), epochs, gradient clip, regularizers (temporal smoothness, magnitude).
7. **Randomness:** master seed, split routing rule, #seeds S for repeated training.
8. **Baselines:** Fixed Hadamard/Grover, global time-varying coin, SKW/AKR (when applicable).
9. **Success criteria:** pre-registered thresholds per task (see §31.1).
10. **Compute budget:** device, dtype, wallclock cap, memory cap.

43.2 Dataset Protocol (Episode Generation & Splits)

Deterministic generation. Each episode is produced by a pure function $(\text{cfg}, \text{master_seed}) \mapsto (G, X, T, \psi_0, \text{task}, \text{noise}, \text{seed})$; validation/test episodes are stored as a *list of tuples or 64-bit seeds* and never regenerated differently.

Leak-free splits. We enforce either *size bands* or a *hash-based router*:

- Train: lines $N \in [64, 192]$, grids $L \in [32, 80]$, cubes $n \in \{6, 7, 8, 9\}$.
- Validation: held-out sizes within band (e.g., particular N, L excluded from train).
- Test (OOD): larger sizes (line $N = 256$, grid $L = 96$, cube $n = 10$) and unseen irregulars.

Curriculum in T . Phase A: $T = 64$, no noise; Phase B: add search, $T \in \{64, 96\}$; Phase C: add robustness and OOD, $T \in \{96, 128\}$.

43.3 Training Protocol (Differentiable Tasks; Backprop)

For each iteration:

1. Sample a mini-batch of episodes (on-the-fly, train split).
2. For each episode, run the ACPL loop for $t = 0, \dots, T - 1$:

$$\theta_{\cdot,t} = \pi_\omega(G, X, t/T), \quad C_t = \bigoplus_v \Phi(\theta_{v,t}), \quad \psi_{t+1} = S C_t \psi_t.$$

3. Readout P_T from $\rho_T = \text{Tr}_C(|\psi_T\rangle\langle\psi_T|)$ and compute the task loss $\mathcal{L}$ plus regularizers (temporal smoothness, magnitude).
4. Backpropagate, clip gradients, optimizer step, log metrics.

Default hyperparameters. Adam (lr 2.5×10^{-4} , wd 10^{-5}), gradient clip 1.0, GNN depth $L \in \{2, 3\}$ with hidden $H \in \{64, 128\}$, Graph/LayerNorm, DropEdge 0.1, feature dropout 0.1, GRU(128), complex128 for ψ , float32 for GNN. SU2 coins for $d = 2$; $\exp(K)$ or Cayley for $d > 2$ with $q \in \{d^2, \lfloor d^2/2 \rfloor\}$.

Stoppers. Early stop if (i) no validation CI improvement for P evaluations, (ii) train–val gap exceeds tolerance, or (iii) improvement CI over best baseline crosses zero.

43.4 RL Protocol (Non-Differentiable Oracles)

When gradients are unavailable:

- Stochastic policy over $\theta_{v,t}$ (Gaussian; squash to valid ranges); environment step: apply $C = \Phi(\theta)$ and DTQW.
- Reward: terminal $r_T = f(\psi_T)$; optional shaping (e.g., sparsity or proximity).
- PPO: clip 0.2, GAE $\lambda = 0.95$, $\gamma = 1$, 4 epochs/rollout, minibatch 8–16, entropy bonus 0.01.

43.5 Evaluation Protocol (“Knowing It Is Right”)

Seeds and repeats. Evaluate each model/baseline on the fixed validation manifest across S random initializations; report mean $\pm$ stderr and 95% CI.

Primary metrics.

- **Transfer:** $P_T(j^*)$; curves vs. distance and vs. T .
- **Search:** $\sum_{m \in M} P_T(m)$; compare to SKW/AKR at matched T .
- **Mixing:** $\text{TV}(\rho_T, \rho^*)$; plot TV vs. t .
- **Robust:** $\mathbb{E}_\xi[P_T(j^*; \xi)]$ plus quantiles and CVaR_α .

Significance rule. Declare a win if the 95% CI of improvement over the *best baseline* exceeds a pre-registered margin ϵ (task-specific; see §31.1).

Ablations (causality checks). NoPE (remove positional encodings), GlobalCoin ($\theta_{v,t} \equiv \theta_t$), TimeFrozen (static coins), PermuteNodes (equivariance sanity).

Physics sanity. Check unitarity $\|\psi_{t+1}\|_2 - \|\psi_t\|_2 \approx 0$, $S^\dagger S = I$, and gauge invariance (port rephasing leaves metrics unchanged).

43.6 Hyperparameter Search Protocol

We perform budgeted sweeps (ASHA/Successive Halving) over at most 3–5 axes, e.g.,

$$L \in \{2, 3, 4\}, \quad H \in \{64, 128\}, \quad \text{DropEdge} \in \{0, 0.1\}, \quad K_{\text{PE}} \in \{8, 16\}, \quad \lambda_{\text{smooth}} \in \{0, 10^{-4}, 10^{-3}\}.$$

Promote top- k to full training; freeze best config for test.

43.7 Reporting Protocol (Figures & Tables)

Tables. Mean $\pm$ stderr over S seeds; boldface indicates CI-improvement $\geq \epsilon$.

Plots. Transfer: $P_T(j^*)$ vs. distance and vs. T ; Search: success vs. T versus SKW/AKR; Mixing: TV vs. t ; Robust: mean/quantile/CVaR $_\alpha$ vs. disorder level.

Coin visualization. Heatmaps of $\theta_{v,t}$ or spectra of $C_v(t)$ over (v, t) ; spatial overlays on graph layouts.

43.8 Reproducibility & Artifacts

Save for each run: full config, model weights, optimizer state, val/test episode lists (tuples/seeds), exemplar coin schedules $\theta_{v,t}$ (and optionally $C_v(t)$), logs (TensorBoard/W&B), and RNG states. Provide a one-command script to regenerate all figures/tables from a checkpoint.

43.9 Quality Gates (Unit Tests & CI)

All tests must pass before training:

1. **Shift unitarity:** $S^\dagger S = I$; S is a permutation unitary.
2. **Coin lifts:** $\text{SU}(2)$ $\det=1$; exp and Cayley produce unitary matrices.
3. **Equivariance:** message passing is permutation-equivariant under node relabelings.
4. **Partial trace:** position marginals from ρ_T match analytic reshape.
5. **Determinism:** identical episodes from identical (cfg, seed).

43.10 Per-Task Recipe Cards

State transfer (line). GIN + GRU + SU2; T proportional to source–target distance; metric $P_T(j^*)$ (target ≥ 0.90 on lines); ablations: GlobalCoin, TimeFrozen.

Search (grid/hypercube). GAT or PNA; $T \approx c\sqrt{|V|}$ (tune c on validation); metric $\sum_m P_T(m)$; compare with SKW/AKR.

Fast mixing. GCN/GIN with stronger smoothness regularization; metric TV to uniform; success if $\text{TV} \leq 0.05$ by $T = 128$.

Robust transport. exp(K) or Cayley; smoothness regularization; disorder $\sigma \in \{0.05, 0.1, 0.2\}$; metric mean success and $\text{CVaR}_{0.1}$; Monte Carlo M chosen so Hoeffding error ≤ 0.01 absolute.

43.11 Runbook (Canonical Commands)

```
1 # train (backprop)
2 python scripts/train.py task=transfer model.gnn.kind=gin model.coin.kind=su2 \
3   data.families=[line] optim.epochs=120 log.run_name=gin-gru-su2-line
4
5 # evaluate (val + test) with CIs & ablations
6 python scripts/eval.py exp=gin-gru-su2-line checkpoint=.../last.ckpt seeds=5
7
8 # sweep (ASHA via W&B or Ray)
9 python scripts/train.py -m model.gnn.kind=gin,gat,pna model.gnn.layers=2,3 \
10  model.gnn.hidden=64,128 optim.lr=1e-3,2.5e-4
```

Summary. These protocols ensure that results are *pre-registered*, *leak-free*, *statistically sound*, and *reproducible*, and that observed gains can be attributed to the *structure-aware*, *time-conditioned* policy rather than artifacts.

References

- [1] S. E. Venegas-Andraca, “Quantum walks: a comprehensive review,” *Quantum Information Processing* 11, 1015–1106 (2012).
- [2] R. Portugal, *Quantum Walks and Search Algorithms*, 2nd ed., Springer (2018).
- [3] C. M. Chandrashekar, R. Srikanth, R. Laflamme, “Optimizing the discrete time quantum walk using a SU(2) coin,” *Phys. Rev. A* 77, 032326 (2008).
- [4] N. Shenvi, J. Kempe, K. B. Whaley, “Quantum random-walk search algorithm,” *Phys. Rev. A* 67, 052307 (2003).
- [5] A. Ambainis, J. Kempe, A. Rivosh, “Coins Make Quantum Walks Faster,” arXiv:quant-ph/0402107 (2004).
- [6] A. Ambainis et al., “Search by quantum walks on two-dimensional grid without amplitude amplification,” arXiv:1112.3337 (2011).
- [7] A. Ambainis, “Search by Quantum Walks on Two-Dimensional Grid,” in *SOFSEM 2012*, LNCS 7147 (2012).
- [8] S. Chakraborty, K. Luh, J. Roland, “How Fast Do Quantum Walks Mix?,” *Phys. Rev. Lett.* 124, 050501 (2020).
- [9] S. Chakraborty, K. Luh, J. Roland, “How fast do quantum walks mix?,” arXiv:2001.06305 (2020).
- [10] C. Moore, A. Russell, “Quantum Walks on the Hypercube,” *RANDOM 2002*, LNCS 2483 (2002); also arXiv:quant-ph/0104137.
- [11] F. L. Marquezino, R. Portugal, G. Abal, R. Donangelo, “Mixing times in quantum walks on the hypercube,” *Phys. Rev. A* 77, 042312 (2008).
- [12] U. Nzogani, M. Skotiniotis, A. Monras, “Quantum circuits for discrete-time quantum walks with position-dependent coin operator,” arXiv:2211.05271 (2022); journal version (2023): *Quantum Information Processing*.
- [13] U. Nzogani et al., “Quantum circuits for discrete-time quantum walks with position-dependent coin operator,” *Quantum Information Processing* 22, 179 (2023).
- [14] S. Frydrych, P. Kok, “Adjustable-depth quantum circuit for position-dependent coin operators,” *Quantum Information Processing* 23, 144 (2024).
- [15] M. Montero, J. A. Méndez-Bermúdez, “Invariance in quantum walks with time-dependent coin operators,” arXiv:1410.0550 (2014).
- [16] R. Ahmad, U. Sajjad, M. Sajid, “One-Dimensional Quantum Walks with a Position-Dependent Coin,” arXiv:1902.10988 (2019).
- [17] Y. Yin, D. E. Katsanos, S. N. Evangelou, “Quantum walks on a random environment,” *Phys. Rev. A* 77, 022302 (2008).
- [18] A. Schreiber et al., “Decoherence and Disorder in Quantum Walks: From Ballistic Spread to Localization,” *Phys. Rev. Lett.* 106, 180403 (2011).
- [19] S. Derevyanko, “Anderson localization of a one-dimensional quantum walker,” *Sci. Rep.* 8, 1795 (2018).

- [20] H. Krovi, T. A. Brun, “Hitting time for quantum walks on the hypercube,” *Phys. Rev. A* 73, 032341 (2006).
- [21] S. Dernbach, D. A. Hudson, M. Kim, A. Manchon, D. Towsley, H. Xiong, “Quantum walk neural networks with feature dependent coins,” *Applied Network Science* 4, 117 (2019).
- [22] V. V. Sivak et al., “Model-Free Quantum Control with Reinforcement Learning,” *Phys. Rev. X* 12, 011059 (2022).
- [23] X. M. Zhang et al., “When does reinforcement learning stand out in quantum control?,” *npj Quantum Information* 5, 85 (2019).
- [24] M. M. Wauters et al., “Reinforcement-learning-assisted quantum optimization,” *Phys. Rev. Research* 2, 033446 (2020).
- [25] PennyLane Team, “Quantum Differentiable Programming” (glossary/doc).
- [26] O. Di Matteo et al., “Quantum Computing with Differentiable Quantum Transforms,” *ACM Computing Surveys* 55, 12 (2023).
- [27] O. Di Matteo, G. Long, M. Valenti, “Automatic differentiation for quantum computing,” *Phys. Rev. A* 106, 052429 (2022).
- [28] J. Li, L. Fuxin, S. Todorovic, “Efficient Riemannian Optimization on the Stiefel Manifold via the Cayley Transform,” *ICLR* (2020).
- [29] P.-A. Absil, R. Mahony, R. Sepulchre, *Optimization Algorithms on Matrix Manifolds*, Princeton University Press (2008).
- [30] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, G. E. Dahl, “Neural Message Passing for Quantum Chemistry,” *ICML* (2017).
- [31] T. N. Kipf, M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” *ICLR* (2017).
- [32] P. Veličković et al., “Graph Attention Networks,” *ICLR* (2018).
- [33] V. P. Dwivedi et al., “Benchmarking Graph Neural Networks,” *arXiv:2003.00982* (2020).
- [34] K. Xu, W. Hu, J. Leskovec, S. Jegelka, “How Powerful Are Graph Neural Networks?,” *ICLR* (2019).
- [35] H. Maron, H. Ben-Hamu, N. Shamir, Y. Lipman, “Invariant and Equivariant Graph Networks,” *ICLR* (2019).
- [36] W. L. Hamilton, R. Ying, J. Leskovec, “Inductive Representation Learning on Large Graphs,” *NeurIPS* (2017).
- [37] K. Xu, J. Li, M. Zhang, S. S. Du, K.-I. Goh, J. Leskovec, S. Jegelka, “Representation Learning on Graphs with Jumping Knowledge Networks,” *ICML* (2018).
- [38] K. Xu, W. Hu, J. Leskovec, S. Jegelka, “How Powerful Are Graph Neural Networks?,” *ICLR* (2019).
- [39] G. Corso, L. Cavalleri, D. Beaini, P. Liò, P. Veličković, “Principal Neighbourhood Aggregation for Graph Nets,” *ICLR* (2020).
- [40] J. Klicpera, A. Bojchevski, S. Günnemann, “Predict then Propagate: Graph Neural Networks meet Personalized PageRank,” *ICLR* (2019).
- [41] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, C.-J. Hsieh, “Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks,” *KDD* (2019).
- [42] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, V. Prasanna, “GraphSAINT: Graph Sampling Based Inductive Learning Method,” *ICLR* (2020).
- [43] Y. Rong, W. Huang, T. Xu, J. Huang, “DropEdge: Towards Deep Graph Convolutional Networks on Node Classification,” *ICLR* (2020).
- [44] L. Zhao, L. Akoglu, “PairNorm: Tackling Oversmoothing in GNNs,” *NeurIPS* (2019).
- [45] C. Cai, W. Wang, Y. Wang, V. Li, “GraphNorm: A Principled Approach to Accelerating Graph Neural Network Training,” *ICML* (2021).
- [46] E. Rossi, B. Frasca, B. Chamberlain, D. Eynard, F. Monti, M. Bronstein, “Temporal Graph Networks for Deep Learning on Dynamic Graphs,” *ICML Workshop* (2020).
- [47] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, K. Achan, “Inductive Representation Learning on Temporal Graphs,” *ICLR* (2020).

- [48] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, H. Harutyunyan, G. Ver Steeg, A. Galstyan, “MixHop: Higher-Order Graph Convolutional Architectures via Sparsified Neighborhood Mixing,” *ICML* (2019).
- [49] A. Nayak, A. Vishwanath, “Quantum Walk on the Line,” arXiv:quant-ph/0010117 (2000).
- [50] N. Konno, “Quantum random walks in one dimension,” *Quantum Information Processing* 1, 345–354 (2002).
- [51] A. M. Childs, J. Goldstone, “Spatial search by quantum walk,” *Phys. Rev. A* 70, 022314 (2004).
- [52] N. J. Higham, *Functions of Matrices: Theory and Computation*, SIAM (2008).
- [53] U. Alon, E. Yahav, “On the Bottleneck of Graph Neural Networks and its Practical Implications,” *NeurIPS* (2021).
- [54] J. Topping, B. Di Giovanni, B. Chamberlain, X. Dong, M. Bronstein, “Understanding Over-Squashing and Bottlenecks on Graphs via Curvature,” *ICLR* (2022).
- [55] V. P. Dwivedi, X. B. Rampašek, M. Goyal, M. Beaini, P. Lio, Y. Bresson, “A Generalization of Transformers to Graphs,” *AAAI* (2021).